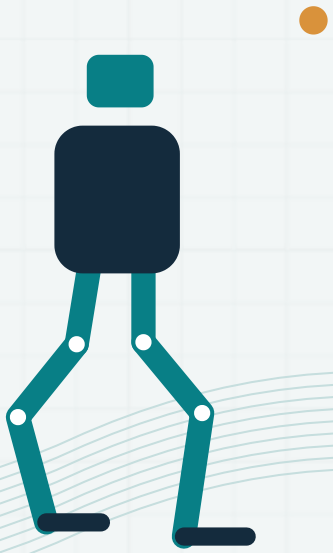
LEARN · IMPLEMENT · EXPERIMENT

强化学习实战

从 PPO 到双足机器人

理解策略、价值、课程与泛化



从一条轨迹，到一次可信的实验

公式推导 • PyTorch 实现 • 机器人控制

PREFACE

前言

强化学习研究这样一类问题：智能体没有现成的动作标签，只能在与环境的交互中观察后果，并从奖励中学习如何行动。对双足机器人来说，一个关节动作既会改变眼前的姿态，也可能影响几步之后的平衡。要理解它怎样学会前进，需要把概率模型、长期回报、优化目标和物理环境联系起来。

本书以 EricBot 双足机器人项目为贯穿案例，沿着“工具与概念、学习算法、机器人环境、实验与泛化”四个层次展开。读者将从 PyTorch 的张量与自动求导出发，逐步掌握策略梯度、价值估计、广义优势估计和 PPO，再深入观测设计、奖励工程、课程学习与训练诊断。

本书面向已经了解基础神经网络、损失函数、梯度与反向传播，但尚未系统学习强化学习的读者。阅读时不必先背熟所有公式，但应愿意手算短轨迹、运行最小程序，并把实验条件和结果保存下来。数学推导解释机制，代码展示实现，实践与验收标准帮助读者判断是否真正掌握。

学习的目标不只是让训练运行起来，还包括提出清楚的问题：奖励是否表达了任务意图？策略是在利用观测，还是记住了地形？成功率的提升能否在留出条件下重现？一次修改带来了收益，也付出了哪些代价？

建议读者保持一种工作习惯：先写出假设，再修改参数；先建立对照，再解释曲线；看到成功的动作之后，继续检查失败案例。能够从证据中判断一个方案为什么有效、又在哪里失效，是独立开展强化学习实验的重要起点。

CONTENTS

目录

前言 2
学习路线 4
如何使用本书 5
第 00 章：认识项目与实验方法 6
第 01 章：PyTorch 基础 14
第 02 章：概率分布与随机策略 22
第 03 章：MDP 与强化学习交互 28
第 04 章：奖励、回报与信用分配 34
第 05 章：Policy Gradient 40
第 06 章：Value 与 Actor-Critic 46
第 07 章：GAE 与批量轨迹 52
第 08 章：PPO 完整实现 58
第 09 章：双足机器人环境 65
第 10 章：观测与高程感知 72
第 11 章：奖励工程与局部最优 78
第 12 章：课程学习与复习机制 85
第 13 章：随机地形、泛化与过拟合 91
第 14 章：训练指标与故障诊断 97
第 15 章：Checkpoint、续训与迁移 104
结语：让证据引导下一次实验 110
综合实践 111
实验记录模板 113

LEARNING PATH

学习路线

01 **工具与问题** 第 00-04 章

张量、概率、环境交互与长期回报

阶段检查：能够说清观测、动作、奖励和任务目标。

02 **从梯度到 PPO** 第 05-08 章

策略梯度、价值估计、GAE 与裁剪更新

阶段检查：能够手算优势，并解释一次 PPO 更新。

03 **策略进入物理世界** 第 09-11 章

控制接口、观测设计与奖励工程

阶段检查：能够用控制、观测和奖励解释机器人行为。

04 **把训练变成实验** 第 12-15 章

课程、泛化、诊断与模型管理

阶段检查：能够提交有对照、有证据、有边界的实验报告。

先完成可以解释的小实验，再逐步扩大任务与训练规模。

READER'S GUIDE

如何使用本书

阅读准备

开始前应熟悉向量、均值与方差、概率密度、对数、链式法则和梯度。PyTorch 与概率模型将在第 01、02 章展开。机器人案例使用 78 维观测与 14 维动作，涉及 GPU 批量环境；阅读代码时，应始终关注张量形状、设备和数据流。

一章的四个步骤

1. **明确问题。** 先读核心问题与学习目标，用一句话写下本章要解决的困难。
2. **追踪数据。** 遇到公式和代码时，标出输入、输出、张量形状，以及哪些量参与求导。
3. **完成实验。** 先运行可以解释的最小例子，再进入机器人项目。把预期结果写在观察结果之前。
4. **独立验收。** 合上教材复述机制，对照验收标准检查薄弱环节，并保留能支持结论的证据。

建立实验档案

每次训练使用独立的运行名称，记录配置、代码版本、随机种子、训练预算和 checkpoint 位置。比较方案时保持评估条件一致，一次只改变一个主要因素。不要只根据总回报或一段动画判断步态质量，还要同时检查成功、摔倒、滑移、腾空与能耗。

建议用 10-12 周完成全书，每周学习 3-5 次，每次 1-2 小时。若某章的公式或实验仍无法解释，应优先补齐这一环节，再扩大训练规模。

PART I · 工具、交互与回报

第 00 章：认识项目与实验方法

> 核心问题：这个项目有哪些部分？一次训练到底产生了什么？

在真正开始学强化学习之前，你需要先建立一张"地图"：知道这个双足机器人项目由哪些模块组成、数据在模块之间怎样流动、一次训练会在磁盘上留下哪些东西。本章不涉及任何训练公式，目标是让你在动手跑第一条命令之前，就能"看懂"整个系统。

0.1 学习目标

完成本章后，你应当能够：

1. 用自己的话描述机器人、环境、策略网络、PPO 训练器、课程调度器之间的数据流。
2. 说明 Python 包、虚拟环境、配置文件、日志和 checkpoint 各自扮演的角色。
3. 解释随机种子、实验名称（`run-name`）和对照实验为什么重要。
4. 指出"训练代码、评估代码、可视化代码必须使用一致的观测定义"这条规则背后的原因。
5. 独立完成一次环境验证，并把"观测 → 策略 → 动作 → 物理环境 → 奖励与下一观测"画成流程图。

0.2 这个项目在做什么

用一句话概括：**在 GPU 上并行训练一个 14 自由度的双足机器人，从平地走到 5.5 米外的终点，并逐渐适应越来越起伏的地形和越来越强的随机侧向推力。**

拆开来看，它同时解决三件事：

维度	问题	本项目的做法
控制	让 14 个关节怎样协调，才能稳定前进？	策略网络输出关节目标，position actuator 执行
泛化	怎样保证不是"记住了这一条固定跑道"？	程序化随机地形 + 留出地形评估
课程	怎样由易到难，且不忘旧技能？	八级课程 + 跨等级复习

你可能会问：为什么不直接告诉机器人"左脚抬、右脚落、交替迈步"？这正是这门课的核心悬念。项目刻意**不把步态写进奖励**，而是只给前进、直立、能效、滑移、腾空这类通用物理约束，让交替步态作为"最优解"自己涌现。这个思想会贯穿后续所有章节。

0.3 系统的五大模块

打开源码目录，你会看到这些文件：

```
TEXT
src/water_cup_rl/
├── assets/biped_mujoco.xml   # 14-DoF 机器人与随机地形的物理描述
├── config.py                 # v23 环境、PPO、地形与推力课程的全部参数
├── terrain.py                # 多变体高度场与跑道映射
├── env.py                    # 78 维观测、奖励与 GPU 批量环境
├── model.py                  # PyTorch Actor/Critic 与观测标准化
├── train.py                  # PPO、课程、TensorBoard 和终端输出
├── evaluate.py               # GPU 并行确定性评估
├── robot_demo.py             # CPU MuJoCo 可视化回放
├── reward_audit.py           # 通用动作奖励审计
├── curriculum.py             # 课程调度
└── verify.py                 # CUDA、物理和模型验证
```

按"职责"可以把它们归成五类。请务必记住这张映射，因为后续每一章都会反复引用：

1. 物理世界（assets/biped_mujoco.xml + MuJoCo）

描述机器人长什么样、关节在哪里、地面是什么、重力是多少。MuJoCo 是物理引擎，负责求解"给一个力矩，机器人下一秒的位置和速度是多少"。

2. 环境（env.py）

把物理世界包装成强化学习标准接口：接收动作 → 推进物理 → 计算奖励 → 返回下一观测和是否结束。它还负责**观测**（observation）的组装和**奖励**（reward）的计算，是整个项目里逻辑最密集的文件（约 600 行）。

3. 策略与价值网络（model.py）

PyTorch 网络。`Actor` 负责"看到观测后决定动作"，`Critic` 负责"估计当前状态有多好"。这是唯一会反向传播、会更新参数的部分。

4. 训练器（train.py）

把环境、网络、优化器串起来：收集 rollout → 计算优势 → PPO 更新 → 打印指标 → 保存 checkpoint。它调用 `curriculum.py` 决定每个环境当前应该练哪个等级。

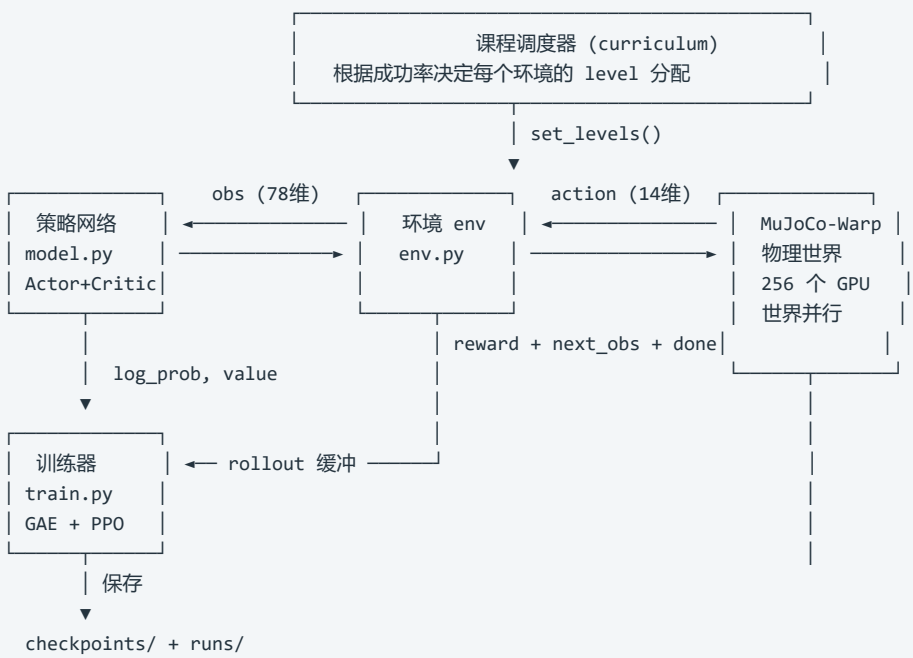
5. 课程调度器（curriculum.py）

维护"当前前沿等级"、统计窗口和晋级/降级逻辑。它不碰物理、不碰网络，只根据成功率决定"下一个等级练什么"。

0.4 数据流：一次训练产生了什么

这是本章最重要的一张图。请逐字理解，并能在纸上默画出来：

TEXT



用自然语言描述一轮：

1. **观测**：环境把机器人当前状态打包成 78 维向量 `obs`，交给策略网络。
2. **决策**：Actor 输出 14 维动作 `action`，Critic 同时输出该状态的价值 `value`。
3. **执行**：环境把动作映射为 14 个关节目标，交给 MuJoCo-Warp 在 256 个 GPU 世界里各推进 4 个物理子步。
4. **反馈**：环境返回 `reward`、`next_obs` 和 `done`（是否摔倒/成功/超时）。
5. **收集**：训练器把 `(obs, action, log_prob, value, reward, done)` 存进 rollout 缓冲。
6. **更新**：攒够一批后，训练器计算优势函数，做若干轮 PPO 更新。
7. **课程**：训练器把每个回合的成功/失败汇报给课程调度器，调度器可能晋级或降级。
8. **记录**：每隔若干步打印指标、写 TensorBoard、保存 checkpoint。

> 关键点：**`obs` 是环境与网络之间的"唯一语言"**。网络从不直接看到关节角度或世界坐标，它只看到一个 78 维向量。反过来，环境也完全不知道网络内部结构，它只知道网络吐出一个 14 维动作。这就是"接口"的意义——只要观测和动作的定义不变，网络可以随便换结构，环境可以随便换物理参数。

0.5 Python 包、虚拟环境与依赖

为什么需要虚拟环境

强化学习项目对版本极其敏感。本项目的 `requirements.txt` 把 `torch==2.11.0+cu128`、`mujoco==3.11.0`、`mujoco-warp==3.11.0`、`warp-lang==1.16.0` 钉死在特定版本，因为这些库需要与你的 CUDA 驱动精确匹配。项目自带的 `.venv` 目录就是一个独立的虚拟环境，避免污染系统 Python，也避免不同项目之间的版本冲突。

安装依赖：

```powershell
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

修改源码后刷新可编辑安装（不会重新下载依赖）：

```powershell
.\.venv\Scripts\python.exe -m pip install -e . --no-deps --no-build-isolation
```

验证环境

跑一遍验证脚本，它会检查 CUDA、GPU 物理、机器人维度和模型参数：

```powershell
.\.venv\Scripts\python.exe -m water_cup_rl.verify
```

正常输出应包含类似：

```text
GPU: NVIDIA GeForce RTX ...
Objective version: 23 (terrain + lateral pushes)
Actuated DoF: 14
Observation dimensions: 78
Model parameters: 1,139,229
MuJoCo-Warp GPU batched step: PASS
PyTorch CUDA model: PASS
```

如果这里失败，说明环境没装好，后续所有章节都无法进行——先解决它，不要跳过。

0.6 配置文件：把"魔法数字"集中到一处

打开 `config.py`，你会看到三个 dataclass：

类	作用	关键字段举例
`EnvConfig`	环境与任务参数	`physics_hz=240`、`control_hz=60`、`track_length_curriculum`、`terrain_amplitude_curriculum`、`push_force_curriculum_n`
`PPOConfig`	算法参数	`num_envs=256`、`rollout_steps=64`、`gamma=0.99`、`clip_coef=0.20`、`learning_rate=3e-4`
`CurriculumConfig`	课程参数	`window_episodes=1500`、`promote_success_rate=0.70`、`review_fraction=0.30`

为什么要把参数集中起来？ 因为强化学习训练高度依赖"公平对照"。如果超参数散落在代码各处，你改了 `gamma` 却忘了记录，一个月后回看结果时根本无法复现。集中配置 + 写入 JSON 日志，才能保证每次实验可追溯。

你会在 `train.py` 里看到这样一行，把完整配置写进运行目录：

```python
(run_dir / "config.json").write_text(
    json.dumps(config_dict(env_config, ppo, curriculum_config), indent=2), encoding="utf-8"
)
```

这份 `config.json` 就是该次训练的"出生证明"。

0.7 运行目录与 checkpoint：磁盘上的产物

一次训练会在两个地方留下东西：

```text
runs/terrain_biped_v23/                    TensorBoard 日志
checkpoints/terrain_biped_v23/latest.pt    最新检查点
checkpoints/terrain_biped_v23/update_*.pt 周期检查点
```

产物	内容	用途
`runs/<name>/`	事件文件（标量曲线、文本）、`config.json`	用 TensorBoard 看训练曲线
`checkpoints/<name>/latest.pt`	模型权重、优化器状态、全局步数、课程状态	续训、评估、可视化
`checkpoints/<name>/update_*.pt`	同上，按迭代序号保存的历史快照	回滚到某个中间状态、对比不同阶段

启动 TensorBoard：

```powershell
.\.venv\Scripts\python.exe -m tensorboard.main --logdir runs\terrain_biped_v23
```

`--run-name` 是实验的身份证。 每次实验都应该用一个新的名字（比如 `terrain_biped_v23_tryA`、`terrain_biped_v23_tryB`），否则新的日志会覆盖旧日志，checkpoint 也会互相覆盖，你再也找不回"上一版到底表现如何"。

0.8 随机种子与可复现性

训练里到处都是随机：地形生成、推力方向、动作采样、初始姿态扰动。如果这些随机量每次都不一样，你运行两次同一配置会得到两条不同的曲线，就无法判断"某个改动是否真的有效"。

随机种子（seed） 就是给随机数发生器一个固定起点，让它每次都生成同一串"伪随机"序列。项目默认 `seed=7`（见 `config.py` 和 `train.py` 的多处 `manual_seed`）。

```python
random.seed(args.seed)
np.random.seed(args.seed)
torch.manual_seed(args.seed)
torch.cuda.manual_seed_all(args.seed)
```

> 注意：固定 seed 只保证"同配置同结果"，不保证"结果与 seed 无关"。 一个真正稳健的结论，应当在多个 seed 下都成立。如果"改了一个奖励权重，效果变好"只在 seed=7 成立、在 seed=8 就反转，那这个"变好"大概率是噪声，不是真改进。这是第 13 章会反复强调的实验纪律。

0.9 对照实验：一次只改变一个变量

强化学习实验最大的陷阱是"同时改了三个东西，然后说有效果"。如果同时调大了奖励权重、降低了学习率、加了噪声，最后曲线变好了，你无法知道到底是哪一项起作用（甚至可能是三项互相抵消后的假象）。

这门课要求你养成下面的实验记录格式，后续每一章的"实践"都会用到它：

```text
1. 问题：想验证什么？
2. 假设：预计哪个指标会怎样变化？
3. 自变量：本次只修改什么？
4. 控制变量：seed、训练步数、环境数量等保持什么不变？
5. 结果：训练与留出地形分别怎样？
6. 证据：曲线、终端数据和可视化是否一致？
7. 结论：支持还是反驳假设？
8. 后续：下一次实验改变什么？
```

为什么"对照"这么重要？ 因为强化学习的训练曲线天然带噪声。两个随机运行的差异可能远大于一个超参数的微小调整。没有对照，你只是在"看运气"。

0.10 为什么三个入口必须共享同一份观测定义

项目里有三处代码会"拼观测"：

1. **训练**：`env.py` 的 `observation()`；
2. **评估**：`evaluate.py` 直接调用 `env.py` 的同一个环境（所以天然一致）；
3. **可视化**：`robot_demo.py` 的 `observation()`（CPU 版本，手动重写了一份）。

如果这三份代码对观测的拼接顺序、缩放因子、归一化方式有一丁点不一致，就会发生灾难：**训练时网络看到的是 A 格式，可视化时喂给它 B 格式，网络会输出完全错误的动作。** 你会看到"训练指标很好，但回放时机器人像喝醉了一样乱走"，而且极难排查——因为问题不在网络，而在"喂给网络的数据变了"。

这正是为什么 `robot_demo.py` 的 `observation()` 函数最后有一句断言：

```python
if result.shape != (CupBipedEnv.OBS_SIZE,):
    raise RuntimeError(f"Viewer observation has shape {result.shape}, expected {(CupBipedEnv.OBS_SIZ
E,)}")
```

它在用运行时检查，防止"维度对不上"这类低级错误静默通过。

> 记住一条铁律：**修改观测定义时，必须同步修改所有生产/消费这份观测的地方，否则就是给自己埋雷。** 这也是第 10 章和第 15 章（`--warm-start` 处理观测维度变化）的伏笔。

0.11 实践：验证环境并画出数据流图

任务 1：跑通验证

```powershell
.\.venv\Scripts\python.exe -m water_cup_rl.verify
```

记录输出中的 `Actuated DoF`、`Observation dimensions`、`Model parameters` 三个数字，与本章 0.5 节的示例对照。

任务 2：运行一次极短的可视化回放

先跑脚本演示（不需要 checkpoint，用内置的脚本化步态）：

```powershell
.\.venv\Scripts\python.exe -m water_cup_rl.robot_demo --seconds 30
```

观察：机器人是不是在"原地踏步式"地摆动？绿色横条是什么？这能帮你直观感受"物理世界 + 执行器"长什么样，即便还没有任何学习。

任务 3：手绘数据流图

不看书，在纸上画出 0.4 节的完整数据流图，并标注每一步传输的数据维度（78 维观测、14 维动作、标量奖励）。画不出来说明还没理解——回到 0.4 节再看一遍。

任务 4：定位产物

运行一次训练的前几十秒（可以随时 `Ctrl+C` 中断）：

```powershell
.\.venv\Scripts\python.exe -m water_cup_rl.train --total-steps 200000 --num-envs 256 --rollout-steps 64 --device cuda:0 --run-name test_ch00
```

中断后，回答以下问题：

1. TensorBoard 日志在哪个目录？
2. 有没有生成 checkpoint？在哪个目录？
3. `runs/test_ch00/config.json` 里能否找到 `gamma` 和 `num_envs` 的值？
4. 终端打印的那几行 `[ITER]`、`[TASK]`、`[MOVE]`… 分别代表什么？（先猜，第 14 章会详解）

0.12 验收标准

能全部答上来，才能进入第 1 章：

- [ ] 能否默画"观测 → 策略 → 动作 → 物理 → 奖励"的数据流图，并标注维度？
- [ ] 能否说出五类模块（物理、环境、网络、训练器、课程）各自的职责和对应文件？
- [ ] 能否指出：训练日志、checkpoint、环境参数、网络参数分别保存在什么位置？
- [ ] 能否解释：为什么训练、评估、可视化必须使用同一份观测定义？
- [ ] 能否说明：为什么"一次只改变一个变量"比"同时改多个"更可信？
- [ ] 能否说出 `--run-name` 和随机种子各自的用途？

0.13 常见误区

1. **"先跑起来再说，参数看不懂没关系"** —— 强化学习里参数就是实验本身。看不懂 `gamma`、`clip_coef` 就跑训练，等于闭着眼睛开车。后续第 4～8 章会逐个讲清楚。
2. **"训练指标高 = 机器人会走路了"** —— 训练回报高可能只是因为机器人学会了滑行、原地跳、甚至钻物理引擎的空子。第 11 章会专门讲"奖励漏洞"。
3. **"可视化好看就大功告成"** —— 一段 30 秒动画只能算"轶事证据"，不能算结论。必须配合批量评估数据（第 13、14 章）。
4. **"忘记记 run-name 和 seed"** —— 这是新手最常见、代价最大的失误。从现在起，每次训练先写实验记录再敲命令。

0.14 本章小结

- 这个项目 = 物理世界 + 环境 + 策略网络 + PPO 训练器 + 课程调度器，五个模块各司其职。
- 一次训练的本质是：**观测进、动作出、物理演化、奖励回流、批量更新、课程调度、落盘记录** 的循环。
- 观测是环境与网络的唯一接口，三处入口必须严格一致。
- 实验纪律（run-name、seed、对照、单变量）不是形式主义，而是判断"真改进 vs 噪声"的唯一手段。

下一步：进入第 01 章：PyTorch 基础，先掌握这门课的"语言工具"——Tensor、自动求导和 `nn.Module`，因为策略网络和 PPO 更新全是 PyTorch 代码。

PART I · 工具、交互与回报

第 01 章：PyTorch 基础

> 核心问题：PyTorch 除了自动求导，还为强化学习提供了什么？

你已经会写 NumPy 和基础的神经网络，但强化学习项目几乎全用 PyTorch 写，不只是因为它能自动求导。本章带你从"会用 NumPy 算矩阵"过渡到"能用 PyTorch 组织一个完整的可训练模块"，并理解为什么**批量（batch）与设备（device）**在 GPU 强化学习里是头等大事。

1.1 学习目标

完成本章后，你应当能够：

1. 解释 Tensor 的形状、dtype、device、广播与批处理。
2. 区分 CPU Tensor 与 CUDA Tensor，并能互相搬移。
3. 说明 `requires_grad`、计算图、`backward()`、`no_grad()` 的关系。
4. 用 `nn.Module`、参数、优化器和 `state_dict` 搭建并保存一个网络。
5. 解释为什么批量环境（256 个并行世界）远比"逐个环境循环"更适合 GPU。

1.2 从 NumPy 到 Tensor：换汤不换药

PyTorch 的 `Tensor` 与 NumPy 的 `ndarray` 在"多维数组"这个层面几乎是同构的。你已经熟悉的索引、切片、广播、`reshape`，在 Tensor 里原样可用：

```python
import torch

a = torch.tensor([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]])
print(a.shape)      # torch.Size([2, 3])
print(a.dtype)      # torch.float32
print(a[0])         # tensor([1., 2., 3.])
print(a[:, 1])      # tensor([2., 5.])
print(a.T)          # 转置
```

但 Tensor 比 ndarray 多了两个 NumPy 没有的维度——**设备（device）**和**自动求导（autograd）**。这两个多出来的能力，正是强化学习选择 PyTorch 的原因。

1.3 形状（shape）与批处理（batch）：RL 里最重要的心智模型

为什么"批量"无处不在

在监督学习里，你处理的是 **（样本数，特征数）** 的矩阵。在强化学习里，我们几乎从不一次只处理一个环境，而是一次处理 256 **个并行环境**（每个环境里有一个机器人在自己独立的世界里训练）。

于是所有数据都天然是"批量的"：

数据	形状	含义
观测 `obs`	`(256, 78)`	256 个环境，每个 78 维观测
动作 `action`	`(256, 14)`	256 个环境，每个 14 维动作
奖励 `reward`	`(256,)`	256 个环境，每个一个标量奖励
价值 `value`	`(256,)`	256 个环境，每个一个标量价值

再叠加上**时间维**（rollout 步数 64），就得到训练缓冲里的真实形状：

```python
# train.py 里分配的 rollout 缓冲
observations = torch.zeros((rollout_steps, num_envs, OBS_SIZE))  # (64, 256, 78)
actions      = torch.zeros((rollout_steps, num_envs, ACTION_SIZE))  # (64, 256, 14)
rewards      = torch.zeros((rollout_steps, num_envs))  # (64, 256)
```

> 请牢牢记住这个约定：**本项目的张量几乎总是 （时间，环境，特征） 或 （环境，特征） 的结构，第一维（或前两维）是"并行/批"维度，最后一维是"每个样本的内容"。** 很多人第一次读 RL 代码被形状绕晕，就是因为没先建立这个心智模型。

广播（broadcasting）

广播规则与 NumPy 完全一致：形状从右往左对齐，缺失的维度补 1，尺寸为 1 的维度会被"拉伸"。

```python
target_speed = torch.as_tensor([0.18, 0.20, ...])  # (8,) 每级一个参考速度
levels = torch.tensor([0, 7, 3])                   # (3,) 每个环境当前的等级
speed = target_speed[levels]                        # (3,) 用等级索引取出对应速度
```

项目里大量使用"按等级索引"这个技巧：`config.py` 里每个课程参数都是一个长度为 8 的元组，环境用当前 `levels` 张量做花式索引，一次性取出 256 个环境各自的参数。这是一种"查表式"的批量配置，比写 256 个 `if` 快得多、也优雅得多。

形状检查是调试的第一道防线

项目在关键处会显式断言形状，例如 `env.py`：

```python
if observation.shape != (self.num_envs, self.OBS_SIZE):
    raise RuntimeError(f"Observation shape mismatch: {tuple(observation.shape)}")
```

养成习惯：**每次写一个新的批量操作后，先 `print(x.shape)` 确认，再继续。** 形状错位是 RL 代码里最高频、也最隐蔽的 bug。

1.4 设备（device）：CPU 与 CUDA Tensor

设备是什么

`device` 表示这个 Tensor 的数据存在哪里：`'cpu'` 还是 `'cuda:0'`（第 0 块 GPU）。PyTorch 的运算要求**参与运算的所有 Tensor 必须在同一个 device 上**，否则会报错：

```python
x = torch.ones(3, device='cpu')
y = torch.ones(3, device='cuda:0')
# x + y 会抛出 RuntimeError: expected all tensors to be on the same device
```

搬移

```python
a = torch.ones(3)                  # 默认在 CPU
b = a.to('cuda:0')                 # 搬到 GPU
c = b.cpu()                        # 搬回 CPU
d = torch.ones(3, device='cuda:0') # 直接在 GPU 上创建
```

项目里的设备约定

项目要求 GPU（`env.py` 里直接拒绝非 CUDA）：

```python
if self.device.type != "cuda":
    raise ValueError("MuJoCo-Warp training requires a CUDA device")
```

原因很简单：MuJoCo-Warp 用 GPU 并行模拟 256 个物理世界，物理状态就存在 GPU 显存里。如果观测（从物理状态派生）在 GPU、而网络参数在 CPU，每次前向都要跨 PCIe 总线搬一次数据，慢到无法接受。

> **一句话记忆：物理输出（观测）、模型参数、动作，三者必须都在同一块 CUDA 设备上。** 这回答了本章的核心问题的一半——PyTorch 为 RL 提供的不仅是自动求导，还有把"数据 + 模型 + 优化器"统一放在 GPU 上、端到端不落 CPU 的能力。

1.5 自动求导：requires_grad、计算图与 backward()

这是你从"手动推导梯度"迈向"让框架算梯度"的关键一步。

计算图

当 requires_grad=True 的 Tensor 参与运算时，PyTorch 会隐式地记录"谁由谁算出来"，形成一张**计算图**：

```python
x = torch.tensor(2.0, requires_grad=True)
y = x * x + 3 * x        # y = x² + 3x
y.backward()             # 自动算 dy/dx
print(x.grad)            # tensor(7.)  因为 dy/dx = 2x + 3 = 7
```

调用 y.backward() 时，PyTorch 从 y 出发沿计算图反向传播，把梯度累加到每个叶子节点的 .grad 上。这就是你已经学过的反向传播，只不过由框架自动完成。

三个常用开关

用法	作用	典型场景
with torch.no_grad():	关闭梯度记录，省显存、省时间	推理/采样动作时（我们只想要动作，不想更新）
torch.inference_mode()	更强的 no_grad，禁止任何会改变张量的操作	评估时
.detach()	把张量从计算图上"摘下来"，得到无梯度的副本	把 loss 值取出来打印、记录

看项目里真实用法（train.py 的采样阶段）：

```python
for step in range(ppo.rollout_steps):
    observations[step] = observation
    with torch.no_grad():
        action, log_prob, value = model.act(observation)  # 采样动作，不追踪梯度
    ...
```

> 为什么采样时要用 no_grad()？因为 PPO 是"先收集一批数据，再统一更新"。收集数据这一步不应该反向传播——我们只想让策略"表演"出动作，然后把动作、log_prob 记下来，等这一批攒够后再对旧数据做更新。如果在采样时就开梯度，计算图会一直挂着，既浪费显存，又可能污染后续的更新。这个"先采后更"的模式会在第 8 章详述。

1.6 nn.Module：把网络组织成对象

你已经知道"网络 = 一组层 + 前向传播"，nn.Module 就是把这个结构封装成类的标准方式。

最小示例：两层回归网络

```python
import torch
from torch import nn

class TwoLayerRegressor(nn.Module):
    def __init__(self, in_dim, hidden, out_dim):
        super().__init__()
        self.fc1 = nn.Linear(in_dim, hidden)
        self.act = nn.Tanh()
        self.fc2 = nn.Linear(hidden, out_dim)

    def forward(self, x):
        return self.fc2(self.act(self.fc1(x)))

net = TwoLayerRegressor(78, 512, 14)
print(net(torch.randn(4, 78)).shape)   # torch.Size([4, 14])
```

项目里的真实模块

打开 `model.py`，你会看到 `ActorCritic` 继承 `nn.Module`，内部包含两个 MLP（`actor` 和 `critic`）和一个观测归一化器（`obs_normalizer`）。它的 `_mlp` 工厂函数构造了 `78 → 512 → 512 → 512 → 14`（Actor）和 `78 → 512 → 512 → 512 → 1`（Critic）两层结构。

注意这段初始化细节，它体现了 RL 特有的网络初始化考量：

```python
def _mlp(input_size, hidden_size, hidden_layers, output_size):
    ...
    for _ in range(hidden_layers):
        linear = nn.Linear(size, hidden_size)
        nn.init.orthogonal_(linear.weight, gain=math.sqrt(2.0))   # 正交初始化
        nn.init.zeros_(linear.bias)
        ...
    output = nn.Linear(size, output_size)
    nn.init.orthogonal_(output.weight, gain=0.01 if output_size > 1 else 1.0)
    ...
```

- **正交初始化 + `gain=√2`**：配合 Tanh 激活，让信号在深层网络里保持尺度，避免梯度消失。
- **输出层 gain 极小（0.01）**：让初始策略输出的均值接近 0，对应"中立动作"（关节停在默认姿态），从"站立不动"开始学习，而不是一上来就乱蹦。这是 RL 里常见的"从稳定初始策略起步"技巧。

> 你暂时不必记这些增益的推导，但要能意识到：**RL 网络的初始化不是为了"分类准确率最高"，而是为了"初始策略安全、可探索"。** 这与监督学习的目标不同。

参数与 state_dict

```python
for name, p in net.named_parameters():
    print(name, p.shape)        # fc1.weight (512,78), fc1.bias (512,), ...
```

`net.state_dict()` 返回一个"参数名 → 张量"的字典，`torch.save` / `torch.load` 保存和恢复的就是它。这也是 checkpoint 的载体（第 15 章会讲它如何与优化器状态、课程状态一起打包）。

1.7 优化器：SGD 之外的 Adam

你已经学过 SGD 和 AdaGrad。本项目用的是 **Adam**：

```python
optimizer = torch.optim.Adam(model.parameters(), lr=ppo.learning_rate, eps=1.0e-5)
```

Adam 融合了动量（类似 SGD with momentum）和自适应学习率（类似 AdaGrad 但用指数移动平均而非累加，避免学习率单调衰减到 0）。它几乎成了现代 RL 的默认选择。`train.py` 还实现了学习率线性衰减：

```python
schedule = 1.0 - (update - 1.0) / updates
optimizer.param_groups[0]["lr"] = ppo.learning_rate * schedule
```

> 请观察这个细节：RL 训练里，我们不是"跑固定 epoch"，而是"按总步数线性退火学习率"。这与监督学习"按 epoch 衰减"思路类似，但调度基准是"训练了多少步（env steps）"，因为 RL 没有固定的数据集 epoch 概念。

1.8 为什么批量环境完胜逐个循环

假设没有 MuJoCo-Warp 的批量能力，你要训练 256 个机器人，只能写一个 for 循环：

```python
# 伪代码：逐环境循环（低效）
for i in range(256):
    obs_i = envs[i].reset()
    action_i = net(obs_i)        # 256 次小前向，每次利用率低
    envs[i].step(action_i)       # 256 次物理步进
```

而 GPU 的优势在于**把 256 份相同形状的数据拼成一个张量，一次性算完**：

```python
# 批量（高效）
obs = envs.reset()               # (256, 78) 一次取全部观测
action = net(obs)                # 一次前向算 256 个动作，充分利用 GPU 并行
envs.step(action)                # 一次推进 256 个物理世界
```

两者结果相同，但性能差一到两个数量级。原因：

1. **算力利用率**：GPU 擅长"一条指令处理一大批数据"（SIMT）。256 次小矩阵乘，每次都没塞满 GPU 的数千个核心；拼成一次大矩阵乘，才能把所有核心都用起来。

2. **启动开销**：每次 kernel 启动都有固定延迟，256 次启动的延迟远超 1 次。

这就是为什么项目把 `num_envs=256` 作为默认值，也是 MuJoCo-Warp 存在的意义——**让物理模拟本身也变成 GPU 上可并行的批量操作**。

1.9 实践：实现并训练一个两层回归网络

任务 1：CPU/GPU 都跑一遍

写一个脚本，用 PyTorch 拟合 `y = sin(x) + 0.1·x`（或者你熟悉的任意回归任务），要求：

1. 构造 `(1000, 1)` 的输入和标签；
2. 用 `nn.Linear` 搭两层网络，Tanh 激活；
3. 用 Adam 或 SGD 训练；
4. 分别用 `device='cpu'` 和 `device='cuda:0'` 各跑一遍，比较耗时；
5. 每层 `forward` 后打印 `x.shape`，确认输入输出形状。

任务 2：验证广播与索引

```python
import torch
# 课程参数（长度 8 的元组）
track = torch.tensor([1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.75, 5.5])
levels = torch.randint(0, 8, (256,), device='cuda:0')   # 256 个随机等级
distances = track[levels]                                  # (256,) 每个环境的距离
print(distances.shape)
print(distances[:10])   # 抽查前 10 个环境
```

确认你理解了"花式索引 + 批量"这个项目里反复出现的模式。

任务 3：读懂 model.py 里的 RunningNormalizer

打开 `model.py` 的 `RunningNormalizer` 类。它维护 running mean / running var，`forward` 时做 `(x - mean) / sqrt(var + eps)` 并裁剪到 `[-10, 10]`。

思考并写下来：

1. 为什么观测需要归一化？（提示：78 维观测里，关节角度在 ±0.5 量级，关节速度除以 8 后也在 ±1 量级，但接触力除以 100 后可能到 3，量纲差异巨大。）
2. 为什么 `mean`、`var` 用 `register_buffer` 而不是普通属性？（提示：buffer 会随 `state_dict` 一起保存，但不参与梯度更新。）
3. 归一化器的 `update` 用了什么公式？它是简单的"所有历史样本的平均"吗？（提示：是 Welford/Chan 式的增量更新，能避免存下全部历史数据。）

1.10 验收标准

- [ ] 能否解释 `(64, 256, 78)` 这三个维度分别代表什么？

- [ ] 能否写出一行代码，把 CPU 上的 Tensor 搬到 `cuda:0`？
- [ ] 能否说明 `no_grad()` 与 `detach()` 的区别和各自用途？
- [ ] 能否解释：为什么采样动作时要用 `no_grad()`，而 PPO 更新时才开梯度？
- [ ] 能否说明：为什么物理输出、模型参数、动作必须在同一 CUDA device？
- [ ] 能否指出 `state_dict()` 里保存的是什么，它和 checkpoint 的关系是什么？

1.11 常见误区

1. **把 `device` 当成"装饰"** —— 忘了 `.to(device)` 会让程序要么报错、要么静默变慢。每次创建 Tensor 或加载 checkpoint 后，第一件事就是确认它在正确的 device 上。
2. **混淆 batch 维与特征维** —— 看到 `(256, 78)` 就以为是"256 个特征"。要时刻问自己：第一维是"并行个体数"还是"时间步"还是"样本数"。
3. **在采样阶段就开梯度** —— 会在 rollout 收集时累积一张巨大的计算图，显存爆炸或拖慢速度。采样用 `no_grad`，更新才 `backward`。
4. **忽视归一化** —— 78 维观测量纲差异巨大，不归一化会让梯度被大数值通道主导。`RunningNormalizer` 的存在就是证据。

1.12 本章小结

- Tensor ≈ NumPy ndarray + `device` + `autograd`，这两个扩展是 RL 选 PyTorch 的根本原因。
- 强化学习的张量天然带**批维**：`(环境数, 特征数)`，叠加时间维变成 `(时间, 环境, 特征)`。
- 自动求导靠 `requires_grad` + 计算图 + `backward()`；推理/采样用 `no_grad()` 关闭。
- `nn.Module` + 优化器 + `state_dict` 构成"可训练模块"的标准组织方式。
- GPU 并行要求"数据、模型、优化器同一设备"，批量环境是 GPU RL 的性能基石。

下一步：进入第 02 章：概率分布与随机策略，回答"机器人为什么不直接输出唯一动作"——这是从确定性神经网络迈向随机策略的关键一跃。

PART I · 工具、交互与回报

第 02 章：概率分布与随机策略

> 核心问题：**机器人为什么不直接输出唯一动作？**

你已经会用神经网络把输入映射到输出。但在这个项目里，策略网络输出的不是"一个动作"，而是"一个动作的**概率分布**"，再从分布里**采样**得到实际执行的动作。为什么多此一举？这就是本章要回答的问题——它背后是强化学习的第一个核心概念：**探索（exploration）与利用（exploitation）的权衡**。

2.1 学习目标

完成本章后，你应当能够：

1. 区分确定性策略与随机策略。
2. 用正态分布的均值、标准差、采样和 `log_prob` 描述一个随机策略。
3. 解释"探索与利用"为什么必须由随机性来保证。
4. 说明 `tanh` 动作压缩和其概率修正项（log-prob correction）从何而来。
5. 用熵衡量策略的随机程度，并解释熵过大/过小的后果。
6. 对应到项目里 `model.py` 的 `_distribution()`、`act()`、`evaluate()`、`_squash_log_prob()`。

2.2 确定性策略的问题

设想最朴素的方案：网络直接输出 14 个关节目标值（确定性策略），即 $\pi(s) = a$。训练时这样做会遇到一个致命问题——**如何发现更好的动作？**

想象机器人第一次面对"前进"任务，确定性策略输出"所有关节保持中立"。它会原地站着，得到低奖励，然后呢？梯度会告诉它"往某个方向微调关节角"，它照做了，还是低奖励，继续微调…… 问题是：

1. **它会陷入局部最优**：一旦某个"局部还行"的动作让奖励变高，确定性策略会死死抓住它，不敢尝试"暂时更差但长期更好"的动作。
2. **它无法探索动作空间**：确定性策略每次在同一个状态都输出同一个动作，永远不会"试试别的"。

> 你可能会想到 ε-greedy（小概率随机乱动），那确实是一种随机策略，但很粗糙。更优雅的做法是：**让策略本身输出一个分布，从分布中采样。** 采样天然带来随机性，而且这个随机性可以被训练过程"塑形"——该探索的地方分布宽，该确定的地方分布窄。

2.3 随机策略与正态分布

正态分布（高斯分布）

你已经学过正态分布。回顾关键点：

- 概率密度函数：$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$
- μ（均值）决定分布中心，σ（标准差）决定散布宽度，σ^2 是方差。
- 对高维动作，常用**各维独立**的高斯：每个动作分量都有自己的 μ_i 和 σ_i。

项目里的分布

打开 `model.py`，`ActorCritic._distribution()`：

```python
def _distribution(self, obs):
    mean = self.actor(self.obs_normalizer(obs))     # 网络输出 14 个均值
    log_std = self.log_std.clamp(-2.5, 0.0)          # 14 个 log 标准差，可学习
    return Normal(mean.float(), log_std.exp().expand_as(mean).float())
```

注意几个设计选择：

1. **网络只输出均值 `mean`**（14 维），标准差由另一个**可学习参数** `log_std` 决定，而非由网络输出。
2. **用 `log_std` 而非 `std`**：因为标准差必须为正，直接学 `std` 可能越过 0；学它的对数 `log_std` 则无约束，取 `exp` 后天然为正。这是"约束变量用对数/softplus 参数化"的经典技巧。
3. **`log_std` 被裁剪到 `[-2.5, 0]`**：对应标准差范围 $[e^{-2.5}, e^{0}] \approx [0.082, 1.0]$，防止标准差过大（动作乱飞）或过小（彻底不探索）。

`log_std` 的初始值是 `-0.5`（对应 $\sigma \approx 0.61$），这是"中等探索"的起点。

采样与 log_prob

从分布里采样：`distribution.sample()` 或 `distribution.rsample()`（后者支持重参数化，能反传梯度）。

`log_prob(a)` 返回"在这个分布下，采样到动作 a 的对数概率密度" $\log \pi(a|s)$。为什么强化学习如此执着于**对数概率**？

因为策略梯度（第 5 章的核心）里会出现 $\pi(a|s)$，而我们对它求梯度时用到了 $\nabla \log \pi(a|s)$。而且多个动作的联合概率是相乘，取对数后变成相加，数值上更稳定（避免大量接近 0 的数相乘导致下溢）。因此，概率连乘通常会转化为对数概率相加。

2.4 探索与利用：随机性存在的理由

强化学习永远面对一个两难：

- **利用（exploit）**：执行当前认为最好的动作，拿到确定的、高的奖励。
- **探索（explore）**：尝试不确定的动作，可能暂时更差，但有机会发现更好的动作。

如果只利用，你会卡在第一个"还行"的动作里永远不进步；如果只探索，你会一直乱试、永远拿不到稳定奖励。**随机策略用"采样"把两者统一起来：**

- 分布窄（σ 小）→ 采样几乎总是均值附近 → 偏向利用；
- 分布宽（σ 大）→ 采样有较大概率偏离均值 → 偏向探索。

而 `log_std` 可学习，意味着**"探索多少"本身也是被训练出来的**：早期学不会任务时，高熵帮助探索；学会后，策略自动收缩分布、走向确定。项目里还显式加了一个**熵奖励项**（entropy bonus），主动鼓励保持一定随机性，防止过早收敛（第 8 章会展开）。

2.5 tanh 压缩与概率修正

为什么需要压缩

动作必须落在 $[-1,1]$（`env.py` 里 `step()` 第一句就 `action.clamp(-1.0, 1.0)`）。但高斯分布的支持域是整个实数轴，采样值可能远超 $[-1,1]$。方案是：先在高斯里采样一个"潜在值"（latent）u，再用 `tanh` 把它压进 $[-1,1]$：

$$a = \tanh(u), \qquad u \sim \mathcal{N}(\mu, \sigma^2)$$

`tanh` 单调、光滑、值域恰好是 $(-1,1)$。这来自 SAC（Soft Actor-Critic）的经典做法，本项目采纳了它。

概率必须修正

关键问题来了：**a 是 u 经过非线性变换得到的，所以 a 的分布不再是高斯。** 如果你天真地写 `log_prob = normal.log_prob(u)`，你算的是"潜在值 u 的概率密度"，而不是"实际动作 a 的概率密度"。二者之间差一个**变量替换**（change of variables）的雅可比修正项。

由概率密度变换公式：

$$\log \pi(a) = \log p(u) - \log \left| \det \frac{da}{du} \right|$$

因为 $a = \tanh(u)$ 是逐元素运算，雅可比矩阵是对角阵，行列式就是各维导数之积。而 $\frac{d\tanh(u)}{du} = 1 - \tanh^2(u) = 1 - a^2$，所以：

$$\log \pi(a) = \log p(u) - \sum_i \log\left(1 - a_i^2\right)$$

项目里正是这样写的（`model.py` 的 `_squash_log_prob`）：

```python
@staticmethod
def _squash_log_prob(distribution, latent, action):
    correction = torch.log(1.0 - action.square() + 1.0e-6)
    return (distribution.log_prob(latent) - correction).sum(-1)
```

> 那个 `+ 1.0e-6` 是为了防止 action 恰好在 ±1 时取 `log(0)`。`sum(-1)` 是把 14 个动作分量的对数概率加起来，得到整个动作向量的联合对数概率（各维独立 → 概率相乘 → 对数相加）。

为什么必须修正好

如果漏掉这个修正项，PPO 里的新旧策略概率比 $\frac{\pi_{new}(a)}{\pi_{old}(a)}$ 就会算错，策略更新的方向和幅度都会偏，训练可能发散或收敛到错误解。**这是本项目与"简化版教程"的重要区别之一**：真实工程必须处理这类细节，否则纸上推导正确、代码却跑不对。

2.6 熵：随机程度的度量

分布的**熵**衡量它的"不确定度"。对高斯分布，熵有闭式解：

$$H = \frac{1}{2}\log(2\pi e) + \log\sigma \qquad \text{（每维）}$$

直观上：σ 越大，熵越大，采样越"随机"。

在 `model.py` 的 `evaluate()` 里：

```python
entropy = distribution.entropy().sum(-1)
```

注意这里 `distribution.entropy()` 算的是**潜在值** u 的高斯熵，而非压缩后 a 的熵。用于"熵奖励"时二者趋势一致（都随 σ 增大而增大），所以作为探索程度的近似是合理的。但要知道这层细微差别。

2.7 对应代码全景

把 `model.py` 的三个方法串起来，你就看到了"随机策略"的完整生命周期：

方法	时机	做什么
`_distribution(obs)`	每次都要	由 obs 算出高斯分布（均值 + 可学 log_std）
`act(obs, deterministic)`	采样/推理	从分布采样（或取均值），tanh 压缩，返回 `(action, log_prob, value)`
`evaluate(obs, action)`	PPO 更新	对**旧动作**重算新策略下的 log_prob、熵、价值

注意 `act` 和 `evaluate` 的关键区别：

- `act` 在**采样**阶段调用：`obs` 是"新"的，动作是"现在采出来"的。
- `evaluate` 在**更新**阶段调用：它接收的是 rollout 缓冲里**已经采好并存下来的旧动作**，用**更新后的新策略**重新评估这些旧动作的概率和熵。

这个"新旧分离"正是 PPO 的核心，第 8 章会深入。现在你只需理解：**同一个动作，在不同时刻的网络参数下，log_prob 会不同**，而 `evaluate` 的作用就是算出"新网络觉得旧动作有多可能"。

```python
def evaluate(self, obs, action):
    distribution = self._distribution(obs)
    safe_action = action.clamp(-0.999999, 0.999999)   # 防止 atanh 溢出
    latent = torch.atanh(safe_action)                  # 反解出潜在值 u
    log_prob = self._squash_log_prob(distribution, latent, safe_action)
    entropy = distribution.entropy().sum(-1)
    return log_prob, entropy, self.value(obs)
```

`evaluate` 里用 `atanh` 把动作反解回潜在值，是为了"给定旧动作，算它在当前分布下的对数概率"。因为旧动作是实际执行过的 a，要算 $\log \pi(a)$ 就得先知道它对应的 u，而 $u = \tanh^{-1}(a)$。`clamp` 到 ± 0.999999 是因为 `atanh` 在 ± 1 处发散。

2.8 实践：固定均值，改变 log_std

任务 1：采样 10000 个动作，观察分布形状

```python
import torch
from torch.distributions import Normal

mean = torch.tensor([0.0])
for log_std in [-2.5, -1.0, -0.5, 0.0]:
    dist = Normal(mean, torch.tensor([log_std]).exp())
    samples = dist.rsample((10000,))       # 或 sample()
    a = torch.tanh(samples)                 # 压缩到 (-1,1)
    print(f"log_std={log_std:>4}  std={samples.std():.3f}  "
          f"action mean={a.mean():.3f}  action std={a.std():.3f}  "
          f"越界样本占比={((a > 0.999) | (a < -0.999)).float().mean():.4f}")
```

回答：

1. `log_std` 从 -2.5 变到 0，采样标准差和动作标准差怎样变化？
2. 为什么 `tanh` 之后动作 std 总是略小于采样 std？（提示：tanh 在两端饱和，把远离 0 的样本"挤"向 ± 1。）

任务 2：手算并验证 log_prob 修正

对 `u = 0.5`、`a = tanh(0.5)`，分别计算：

1. 直接 `Normal(0,1).log_prob(torch.tensor(0.5))`；
2. 加上修正项 `-log(1 - a²)` 后的值；
3. 用 `model.py` 里 `_squash_log_prob` 的思路重算，确认二者一致。

你应当发现：**未修正的 log_prob 是"潜在值的密度"，修正后才等于"动作的密度"**。体会这两个数的差别，正是"为什么必须修正"的直接证据。

任务 3：统计"越界"与探索

模拟训练初期的策略：`log_std = -0.5`，网络均值全 0。采样 10000 个 14 维动作，统计：

1. 有多少比例的动作分量落在 $[-0.1, 0.1]$（接近中立）？
2. 有多少落在 $[-1, -0.9]$ 或 $[0.9, 1]$（接近饱和）？

这解释了为什么 `env.py` 还要做 `action.clamp(-1,1)` 和 `max_action_delta=0.10` 的变化率限制：即便采样越界，也要保证最终目标是合法的。

2.9 验收标准

- [ ] 能否说明：确定性策略为什么难以探索、容易陷入局部最优？
- [ ] 能否解释：为什么学 `log_std` 而不是直接学 `std`？
- [ ] 能否推导：`a = tanh(u)` 时，$\log \pi(a) = \log p(u) - \log(1 - a^2)$ 从何而来？
- [ ] 能否说明：`log_std` 太大和太小分别会怎样影响机器人学习？（太大→动作乱飞、难收敛；太小→探索不足、卡局部最优）
- [ ] 能否区分 `act()` 与 `evaluate()` 各自在什么时候调用、为什么二者存在？
- [ ] 能否解释：熵作为"探索程度"的度量，为什么被加进 PPO 的损失里？

2.10 常见误区

1. **把 `tanh(mean)` 当成动作** —— 确定性推理时才取均值压缩，训练时是"从高斯采样再 tanh"。混淆二者会让你误以为策略是确定性的。
2. **忘记概率修正项** —— 会导致 PPO 的概率比算错。任何做"压缩动作"的算法都必须带雅可比修正。
3. **以为 `log_std` 越大越好** —— 过大导致动作长期饱和在 ±1、机器人抽搐、无法学到精细控制；过小导致探索不足。它需要被学习到一个合适区间。
4. **混淆采样与 log_prob 的对象** —— 采的是潜在值 u，算 log_prob 要对动作 a 做修正，不能拿 u 的 log_prob 冒充 a 的 log_prob。

2.11 本章小结

- 随机策略输出**分布**而非单点，用采样天然提供探索，权衡探索与利用。
- 项目用"网络输出均值 + 可学习 `log_std`"参数化各维独立高斯。
- `tanh` 把无界采样压进 $[-1, 1]$，代价是必须做雅可比概率修正。
- 熵度量随机程度，被用作 PPO 的探索激励。
- `act()`（采样）与 `evaluate()`（更新时重评旧动作）的分工，是 PPO 的伏笔。

下一步：进入第 03 章：MDP 与强化学习交互，把"随机策略"放进完整的学习循环里——回答"没有标签时，学习信号从哪里来"。

PART I · 工具、交互与回报

第 03 章：MDP 与强化学习交互

> 核心问题：**没有标签时，学习信号从哪里来？**

监督学习给你 (x, y)：输入和标准答案，网络照着答案学。强化学习没有标准答案——没有人告诉你"在这一帧，机器人应该迈左腿还是右腿"。学习信号只能来自环境反馈的**奖励**，以及"试错"本身。本章建立强化学习的理论骨架——**马尔可夫决策过程（MDP）**，并解释学习循环里每一步数据的含义。

3.1 学习目标

完成本章后，你应当能够：

1. 用五元组 $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ 描述一个 MDP。
2. 区分**状态（state）**与**观测（observation）**，并说明本项目属于哪种情况。
3. 解释动作、状态转移、奖励、终止在交互循环中的作用。
4. 区分 trajectory、episode、rollout 三个概念。
5. 说明 on-policy 与 off-policy 的基本区别。
6. 把监督学习的 (x, y) 与强化学习的 $(s, a, r, s', done)$ 对照说明。

3.2 从监督学习到试错学习

先明确一个根本差异：

	监督学习	强化学习
输入	特征 x	观测 o
输出	预测 $\hat{y}$	动作 a
监督信号	标签 y（已知答案）	奖励 r（一个标量，滞后且稀疏）
目标	最小化预测误差	最大化累积奖励
反馈时机	每个样本立即有对错	可能要很多步之后才知道当初动作好不好

> 强化学习最反直觉的一点：**你做的某个动作，可能要到几秒、几十步之后才体现其好坏。** 比如机器人迈出一步，这一步"迈得好不好"可能要到它是否因此摔倒才知道。这就是"信用分配（credit assignment）"问题，第 4 章专门讲。

3.3 MDP：五元组形式化

强化学习把"智能体与环境交互"抽象成**马尔可夫决策过程**，用五个元素描述：

$$\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$$

状态空间 $\mathcal{S}$

环境所有可能状态的集合。在本项目里，"状态"是机器人完整的物理状态：所有关节的位置、速度、身体姿态、接触情况、地形高度场…… 简言之，**足以完全决定未来演化的一切信息**。

动作空间 $\mathcal{A}$

智能体可执行动作的集合。本项目是连续的 14 维向量 $a \in [-1, 1]^{14}$（连续动作空间）。另一类常见的是离散动作空间（如游戏里的"上/下/左/右"）。

状态转移概率 $P(s' \mid s, a)$

在状态 s 执行动作 a 后，转移到 s' 的概率。物理引擎（MuJoCo）就是这个 P 的具体实现：给定当前关节状态和力矩，它精确算出下一时刻的状态。所以机器人项目的转移是**确定性的**（给定 s, a，s' 唯一），但推力、地形抽样、初始扰动等引入了随机性。

奖励函数 $R(s, a)$ 或 $R(s, a, s')$

环境返回的标量信号，衡量"这一步做得好不好"。它是**人为设计**的（第 11 章会详细讲如何设计它，以及设计错了会怎样）。

折扣因子 $\gamma \in [0, 1]$

决定"未来的奖励值多少钱"。$\gamma = 0$ 只看眼前，$\gamma \to 1$ 眼光长远。第 4 章展开。

3.4 状态 vs 观测：为什么"看得见"的不等于"全部的"

这是新手最容易混淆的一对概念：

- **状态** s：环境的**完整、真实**内部信息（马尔可夫意义上的"充分统计量"）。
- **观测** o：智能体**实际收到**的那部分信息，通常是状态的函数 $o = f(s)$，可能是不完整的。

本项目是典型的**部分可观测**情形：策略收到的是 **78 维观测向量**，它**不包含**：

- 地形高度场的完整内容（只有前方 15 个采样点）；
- 即将到来的推力大小和方向（推力发生后才通过物理反馈间接感知）；
- 机器人身体所有部位的精确位置（只有关节角、速度、IMU 等汇总信息）。

策略看不到完整状态，只能根据有限的观测做决策。这不代表设计缺陷——恰恰相反，**限制信息正是为了逼策略学会"泛化"而非"记住答案"**（第 10 章、第 13 章深入）。如果直接把"前方地形是否平坦"这类答案喂给它，它就学不会"看着前方起伏自己抬脚"。

> 严格说，本项目的观测包含了关节角、关节速度、身体线速度、角速度、投影重力等，配合物理积分器，这个观测在多数情况下已经接近马尔可夫。但"推力何时来、地形远处什么样"仍是不可观测的，因此保留"部分可观测"的视角是更严谨、也更实用的理解。

3.5 交互循环：一次 step 的完整解剖

强化学习的核心是一个循环。伪代码：

```text
o0 = env.reset()                # 初始观测
for t = 0, 1, 2, ...:
    a_t = policy(o_t)           # 策略根据观测选动作
    o_{t+1}, r_t, terminated, truncated, info = env.step(a_t)
    if terminated or truncated:
        o0 = env.reset()        # 回合结束，重置
```

对照项目里 `env.step()` 的返回值签名（`env.py`）：

```python
def step(self, action):
    ...
    return self.observation(), reward, terminated, truncated, final_infos
```

注意它返回了**两个**结束标志，而不是一个 `done`：

- **`terminated`**：任务目标达成本身触发的结束。比如摔倒（`fallen`）或成功（`reached`）。语义是"这条轨迹**已经**终结，后面没有未来"。
- **`truncated`**：不是因为任务完成，而是"时间耗尽/步数上限"触发的强制截断（`timeout`）。语义是"轨迹**本来还能继续**，只是被我们掐断了"。

> 为什么要把二者分开？这直接关系到第 7 章 GAE 的 bootstrap：`terminated` 之后价值应为 0（真的结束了），而 `truncated` 之后**仍有未来**（只是我们不采了），所以价值应该用当前状态的价值估计去"接续"（bootstrap），而不是当成 0。这个区别处理错，会让价值估计系统性偏低。现在先记住"它们是两个不同的东西"，第 7 章再推导。

3.6 trajectory、episode、rollout 的区别

概念	含义	本项目里的对应
trajectory（轨迹）	从开始到结束的一条 $(s_0, a_0, r_0, s_1, a_1, r_1, \ldots)$ 序列	一个机器人从 reset 到摔倒/成功/超时的完整过程
episode（回合）	同 trajectory，强调"一段有始有终的尝试"	一次"从起点走到终点或摔倒"的尝试

概念	含义	本项目里的对应
rollout（采样）	用**当前策略**采集一批交互数据的过程/产物	`train.py` 里 `rollout_steps=64` 步 × 256 环境的缓冲

可以这样记忆：**一个 rollout 可能包含许多个（片段的）episode**。因为 256 个环境里，有些在第 3 步就摔了、有些走到第 40 步才超时，它们的 episode 长度各不相同；而 rollout 是按"固定 64 个控制步"来收集的，所以一个 rollout 缓冲里交错着多个不同 episode 的片段。这也解释了为什么 `train.py` 的缓冲里有 `dones` 数组来标记每个环境每一步"是否刚结束"。

3.7 on-policy 与 off-policy：数据从哪个策略来

这是一个贯穿全书的分类，先建立直觉：

- **on-policy（同策略）**：用于更新的数据，必须由**当前最新策略**采集。策略一更新，旧数据就"过期"不能再用了。
- **off-policy（异策略）**：可以学习**别的策略**（甚至历史策略）采的数据，可以反复用、甚至用人类演示数据。

本项目用的是 PPO，属于 on-policy 家族。 证据就在 `train.py` 的流程里：先 `rollout_steps=64` 步收集数据 → 立刻做 5 轮更新 → 丢弃这批数据 → 再用更新后的策略重新收集。

为什么 PPO 要"用完就丢"？因为它用"新旧策略概率比"来约束更新幅度（第 8 章），这个约束只在"数据是当前策略刚采的"时才严格成立。一旦策略更新太多次，旧数据的概率比会严重失真，更新就不可信了。所以 PPO 牺牲了数据复用效率（on-policy 的代价），换来了稳定的更新（不用 off-policy 复杂的重要性采样修正）。

3.8 实践：多臂老虎机与离散小环境

任务 1：多臂老虎机（不需要神经网络）

多臂老虎机（multi-armed bandit）是"单步强化学习"：每次拉一个摇臂，立刻得到奖励，目标是找到平均奖励最高的那个臂。它是 MDP 在"没有状态转移、每步独立"时的退化情形，最适合第一次手写"探索 vs 利用"。

实现一个 ϵ-greedy 老虎机算法：

```python
import numpy as np
rng = np.random.default_rng(0)
true_means = [0.1, 0.5, 0.9, 0.3]      # 4 个臂的真实平均奖励（算法不知道）
q = np.zeros(4)                         # 估计的每个臂价值
counts = np.zeros(4)
epsilon = 0.1
total = 0.0

for t in range(10000):
    if rng.random() < epsilon:          # 探索：随机选
        a = rng.integers(4)
```

```
    else:                               # 利用：选当前最好的
        a = np.argmax(q)
    r = true_means[a] + rng.normal(0, 0.2)   # 采样奖励
    counts[a] += 1
    q[a] += (r - q[a]) / counts[a]      # 增量更新均值
    total += r

print("估计价值", q)
print("选择次数", counts)
```

回答：

1. 算法是否收敛到"总是选臂 2（真实均值 0.9）"？为什么偶尔还会选别的臂？
2. 如果把 `epsilon` 设为 0，会发生什么？（可能永远卡在第一次"看起来还行"的臂上）
3. 这句 `q[a] += (r - q[a]) / counts[a]` 是什么数学式子的增量形式？（提示：样本均值的递推更新，与 `RunningNormalizer` 的思想同源）

任务 2：离散小环境的随机策略

不用神经网络，用 Gymnasium 或自己写一个 1D 网格世界（比如"从格子 0 走到格子 5，左/右两个动作"），实现一个**随机策略**（各动作等概率），跑若干回合，记录每回合步数和总奖励。观察：随机策略的平均表现如何？它"学到了"什么吗？（答案：什么都没学到，但数据里已经包含"哪些路径奖励高"的信息，这正是后续策略梯度要利用的东西。）

任务 3：对照 (x, y) 与 $(s, a, r, s', done)$

写一段文字，对照说明：

- 监督学习里 `x`、`y` 分别对应强化学习里的什么？
- 强化学习里的 `r`、`s'`、`done` 在监督学习里有没有对应物？

把答案写下来，这是本章的"验收题"。

3.9 验收标准

- [ ] 能否写出 MDP 五元组并各举本项目一例？
- [ ] 能否说明：为什么本项目是"部分可观测"的？策略不知道哪些信息？
- [ ] 能否区分 `terminated` 与 `truncated`，并说出为什么它们必须分开？
- [ ] 能否区分 trajectory / episode / rollout，并说明一个 rollout 为何可能包含多个 episode 片段？
- [ ] 能否用一句话说明 on-policy 与 off-policy 的区别，并指出 PPO 属于哪一类、证据何在？
- [ ] 能否把监督学习 (x, y) 与强化学习 $(s, a, r, s', done)$ 逐项对照？

3.10 常见误区

1. **把观测当成状态** —— 观测是状态的"有损投影"。混淆会导致你误以为"策略已经掌握了全部信息"，从而忽略泛化问题。

2. **把 `done` 当成单一概念** —— 本项目特意拆分 `terminated`/`truncated`，正是因为你不能把"摔倒"和"超时"混为一谈。二者对价值估计的影响不同。
3. **以为 PPO 是 off-policy** —— PPO 有经验回放吗？没有。它每采一批就更新、用完就丢，是标准的 on-policy 算法（虽然有 clip 让它对少量旧数据不太敏感）。
4. **以为"强化学习 = 有奖励的监督学习"** —— 监督学习的标签是"逐样本立即正确"，强化学习的奖励是"滞后、稀疏、可人为操纵"的标量。目标函数、优化对象都不同。

3.11 本章小结

- MDP 用 $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ 形式化"智能体-环境交互"。
- 状态是完整信息，观测是智能体实际收到的部分信息；本项目是部分可观测的。
- 交互循环是 RL 的一切：观测 → 动作 → 转移 → 奖励 → 下一观测。
- `terminated`（任务终结）与 `truncated`（时间截断）必须区分，关系到价值 bootstrap。
- on-policy（PPO）用当前策略的数据、用完即丢；off-policy 可复用旧数据但更复杂。

下一步：进入第 04 章：奖励、回报与信用分配，回答"一步动作怎样为几秒后的成功负责"——把稀疏奖励变成可学习的长远目标。

PART I · 工具、交互与回报

第 04 章：奖励、回报与信用分配

> 核心问题：一步动作，怎样为几秒后的成功负责？

上一章我们知道学习信号是奖励。但奖励是"每步一个"的即时标量，而机器人真正在乎的是"最终有没有走到终点"。一个动作的好坏，往往要等到很久之后才体现——这一步迈得稳不稳，决定了三秒后会不会摔倒。如何把"眼前每步的奖励"和"长远的成败"联系起来？这就是**回报（return）**和**信用分配（credit assignment）**要解决的问题。

4.1 学习目标

完成本章后，你应当能够：

1. 区分即时奖励与长期回报。
2. 用折扣因子 γ 写出回报的定义，并解释它的作用。
3. 手算蒙特卡洛（Monte Carlo）回报。
4. 解释信用分配问题，以及"奖励稀疏"带来的学习困难。
5. 理解奖励尺度与终局奖励对学习的影响。
6. 说明为什么 γ 既不是学习率，也不是"固定打折的惩罚"。

4.2 即时奖励 vs 长期回报

- **即时奖励（reward）** r_t：环境在时刻 t 返回的标量，衡量"这一步立刻做得多好"。
- **回报（return）** G_t：从时刻 t 起，未来所有奖励的（折扣）总和，衡量"从现在起，长期会有多好"。

折扣回报（discounted return） 的定义：

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

其中 $\gamma \in [0, 1)$ 是折扣因子。为什么需要 γ？两个原因：

1. **数学上**：保证无限长轨迹的回报有限（几何级数收敛），尤其当任务没有自然终止时。
2. **决策上**：体现"眼前的奖励比遥远的奖励更确定、更值钱"。机器人对"下一毫秒会不会摔倒"很有把握，对"十秒后会不会被推倒"完全没把握，所以后者要打折。

一个能记一辈子的递推式

把定义展开一项，就得到回报的**自回归（递推）形式**：

$$G_t = r_t + \gamma\, G_{t+1}$$

这个式子极其重要——它是第 6 章 TD 学习、第 7 章 GAE 的基石。请现在就记住它。

4.3 手算：不同 γ 下的回报

假设一条 5 步轨迹的奖励是 $[0, 0, 0, 1, 10]$（前几步是铺垫，最后成功拿到大奖励）。

$\gamma = 1.0$（只看总和）：

$$G_0 = 0 + 0 + 0 + 1 + 10 = 11$$

$\gamma = 0.9$：

$$\begin{aligned} G_0 &= 0 + 0.9 \cdot 0 + 0.9^2 \cdot 0 + 0.9^3 \cdot 1 + 0.9^4 \cdot 10 \\ &= 0 + 0 + 0 + 0.729 + 6.561 = 7.29 \end{aligned}$$

$\gamma = 0.5$：

$$G_0 = 0 + 0 + 0 + 0.125 + 0.625 = 0.75$$

观察规律：

1. γ 越小，远处的 10 被折扣得越厉害，G_0 越小。
2. γ 越大，策略越"有耐心"：它愿意为了遥远的成功忍受眼前的低奖励。
3. 本项目用 $\gamma = 0.99$（见 `config.py`），意味着策略非常有远见——每步只衰减 1%。

> 另一个视角：γ 决定了一个"有效时间视野（effective horizon）" $\frac{1}{1-\gamma}$。$\gamma = 0.99$ 对应约 100 步的视野，$\gamma = 0.9$ 对应约 10 步。机器人任务需要把几十步之外的"走到终点"纳入考量，所以必须用接近 1 的 γ。

4.4 Monte Carlo 回报：等回合结束再结算

蒙特卡洛（MC） 方法的核心思想：**用一条真实轨迹跑到底，把实际累积的折扣奖励作为回报的估计。**

$$\hat{G}_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}$$

其中 T 是回合结束的时刻。特点：

- **无偏（unbiased）**：它用的就是真实发生的奖励，没有用任何"猜测"。

- **高方差（high variance）**：一条轨迹的随机性（推力、地形、采样）会直接反映在回报上。同样从状态 s 出发，跑两次可能一次成功一次摔倒，回报天差地别。

MC 回报在第 5 章 REINFORCE 里会直接作为"权重"使用。它的问题（高方差）则是第 6 章引入 Critic 的动机。

4.5 信用分配：谁该为最终结果负责

回到核心问题：机器人第 3 步迈得很好，第 30 步摔倒了。第 3 步那个"好动作"该被表扬吗？还是该为第 30 步的失败背锅？

这就是**信用分配问题**：把一条轨迹的最终结果，合理地"分摊"到沿途的每一个动作上。

MC 的做法是"一律摊派"：G_t 把从 t 到结尾的所有奖励都算到第 t 步头上。这简单、无偏，但方差大——第 3 步的 G_3 里混进了大量与第 3 步无关的、后面 27 步的噪声。

更好的思路是"分清楚哪些奖励**因果地**由第 t 步引起"。这正是**优势函数（advantage）**和 **TD 学习**要做的：用一个"基准线"把与当前动作无关的"环境好运气/坏运气"减掉。第 6 章会给出严格定义，这里先建立直觉：

> **信用分配 = 让每个动作只为它"真正贡献"的那部分回报负责，而不是为整条轨迹的运气负责。**

4.6 奖励稀疏：为什么"只有终点奖励"很难学

如果只在到达终点时给 `+1`，其余全是 `0`，会怎样？

- 早期，机器人随机乱动，几乎永远到不了终点，于是**每条轨迹都是全 0**，G_t 恒为 0。
- 策略收到的信号是"所有动作都一样好"（都是 0），梯度没有方向，**学不动**。
- 只有偶然某次碰巧走到终点，才会突然出现一个非零回报——但这个信号又极稀疏、极偶然。

这就是**稀疏奖励（sparse reward）**问题：信号太稀，探索空间太大，纯靠运气"碰"到奖励几乎不可能。

解决方案之一是奖励塑形（reward shaping）：给"朝目标前进的每一步"一个小的进度奖励，把稀疏的终点奖励"摊薄"成沿途的密集信号。

本项目正是这么做的（`env.py` 的奖励函数）：

```python
reward = (
    10.0 * rewarded_progress          # 每步有效前进位移，密集信号
    + 0.006 * upright * useful_speed  # 直立前进奖励
    + 0.012 * energy_efficiency       # 能效奖励
    - ...惩罚项...
)
```

其中 `10.0 * rewarded_progress` 就是**进度奖励**：机器人每向前移动一点（最多每周期计 0.01 m），就立刻拿到奖励。这保证了即便还没走到终点，策略也能从"我往前挪了"这件事里学到正反馈。

> 但要注意：进度奖励是**双刃剑**。它能让学习启动，也可能诱发"为了奖励而前进"的作弊行为（比如滑行、身体前扑）。第 11 章会专门讲这些"奖励漏洞"。

4.7 奖励尺度与终局奖励

奖励尺度（reward scale）

奖励的**绝对数值大小**会显著影响学习稳定性和收敛速度：

- 奖励太大 → 梯度爆炸，策略震荡；
- 奖励太小 → 梯度消失，学习龟速。

所以设计奖励时要让各项量级**可比**。本项目的奖励各项都在 `0.002` 到 `0.02` 的惩罚、`0.006` 到 `0.012` 的正奖励、`10.0` 的前进奖励这个量级，且前进奖励每步有 `0.01 m` 的封顶，防止"大步冲刺"刷出超大奖励。

终局奖励（terminal reward）

回合结束时给的那个"大块奖励"（或惩罚）尤其重要，因为它一次性概括了"这个回合总体成败"。

本项目里（`env.py` 的 `step()` 末尾）：

```python
reward += torch.where(done,
    torch.where(success, finish_bonus, failure_reward), 0.0)
```

- **成功**：额外 `finish_bonus = 10.0`。
- **失败**：`failure_reward`，其公式是（见 `_failure_penalty`）：

$$\text{failure} = -10 - 2 \cdot \text{remaining} \cdot (1 - \text{progress}) + 4 \cdot \text{progress}$$

拆解这个设计（非常值得琢磨）：

1. 基础失败惩罚 `-10`：摔倒是大事，明确惩罚。
2. `-2 · remaining · (1 - progress)`：**越早摔倒、且走得越少，惩罚越重**。这鼓励机器人"先坚持走远一点"，而不是"开局就摆烂"。
3. `+4 · progress`：**失败也能按已走距离拿回一点信用**。这样"走了 4 米才摔"明显优于"原地不动就摔"，策略能区分"接近成功"和"完全没尝试"。

> 这个设计体现了奖励工程的核心智慧：**不要只给"成功/失败"二值信号，要让奖励能区分"接近成功的失败"和"彻底的失败"**，否则策略学不到"往哪个方向努力是更接近成功的"。第 11 章会系统化这套方法论。

4.8 γ 的三个常见误解

1. **γ 不是学习率**。学习率控制"每次更新迈多大步"（梯度下降的步长）；γ 控制"未来奖励的打折程度"（目标函数的定义）。二者作用于完全不同的环节。

2. **γ 不是"惩罚未来的固定税"**。γ 不是"因为我们不喜欢未来所以打九折"，而是数学上保证收敛 + 决策上表达"远期不确定"。即便你"完全喜欢未来"，无限序列的奖励和也可能发散，仍需要 γ（或平均回报）来收敛。
3. **γ 不是越大越好**。γ 越大，价值估计的方差越大（远期噪声都算了进来）、收敛越慢；γ 越小，策略越短视。要按任务的"时间尺度"选：需要长远规划的（走 5.5 米、抗推力恢复）用大 γ（如 0.99）；只需即时反应的用较小 γ。

4.9 实践：手算回报 + 对比学习难度

任务 1：手算不同 γ 的回报

对轨迹 $[0, 0, 0, 1, 10]$，分别计算 $\gamma \in \{0, 0.5, 0.9, 0.99, 1.0\}$ 时的 G_0, G_1, G_2, G_3, G_4，列成一张表。

再验证递推式：用 $G_t = r_t + \gamma G_{t+1}$ 从后往前递推，确认与直接求和的结果一致。**这一步是第 6、7 章的手工基础，务必亲手算一遍。**

任务 2：体会稀疏 vs 密集奖励

用第 3 章的多臂老虎机或网格世界，做两个变体：

- **变体 A（稀疏）**：只有到达终点给 `+1`，其余 `0`；
- **变体 B（密集）**：每向终点靠近一格给 `+0.1`，到达给 `+1`。

各训练若干回合，观察：哪个变体能更快地"学会往终点走"？记录平均步数和成功率。你应该能直观感受到稀疏奖励的启动困难。

任务 3：分析本项目的终局奖励

打开 `config.py` 和 `env.py` 的 `_failure_penalty`，回答：

1. 为什么失败惩罚里要加上 `+4 · progress` 这一项？去掉它会怎样？
2. 为什么要有 `remaining_fraction`（剩余时间比例）这个因子？它惩罚的是哪种行为？
3. 你能想出一个"策略钻这个失败奖励空子"的例子吗？（提示：会不会有一种策略，故意慢慢走、拖时间，来最大化 progress 同时避免摔倒？）—— 这是第 11 章的预告。

4.10 验收标准

- [ ] 能否写出折扣回报 G_t 的定义和递推式 $G_t = r_t + \gamma G_{t+1}$？
- [ ] 能否解释：γ 越小，远处奖励被折扣得越狠，策略越短视？
- [ ] 能否说明 MC 回报"无偏但高方差"的含义？
- [ ] 能否用自己的话解释信用分配问题，以及为什么 MC 的"一律摊派"不够好？
- [ ] 能否说明稀疏奖励为什么难学，以及进度奖励如何缓解？
- [ ] 能否举例说明奖励尺度不当（过大/过小）的后果？
- [ ] 能否驳斥"γ 是学习率"和"γ 是固定折扣惩罚"两种说法？

4.11 常见误区

1. **把 reward 和 return 混用** —— reward 是每步的即时值，return 是折扣和。策略要最大化的是 return（的期望），不是单步 reward。
2. **以为 γ=0.99 只衰减 1% 所以无所谓** —— 1% 在几十步后累乘是显著的（$0.99^{100} \approx 0.37$），且 γ 直接决定有效视野，影响算法行为。
3. **认为奖励越大越好** —— 奖励大但不可比、不稠密、有漏洞，反而会引入作弊和不稳定。
4. **忽略终局奖励的"区分度"** —— 只给"成功+1/失败-1"会让策略分不清"差一点成功"和"完全没试"。

4.12 本章小结

- 回报是折扣奖励和 $G_t = \sum_k \gamma^k r_{t+k}$，有递推式 $G_t = r_t + \gamma G_{t+1}$。
- γ 决定有效时间视野，非学习率、非惩罚税。
- MC 回报无偏高方差；信用分配是要把结果合理分摊给沿途动作。
- 稀疏奖励难启动，进度奖励是常见缓解；奖励尺度与终局奖励设计直接决定能否学会。
- 本项目的终局奖励用"基础惩罚 + 早摔加重 + 进度信用"区分不同质量的失败。

下一步：进入第 05 章：Policy Gradient，把回报变成可以反向传播的目标——回答"采样动作不可直接求导，策略为什么仍能更新"。

PART II · 策略梯度到 PPO

第 05 章：Policy Gradient

> 核心问题：采样动作不可直接求导，策略为什么仍然能更新？

现在终于到了"把强化学习变成可训练"的关键一步。监督学习里损失函数对参数可微，直接反向传播。但在强化学习里，动作是从分布里采样出来的——"采样"这个操作没有导数。那我们凭什么更新策略参数？答案是一套优雅的数学工具：**策略梯度定理**（Policy Gradient Theorem），以及它的一个经典别名 REINFORCE。

5.1 学习目标

完成本章后，你应当能够：

1. 给出策略梯度的直觉：让"高回报动作的概率上升"。
2. 理解 $\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a|s)\, \Psi_t\right]$ 的来历（含似然比技巧）。
3. 实现 REINFORCE。
4. 解释为什么策略梯度有高方差。
5. 证明（直觉层面）baseline 不改变期望梯度，却能降低方差。

5.2 目标：最大化期望回报

策略 π_θ 是一个参数为 θ 的分布。我们定义**目标函数**为"从初始状态出发，遵循策略得到的期望回报"：

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[G_0(\tau)\right] = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_t \gamma^t r_t\right]$$

其中 $\tau = (s_0, a_0, r_0, s_1, a_1, \dots)$ 是轨迹，G_0 是整条轨迹的回报。我们要找 θ 使 $J(\theta)$ 最大。

难点：J 对 θ 求导时，被积对象里既有"策略生成轨迹的概率"，又有"动作采样"。这些随机性怎么处理？

5.3 似然比技巧：把"采样"变成"加权对数概率"

这是策略梯度最核心的一步。先看一个恒等式（**似然比 / score function 技巧**）：

$$\nabla_\theta \log p_\theta(x) = \frac{\nabla_\theta p_\theta(x)}{p_\theta(x)}$$

反过来写就是：

$$\nabla_\theta p_\theta(x) = p_\theta(x)\,\nabla_\theta \log p_\theta(x)$$

它的意义是：**对一个概率分布的参数求导，可以"通过"这个分布本身，转换成一个对数概率的梯度，再乘以概率。** 这样，期望里的 $\nabla_\theta p_\theta$ 就能被改写成可采样估计的形式。

对整条轨迹，其概率是：

$$p_\theta(\tau) = \underbrace{\mu(s_0)}_{\text{初始状态分布}} \prod_{t=0}^{T-1} \underbrace{\pi_\theta(a_t \mid s_t)}_{\text{策略}} \underbrace{P(s_{t+1} \mid s_t, a_t)}_{\text{环境转移}}$$

注意：**只有策略项 π_θ 依赖 θ，初始分布 μ 和环境转移 P 都不依赖 θ。** 取对数后，与环境有关的项在求导时都会消失（它们的梯度为 0），这大大简化了问题：

$$\nabla_\theta \log p_\theta(\tau) = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

于是：

$$\begin{aligned}
\nabla_\theta J(\theta) &= \nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[G_0] \\
&= \mathbb{E}_{\tau\sim\pi_\theta}\left[G_0 \cdot \nabla_\theta \log p_\theta(\tau)\right] \\
&= \mathbb{E}_{\tau\sim\pi_\theta}\left[G_0 \cdot \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]
\end{aligned}$$

> 这一步的直觉：**我们不需要对"动作"求导，只需要对"动作的对数概率"求导。** 采样本身不可微没关系，因为梯度表达式中出现的是"该动作在当前策略下的对数概率的梯度"，而这是网络输出（分布参数）的可微函数。

5.4 从梯度到损失：为什么"高回报动作概率上升"

用蒙特卡洛采样近似期望，得到REINFORCE 更新：

$$\theta \leftarrow \theta + \alpha\, G_t\, \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

把它看成一个"损失函数"（注意是**梯度上升**，等价于最小化负的它）：

$$L(\theta) = -\,G_t \log \pi_\theta(a_t \mid s_t)$$

直觉非常清晰：

- 若 $G_t > 0$（这个动作最终导致了好结果），梯度上升方向是**增大** $\log \pi_\theta(a_t|s_t)$，即**提高这个动作的概率**；
- 若 $G_t < 0$（导致坏结果），则**降低**这个动作的概率；
- 幅度由 $|G_t|$ 决定：回报越大，概率调整得越猛。

从损失函数的符号可以直接看出：**log_prob 前乘一个回报权重 G_t，回报为正就抬高概率、为负就压低概率。**

5.5 REINFORCE 的完整算法

```text
1. 初始化策略 π_θ (参数 θ)
2. 循环:
   a. 用 π_θ 跑一条完整轨迹，得到 (s0,a0,r0,...,sT)
   b. 对每个时刻 t，计算回报 G_t = Σ γ^k r_{t+k}
   c. 更新 θ += α · Σ_t G_t ∇_θ log π_θ(a_t|s_t)
```

注意第 4.4 节讲过，这里用的 G_t 是 **Monte Carlo 回报**（要等回合结束、用真实累计奖励）。这是 REINFORCE 名字里"蒙特卡洛"的来源：它用整条采样轨迹的回报，而非任何价值估计。

5.6 高方差：REINFORCE 的致命弱点

REINFORCE 简单优雅，但**方差极大**，实际训练很不稳定。来源有二：

1. **回报本身的方差**：同一状态，两条轨迹的 G_t 可能天差地别（一次成功一次摔倒）。策略会被这种"运气"牵着走。
2. **G_t 没有减去基准**：即便某个动作其实是"还行"的，但因为这条轨迹整体运气差（比如途中被随机推力推倒），G_t 为负，策略反而会惩罚这个无辜的好动作。

高方差导致：需要大量样本、训练曲线剧烈震荡、可能发散。

两个经典缓解手段（后续章节展开）：

1. baseline：从 G_t 里减去一个与动作无关的基准 $b(s_t)$，让"好动作"和"坏运气"分离。
2. Critic / Actor-Critic：用一个网络去估计"期望回报"作为 baseline，同时用 TD 代替 MC 降低方差（第 6 章）。

5.7 baseline 为什么不改变期望，却能降方差

设 $b(s_t)$ 是只依赖状态的基准（**不依赖动作 a_t**），用 $G_t - b(s_t)$ 替换 G_t：

$$\nabla_\theta J = \mathbb{E}\left[\sum_t (G_t - b(s_t))\nabla_\theta \log \pi_\theta(a_t|s_t)\right]$$

为什么**期望不变**？关键在下面这个恒等式（对所有只依赖状态的 $b(s)$ 成立）：

$$\mathbb{E}_{a\sim\pi_\theta}\left[b(s)\,\nabla_\theta \log \pi_\theta(a|s)\right] = b(s)\,\nabla_\theta \mathbb{E}_{a\sim\pi_\theta}[1] = b(s)\cdot 0 = 0$$

因为概率分布积分为 1，其对参数求导为 0，所以任何"只依赖状态、不依赖动作"的项乘上 score 后，期望都为 0。**baseline 只是"减去一个期望为 0 的项"，所以不改变梯度方向（的期望）。**

那为什么能降方差？直观地说，G_t 里同时混着"动作本身的好坏"和"环境的固有难度"。比如在很难的地形上，即便动作完美，G_t 也可能偏低。$b(s_t)$ 估计的是"这个状态本身值多少"（与环境难度有关、与当前动作无关），减去它，剩下的 $G_t - b(s_t)$ 就更纯粹地反映"这个动作相对于平均水平做得多好"。这就是**优势**（advantage）的雏形，下一章正式定义。

5.8 与项目的连接：你已经站在 Actor 的门口

回头看 `model.py` 的 `ActorCritic`：它同时输出策略分布（Actor）和价值（Critic）。你现在已经理解前半部分——Actor 就是策略梯度要优化的那个 π_θ。

而后半部分（Critic、优势函数）正是为了修复 REINFORCE 的高方差而引入的。换句话说：

> REINFORCE = 只用 Actor，用 MC 回报当权重，高方差； Actor-Critic = Actor + Critic，用优势当权重，低方差； PPO = 在 Actor-Critic 基础上再加"裁剪"，防止步子迈太大（第 8 章）。

这条进化路线是这门课的主线，请把它刻在脑子里。

5.9 实践：最小 REINFORCE

任务 1：CartPole 上的 REINFORCE

用 Gymnasium 的 `CartPole-v1`（离散动作 2 个），手写一个最小 REINFORCE，不要用任何现成 RL 库：

1. 策略网络：输入 4 维观测，输出 2 个 logits，用 softmax 得到分布，`Categorical` 采样；
2. 收集一条完整轨迹（直到 done），存下每个 `log_prob` 和 `reward`；
3. 从后往前算折扣回报 G_t；
4. 损失 $L = -\sum_t G_t \log \pi(a_t|s_t)$，`backward` + 优化器更新；
5. 记录每回合总回报，画曲线（或打印滑动平均）。

目标：能看到平均回报从几十涨到 400+（CartPole 的满分约 500）。

关键代码骨架：

```python
import torch
from torch.distributions import Categorical

logits = net(state)                       # (2,)
dist = Categorical(logits=logits)
action = dist.sample()
log_prob = dist.log_prob(action)          # 标量

# ... 存 log_prob 和 reward 到列表 ...

# 计算折扣回报（从后往前）
returns = []
G = 0.0
for r in reversed(rewards):
    G = r + gamma * G
```

```
    returns.insert(0, G)
returns = torch.tensor(returns)

loss = -(torch.stack(log_probs) * returns).sum()
optimizer.zero_grad(); loss.backward(); optimizer.step()
```

任务 2：加入 baseline，对比方差

在上面的基础上，加一个价值网络 $V(s)$ 作为 baseline，用 $G_t - V(s_t)$ 替换 G_t。跑两组对照（有/无 baseline），各固定随机种子，比较：

1. 平均回报的收敛速度；
2. 回报曲线的震荡幅度（方差）。

你会直观看到 baseline 的作用。

任务 3：验证"高回报动作概率上升"

在 CartPole 训练中途，挑一个状态，打印训练前/后该状态下两个动作的概率分布，确认"回报更高的那个动作"概率确实上升了。用数据印证 5.4 节的推导。

5.10 验收标准

- [ ] 能否写出似然比技巧 $\nabla_\theta p = p\,\nabla_\theta \log p$，并说明它为何能"绕开采样不可微"？
- [ ] 能否推导：策略梯度中环境转移项和初始分布项的梯度为何为 0？
- [ ] 能否写出 REINFORCE 的更新式，并解释 G_t 为正/负时概率如何变化？
- [ ] 能否说明 REINFORCE 高方差的两个来源？
- [ ] 能否给出 baseline 不改变期望梯度的直觉（为什么乘上 score 后期望为 0）？
- [ ] 能否画出"REINFORCE → Actor-Critic → PPO"的进化主线？

5.11 常见误区

1. **以为要对"采样动作"求导** —— 梯度表达式里只有 $\log \pi(a|s)$ 的梯度，动作只是一个"被查概率"的输入。采样不可微，但我们不求它的导。
2. **用单步 reward 当权重** —— 应该用回报 G_t（未来折扣和），否则就变成"只看眼前"的即时奖励，丢失信用分配。
3. **baseline 用成"依赖动作的量"** —— 那样就不再有"期望为 0"的性质，会改变梯度，甚至引入偏差。
4. **REINFORCE 无 baseline 直接用于连续控制** —— 会因方差过大而极不稳定，实践中几乎总是需要 baseline/Critic。

5.12 本章小结

- 目标最大化期望回报 $J(\theta)$，用似然比技巧把对期望的求导转成对 $\log \pi$ 的求导。

- REINFORCE：$\theta \leftarrow \theta + \alpha G_t \nabla \log \pi(a_t|s_t)$，高回报动作概率上升。
- 高方差源于 MC 回报与无基准；baseline 减方差且不改变期望。
- 项目里的 Actor 就是策略梯度优化的 π_θ，后续章节为其补上 Critic、优势和裁剪。

下一步：进入第 06 章：Value 与 Actor-Critic，引入价值函数和优势函数，正式回答"Critic 怎样帮 Actor 降方差"。

PART II · 策略梯度到 PPO

第 06 章：Value 与 Actor-Critic

> 核心问题：Critic 是怎样帮助 Actor 降低方差的？

上一章的策略梯度靠 MC 回报当"权重"，方差太大。这一章引入一个全新的角色——Critic（价值网络），用它估计"状态有多好"，从而把"整条轨迹的运气"从动作的好坏里剥离出来。这是从 REINFORCE 迈向现代算法（PPO）的必经之路。

6.1 学习目标

完成本章后，你应当能够：

1. 定义状态价值 $V(s)$ 与动作价值 $Q(s,a)$，并说明二者关系。
2. 定义优势函数 $A(s,a)=Q(s,a)-V(s)$，解释它为什么比原始回报更好用。
3. 写出 TD target 与 TD error，并区分 MC 与 TD 两种价值估计方式。
4. 说明 Actor 与 Critic 的目标有何不同、如何协作。
5. 对应到项目里 `ActorCritic` 中 Actor/Critic 的分工。

6.2 价值函数：把"未来"压缩成一个数

状态价值 $V^\pi(s)$

在策略 π 下，从状态 s 出发，期望能获得的（折扣）回报：

$$V^\pi(s)=\mathbb{E}_\pi\left[G_t \mid s_t=s\right]=\mathbb{E}_\pi\left[\sum_{k=0}^{\infty}\gamma^k r_{t+k} \mid s_t=s\right]$$

它回答："站在状态 s，按策略 π 走下去，平均能拿多少分？"

动作价值 $Q^\pi(s,a)$

在状态 s 采取动作 a（之后仍按 π），期望回报：

$$Q^\pi(s,a)=\mathbb{E}_\pi\left[G_t \mid s_t=s, a_t=a\right]$$

它回答："在状态 s 先做动作 a，然后继续按 π，平均能拿多少分？"

二者关系：$V^\pi(s)=\mathbb{E}_{a\sim\pi}[Q^\pi(s,a)]$，即"状态价值 = 该状态下所有动作价值的按策略平均"。

6.3 优势函数：把"环境难度"减掉

优势函数（advantage） 定义：

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$$

直觉：

- $Q(s,a)$ 包含两部分：**这个状态本身有多好**（环境难度、当前位置）+ **这个动作做得有多好**（相对其他动作）。
- $V(s)$ 只包含前者（状态本身的好坏，与当前具体动作无关）。
- 二者相减，剩下的 $A(s,a)$ 就纯粹是"**这个动作比平均好多少**"。

这正是上一章 baseline 的严格版本：$V(s)$ 就是那个"只依赖状态、不依赖动作"的基准。所以：

> **用优势 A 替换策略梯度里的原始回报 G，是"减 baseline 降方差"的理论最优形式。**

对比：

权重	含义	方差
原始回报 G_t	整条轨迹的运气总和	高
$G_t - b(s_t)$（baseline）	减去状态基准	中
优势 $A(s,a)$	动作相对平均水平多好	低

6.4 从 MC 到 TD：Critic 怎样"在线"估计价值

Critic 的任务就是估计 $V(s)$（本项目用 Critic 输出 V，见 `model.py` 的 `critic`）。估计方式有两类：

蒙特卡洛（MC）估计

等回合结束，用真实累计回报作为标签：

$$\text{target}_t = G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots$$

优点无偏；缺点必须等回合结束、方差大。

时序差分（TD）估计

不等回合结束，用**一步真实奖励 + 下一状态的价值估计**作为标签：

$$\text{target}_t = r_t + \gamma\, V(s_{t+1})$$

这就是 TD target。TD error 是它和当前估计的差：

$$\delta_t = r_t + \gamma\, V(s_{t+1}) - V(s_t)$$

TD 的核心思想正是第 4 章的递推式 $G_t = r_t + \gamma G_{t+1}$：用"下一步的估计"代替"未来所有真实奖励"。这叫做 bootstrap（自举）——用估计去估计估计。

	MC	TD
何时能更新	回合结束后	每步都能更新
方差	高	低（只涉及一步随机性）
偏差	无偏	有偏（依赖不完美的 $V(s_{t+1})$）
收敛速度	慢	快

> 这个"偏差-方差权衡"是理解后续 GAE（第 7 章）的钥匙：MC 无偏高方差，TD 低方差有偏，GAE 用 λ 在两者之间连续插值。

6.5 一个关键的递推关系：δ 与优势的关系

可以证明（先接受，第 7 章会展开）：TD error 的某种折扣和，恰好等于优势估计。具体地，单步 TD error $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ 是 $A(s_t, a_t)$ 的一个**无偏估计**：

$$\mathbb{E}[\delta_t] = A^{\pi}(s_t, a_t)$$

（因为 $Q(s,a) = \mathbb{E}[r + \gamma V(s')]$，代入即得。）这个事实极其有用：**Critic 算出的 TD error，可以直接拿来当 Actor 的优势信号。** 这也是为什么 Actor-Critic 里 Actor 和 Critic 能"共享同一个误差"。

6.6 Actor 与 Critic 的目标不同

两个网络，两个目标，各司其职：

	Actor（策略）π_θ	Critic（价值）V_ϕ
输出	动作分布	标量价值
目标	最大化期望回报	准确估计价值
损失	$-\hat A_t \log\pi_\theta(a_t	s_t)$（策略损失）
更新信号	优势 $\hat A_t$（来自 Critic）	TD target（奖励 + 自举）

关键理解：

1. **Critic 预测错误会降低 Actor 的学习效率，但不直接控制动作。** 因为 Critic 只提供"优势信号"，Actor 用它来算梯度。如果 Critic 把优势估计得乱七八糟，Actor 就会朝错误方向更新（学得慢、甚至学错），但机器人实际执行的动作始终只由 Actor 决定。

2. **Actor 与 Critic 是互相依赖的**：Critic 需要 Actor 采的轨迹来学习（因为 V^π 依赖 π）；Actor 需要 Critic 的优势来降低方差。这就是 "Actor-Critic" 名字的含义。
3. **二者可以共享或分离主干网络。** 本项目选择**分离**（`model.py` 里 `self.actor` 和 `self.critic` 是各自独立的 MLP），这样 Critic 的更新不会干扰 Actor 的特征，反之亦然。是否共享是一个工程权衡（共享省参数但可能互相干扰）。

6.7 对应代码：项目里的 Actor 与 Critic

打开 `model.py`：

```python
class ActorCritic(nn.Module):
    def __init__(self, obs_size, action_size, hidden_size=512, hidden_layers=3):
        super().__init__()
        self.obs_normalizer = RunningNormalizer(obs_size)
        self.actor = _mlp(obs_size, hidden_size, hidden_layers, action_size)  # → 14 个均值
        self.critic = _mlp(obs_size, hidden_size, hidden_layers, 1)           # → 1 个价值
        self.log_std = nn.Parameter(torch.full((action_size,), -0.5))
```

- `actor` 输出 14 个动作均值（配合 `log_std` 组成高斯），即 π_θ。
- `critic` 输出 1 个标量，即 $V_\phi(s)$。
- 二者共享同一个 `obs_normalizer`（归一化是公共输入预处理），但网络主体分离。

`value()` 方法就是 Critic 的前向：

```python
def value(self, obs):
    return self.critic(self.obs_normalizer(obs)).squeeze(-1).float()
```

在 `train.py` 里，你会看到价值损失的计算：

```python
value_loss = 0.5 * torch.maximum(
    (new_value - flat_returns[batch]).square(),
    (value_clipped - flat_returns[batch]).square(),
).mean()
```

其中 `flat_returns` 是回报（第 7 章会讲它由 GAE 算出），`new_value` 是 Critic 当前预测。这就是"价值损失 = 均方误差"的具体实现（外加一个 PPO 式的裁剪，第 8 章讲）。

6.8 实践：手算 TD error + 观察两类损失

任务 1：手算 TD error

给定一组数据（$\gamma = 0.99$）：

t	reward	V(s_t)（当前估计）	V(s_{t+1})
0	0.1	0.5	0.6
1	0.2	0.6	0.55
2	-1.0（摔倒）	0.55	0（terminated）

对每个 t 计算 TD target 和 TD error：

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

注意 $t=2$ 时因为 terminated，$V(s_{t+1})$ 应取 0（回合结束了，没有未来）。回答：

1. 哪个时刻的 TD error 是"惊喜"（正值，说明实际比预期好）？哪个是"失望"（负值）？
2. 如果 $t=2$ 不是摔倒而是超时（truncated），$V(s_{t+1})$ 还应该取 0 吗？为什么？（这是第 7 章的核心伏笔。）

任务 2：观察 Actor loss 与 Critic loss

回到第 5 章的 CartPole Actor-Critic 实验，在训练过程中分别记录并打印：

1. Actor 损失（策略损失）的曲线；
2. Critic 损失（价值损失）的曲线；
3. 平均回报的曲线。

观察：Critic 损失下降是否总与回报上升同步？为什么可能不同步？（提示：Critic 学的是"价值的准确性"，回报上升说明策略变好，二者相关但不完全同步。）

任务 3：故意让 Critic 变差，看 Actor 怎样

把 Critic 的学习率设成 0（冻结 Critic，只更新 Actor），再训练，对比回报曲线。你会看到：**Critic 不动时，Actor 仍能靠一个（固定的、可能不准的）价值信号学，但效率和最终效果都变差。** 这直观印证"Critic 预测错误会降低学习效率，但不直接控制动作"。

6.9 验收标准

- [ ] 能否写出 $V(s)$、$Q(s,a)$、$A(s,a)$ 的定义及三者关系？
- [ ] 能否说明优势函数为什么比原始回报方差更小？
- [ ] 能否写出 TD target 和 TD error，并说明 bootstrap 的含义？
- [ ] 能否对比 MC 与 TD 的偏差、方差、更新时机？
- [ ] 能否说明：为什么 δ_t 是 $A(s_t,a_t)$ 的无偏估计（用 $Q(s,a)=\mathbb{E}[r+\gamma V(s')]$ 推导）？
- [ ] 能否解释：Critic 预测错误为何降低学习效率，却不直接控制动作？

6.10 常见误区

1. **以为 Critic 会"指挥" Actor** —— Critic 只输出一个价值标量，它不产生动作。它通过"提供优势信号"间接影响 Actor 的梯度，但永远不直接决定机器人怎么动。
2. **把 $V(s)$ 和 $Q(s,a)$ 混为一谈** —— V 只依赖状态，Q 依赖状态+动作。本项目 Critic 输出的是 V（因为只需优势，不需要完整 Q）。
3. **terminated 与 truncated 的 bootstrap 处理混淆** —— terminated 后价值取 0，truncated 后价值应自举到 $V(s)$。这个区别在 TD target 里体现，第 7 章会严格处理。
4. **以为 TD 一定比 MC 好** —— TD 有偏，偏差在价值函数不准确时会累积。GAE 的存在正是为了在两者间折中。

6.11 本章小结

- $V(s)$ 是状态价值，$Q(s,a)$ 是动作价值，$A = Q - V$ 是优势（动作相对平均的好坏）。
- TD target $= r + \gamma V(s')$，TD error $\delta = r + \gamma V(s') - V(s)$，是优势的无偏估计。
- MC 无偏高方差、TD 低方差有偏，二者是"偏差-方差"权衡的两端。
- Actor 最大化回报、Critic 最小化价值预测误差，二者协作：Critic 提供低方差优势信号，Actor 据此更新。
- 本项目 `ActorCritic` 分离 Actor/Critic 主干，共享观测归一化。

下一步：进入第 07 章：GAE 与批量轨迹，回答"怎样在偏差与方差之间折中"，并看清项目里并行环境下的 rollout 张量布局。

PART II · 策略梯度到 PPO

第 07 章：GAE 与批量轨迹

> 核心问题：怎样在偏差与方差之间折中？

第 6 章我们看到 MC（无偏高方差）和 TD（低方差有偏）是价值估计的两个极端。这一章引入**广义优势估计（GAE, Generalized Advantage Estimation）**，用一个参数 λ 在两者之间连续插值。同时，我们要真正读懂项目里并行环境下的 rollout 张量布局，以及 GAE 的递推实现。

7.1 学习目标

完成本章后，你应当能够：

1. 写出 n-step return 的定义，说明 n 越大越像 MC、越小越像 TD。
2. 推导 GAE 的递推形式，理解 γ 与 λ 各自的作用。
3. 说明 terminated 与 truncated 在 bootstrap 处理上的区别。
4. 读懂并行环境中 rollout 的 **(时间，环境，特征)** 张量布局。
5. 独立写出一个只接收 Tensor 的 GAE 函数，并用手算样例验证。

7.2 n-step return：MC 与 TD 之间的连续谱

第 6 章的 TD target 用"一步真实奖励 + 自举"：$r_t + \gamma V(s_{t+1})$。如果多用几步真实奖励呢？

- 1-step（TD）：$r_t + \gamma V(s_{t+1})$
- 2-step：$r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2})$
- n-step：$r_t + \gamma r_{t+1} + \cdots + \gamma^{n-1} r_{t+n-1} + \gamma^n V(s_{t+n})$

当 $n \to \infty$（走到回合结束），V 项消失，就退化成 MC 回报。

规律：

n	性质	偏向
小（如 1）	低方差、有偏	TD
大（如 ∞）	无偏、高方差	MC

所以"选 n"本身就是一次偏差-方差权衡。但 n 是个离散旋钮，GAE 把它变成连续的。

7.3 GAE：用指数加权平均混合所有 n-step

GAE 的核心想法：**不选某一个 n，而是把所有 n-step return 按几何权重 λ^{n-1} 加权平均。** 得到的优势估计：

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = (1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}\hat{A}_t^{(n)}$$

其中 $\hat{A}_t^{(n)}$ 是 n-step 优势（即 n-step return 减去 $V(s_t)$）。当 $\lambda \to 0$，退化为 1-step（TD）；当 $\lambda \to 1$，退化为 MC。

参数各司其职

参数	作用	项目取值
γ	折扣：控制"未来奖励的价值衰减"，决定时间视野	`0.99`
λ（`gae_lambda`）	偏差-方差权衡：控制"信赖价值估计 vs 信赖真实奖励"	`0.95`

> 一个高频混淆点：γ 和 λ 都出现在递推式里，但**作用完全不同**。γ 是任务本身的折扣（"未来值多少钱"），λ 是估计器的混合系数（"信 TD 还是信 MC"）。γ 不是学习率，也不是固定打折惩罚；同样，λ **也不是折扣因子**，它是偏差-方差旋钮。

7.4 GAE 的递推形式：为什么工程上这么写

直接对 n-step 加权求和太慢。GAE 有一个极简洁的递推式，只需要从轨迹末尾倒着扫一遍。先定义 TD error：

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

则 GAE 递推为：

$$\hat{A}_t = \delta_t + \gamma\lambda\,\hat{A}_{t+1}$$

（严格地说还要乘 $1-done$，见下节。）展开几项你就明白为什么它等价于"所有 n-step 的指数加权平均"：

$$\hat{A}_t = \delta_t + \gamma\lambda\,\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \cdots$$

而 δ_t 正是 1-step 优势，$\delta_t + \gamma\lambda\delta_{t+1}$ 混入了更长的步…… **这就是 GAE 的精髓：一个单行递推，实现了"对所有 n-step return 的几何加权"。**

7.5 terminated 与 truncated 的 bootstrap 处理

第 3 章我们埋下伏笔：`terminated` 和 `truncated` 是不同的。现在它在 GAE 里显形了。

GAE 递推里，如果当前步是"回合的最后一步"，那么 $\hat{A}_{t+1}$ 应该是什么？

- `terminated`（**摔倒/成功，任务真正终结**）：未来真的没有了，$V(s_{t+1})$ 应取 0。递推时 $\hat{A}_{t+1}$ 直接归零。
- `truncated`（**超时，只是我们停止采样**）：回合本可继续，未来仍在，$V(s_{t+1})$ 应**自举到当前价值估计**（不归零）。

看项目里 `train.py` 的实现，它用 `nonterminal` 掩码统一处理：

```python
next_value = model.value(observation)   # rollout 末尾之后的状态价值 (bootstrap 起点)
advantages = torch.zeros_like(rewards)
gae = torch.zeros(ppo.num_envs, device=device)
for step in reversed(range(ppo.rollout_steps)):
    following = next_value if step == ppo.rollout_steps - 1 else values[step + 1]
    nonterminal = 1.0 - dones[step]
    delta = rewards[step] + ppo.gamma * following * nonterminal - values[step]
    gae = delta + ppo.gamma * ppo.gae_lambda * nonterminal * gae
    advantages[step] = gae
returns = advantages + values
```

逐行解读（这是本项目最值得逐字符读懂的一段）：

1. **`next_value = model.value(observation)`**：先对 rollout 之后的最新观测估一个价值，作为递推的"未来起点"。这就是 bootstrap 的种子。
2. **`following`**：第 t 步的 $V(s_{t+1})$。对非最后一步，用缓冲区里存好的 `values[step+1]`；对最后一步，用 `next_value`。
3. **`nonterminal = 1.0 - dones[step]`**：`dones[step]` 是 `terminated|truncated` 的浮点形式。若该步结束，`nonterminal=0`，则：
   - `delta = r_t - V(s_t)`（$V(s_{t+1})$ 项被抹掉）；
   - `gae = delta`（递推项也被抹掉）。
4. **`delta`**：标准 TD error，其中 `following * nonterminal` 实现了"结束时未来价值归 0"。
5. **`gae` 递推**：`delta + gamma * gae_lambda * nonterminal * gae`，这就是 $\hat{A}_t = \delta_t + \gamma\lambda(1-done)\hat{A}_{t+1}$。
6. **`returns = advantages + values`**：回报 = 优势 + 价值。因为 $A_t = G_t - V(s_t)$，移项得 $G_t = A_t + V(s_t)$。

关键细节：terminated 和 truncated 在这里"合并"了

注意项目里 `dones = terminated | truncated`，二者在 GAE 里被**同样处理**（都让 `nonterminal=0`，未来归零）。这是否与第 3 章"必须区分二者"矛盾？

并不矛盾——这里有一个微妙的工程取舍：

- 严格来说，`truncated` 后应自举到 $V(s_{t+1})$（不归零），`terminated` 后应归零。

- 但本项目把 rollout 长度（64 步）设为远大于大多数回合长度（Level 0 时限 12s × 60Hz = 720 步，但很多回合会提前摔倒），所以"在 rollout 边界处恰好被 truncate"的情况较少，且 64 步的时间尺度内 $\gamma^{64} \approx 0.53$ 仍有残余影响。
- 更重要的原因：**这里所谓"truncated"主要是"时间耗尽/步数上限"，即真正结束了这个 episode**（该环境会 reset 到新 episode），所以归零是合理的。`truncated` 更多在语义上标记"这不是摔倒，是超时"，用于统计（fall vs timeout 分开计数），而在 64 步的 GAE 窗口内，两者对"未来"的处理差异被有意简化了。

> 需要你理解的是**原理层面**的区分：如果未来还要继续（比如截断后环境不 reset、状态延续），truncated 必须 bootstrap 到 V；如果未来彻底终结，则归零。本项目因为 episode 结束就 reset，两者都可以安全归零。**理解摔倒与超时为何可能需要不同的 bootstrap 处理，是实现 GAE 时必须掌握的能力。**

7.6 并行环境中的 rollout 张量布局

这是从"单环境伪代码"到"真实 GPU 代码"的跨越。回顾 `train.py` 里缓冲区的分配：

```python
shape = (ppo.rollout_steps, ppo.num_envs)   # (64, 256)
observations = torch.zeros((*shape, OBS_SIZE))    # (64, 256, 78)
actions      = torch.zeros((*shape, ACTION_SIZE)) # (64, 256, 14)
log_probs    = torch.zeros(shape)                 # (64, 256)
rewards      = torch.zeros(shape)                 # (64, 256)
dones        = torch.zeros(shape)                 # (64, 256)
values       = torch.zeros(shape)                 # (64, 256)
```

记住两个约定：

1. **第 0 维 = 时间（rollout 步），第 1 维 = 环境（256 个并行世界），后面才是特征。**
2. **每个环境独立演化**：256 个环境里，同一个 `t` 时刻，有的环境已经跑完一个 episode 又 reset 了，有的还在走。`dones[t, e] = 1` 标记"环境 e 在第 t 步结束时 reset 了"。

为什么 GAE 要按时间维倒着扫

GAE 递推 $\hat{A}_t$ 依赖 $\hat{A}_{t+1}$，所以必须 `for step in reversed(range(rollout_steps))`。而 256 个环境在同一时刻是**互不依赖**的，所以内层可以**向量化**：`gae` 是一个 `(256,)` 张量，`delta` 是 `(256,)`，一次运算同时更新 256 个环境。这就是"时间上串行、环境上并行"的结构——**循环只在时间维（64 次），环境维（256 个）全部并行。**

展平：把 (64, 256, ...) 变成 (16384, ...)

PPO 更新前，要把所有数据展平成一个大 batch：

```python
flat_obs = observations.flatten(0, 1)   # (64*256, 78) = (16384, 78)
```

因为 PPO 的 minibatch 是从"所有（时间×环境）样本"里随机抽取的，样本之间可以打乱、不再区分"来自哪个环境哪一步"。`batch_size = rollout_steps * num_envs = 16384`，`minibatch_size = 2048`，即每个 PPO epoch 把 16384 个样本切成 8 个 minibatch 分别更新。

7.7 实践：手写纯 Tensor 的 GAE 函数

任务 1：写一个 GAE 函数

不依赖项目，写一个只接收 Tensor 的 GAE 函数：

```python
import torch

def compute_gae(rewards, values, dones, next_value, gamma, gae_lambda):
    """
    rewards:   (T,) 或 (T, E)
    values:    (T,) 或 (T, E)
    dones:     (T,) 或 (T, E)  浮点 0/1
    next_value: (E,) 或标量, rollout 之后的状态价值
    返回: advantages (T,) / (T,E), returns = advantages + values
    """
    T = rewards.shape[0]
    advantages = torch.zeros_like(rewards)
    gae = torch.zeros_like(next_value)
    for t in reversed(range(T)):
        following = next_value if t == T - 1 else values[t + 1]
        nonterminal = 1.0 - dones[t]
        delta = rewards[t] + gamma * following * nonterminal - values[t]
        gae = delta + gamma * gae_lambda * nonterminal * gae
        advantages[t] = gae
    return advantages, advantages + values
```

任务 2：用手算样例测试

构造一条 4 步轨迹（单环境）：`rewards=[1, 1, 1, 1]`、`values=[5, 5, 5, 5]`、`dones=[0,0,0,1]`、`next_value=0`，取 $\gamma = 0.9$、$\lambda = 1.0$。

先手算：每步的 δ_t、$\hat{A}_t$ 各是多少？再跑代码验证。

然后改成 $\lambda = 0.0$（纯 TD）和 $\lambda = 1.0$（纯 MC），对比三组优势值，直观理解 λ 的作用。

任务 3：验证 terminated vs truncated

在上面样例里，把最后一步的 `dones` 从"terminated（next_value=0）"改成"truncated（next_value 取某个非零值，比如 4.5）"，观察 $\hat{A}_3$ 如何变化。体会"截断后自举"和"终结后归零"对优势估计的影响。

7.8 验收标准

- [ ] 能否写出 n-step return，并说明 n 增大如何从 TD 过渡到 MC？
- [ ] 能否推导 GAE 递推式 $\hat{A}_t = \delta_t + \gamma\lambda\hat{A}_{t+1}$，并解释 λ 的作用？
- [ ] 能否区分 γ（折扣/视野）与 λ（偏差-方差）两个参数？

- [ ] 能否说明 terminated 与 truncated 在 bootstrap 上应如何处理，以及本项目为何能简化？
- [ ] 能否解释 rollout 缓冲 `(64, 256, 78)` 的三维含义，以及为何"时间串行、环境并行"？
- [ ] 能否独立写出 GAE 函数并通过手算样例测试？

7.9 常见误区

1. **以为 λ 是折扣因子** —— λ 是估计器混合系数，与任务本身的折扣 γ 无关。
2. **GAE 递推忘了乘 `nonterminal`** —— 会导致回合结束后的优势被错误地"延续"到下一回合，污染价值估计。
3. **把 `returns` 当成 `advantages`** —— `returns = advantages + values`，两者只差一个 $V(s_t)$。价值损失用 returns 当标签，策略损失用 advantages 当权重，用途不同。
4. **在环境维上写循环** —— 256 个环境应向量化并行，循环只出现在时间维（64 次）。写了 `for e in range(256)` 就是性能灾难。

7.10 本章小结

- n-step return 是 MC 与 TD 的连续谱；GAE 用 λ 指数加权混合所有 n-step。
- GAE 递推 $\hat{A}_t = \delta_t + \gamma\lambda\hat{A}_{t+1}$，一次倒扫即可算出所有优势。
- γ 管折扣视野，λ 管偏差-方差；terminated 归零、truncated 自举（本项目因 reset 而简化）。
- rollout 缓冲按 `(时间, 环境, 特征)` 组织，GAE 时间串行、环境并行，更新前展平成 `(16384, ...)`。

下一步：进入第 08 章：PPO 完整实现，回答"为什么不能用同一批轨迹无限更新策略"，并看清 clipped objective 的完整数学与代码。

PART II · 策略梯度到 PPO

第 08 章：PPO 完整实现

> 核心问题：为什么不能用同一批轨迹无限更新策略？

现在所有零件都已就位：随机策略（第 2 章）、MDP 交互（第 3 章）、回报（第 4 章）、策略梯度（第 5 章）、Actor-Critic（第 6 章）、GAE（第 7 章）。这一章把它们组装成 PPO（Proximal Policy Optimization）——本项目使用的算法，也是当前深度强化学习里最稳健、应用最广的算法之一。

PPO 要解决的核心问题：**策略梯度对更新步长极其敏感，步长太大直接崩溃。** 它用一个巧妙的"裁剪（clipping）"目标，让更新既高效又不会迈太大步。

8.1 学习目标

完成本章后，你应当能够：

1. 定义新旧策略概率比 $r_t(\theta)$，并说明它的含义。
2. 写出 clipped surrogate objective，解释裁剪如何限制更新幅度。
3. 说明 value loss、entropy bonus 在总损失里的作用。
4. 解释 minibatch 与多轮 update epoch 的关系。
5. 说明 approximate KL、target KL 和梯度裁剪的用途。
6. 解释为什么 PPO 仍然是 on-policy 算法。
7. 逐段读懂 `train.py` 的 PPO 更新循环。

8.2 概率比：新旧策略差多少

PPO 的核心量是**概率比（importance ratio）**：

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$$

- 分子 π_θ 是**更新中**的策略（参数 θ 在变）；
- 分母 $\pi_{\theta_{\text{old}}}$ 是**采集这批数据时**的策略（固定不动）。

直观含义：**同一个动作，新策略比旧策略"多想做"多少倍。**

- $r_t > 1$：新策略更想重复这个动作（概率上升）；
- $r_t < 1$：新策略更想避免这个动作（概率下降）；
- $r_t = 1$：新旧策略在这个动作上没变化。

在代码里（`train.py`）：

```python
new_log_prob, entropy, new_value = model.evaluate(flat_obs[batch], flat_actions[batch])
log_ratio = new_log_prob - flat_log_probs[batch]   # log 相减 = log 之比
ratio = log_ratio.exp()                             # exp 后就是概率比
```

注意实现技巧：**算的是 log 概率，相减后再 exp**，而不是直接除概率。因为概率是 $[0,1]$ 的小数，直接相除数值不稳，而对数空间里相减再 exp 数值更稳、也更符合 `log_prob` 的天然形式。

8.3 未裁剪的目标：TRPO 的问题

如果完全不做约束，策略梯度里"用概率比加权优势"的目标是：

$$L(\theta) = \mathbb{E}_t \left[r_t(\theta)\, \hat{A}_t \right]$$

（其中 $\hat{A}_t$ 是第 7 章 GAE 算出的优势。）这就是**用旧数据、更新新策略**的标准形式。它的意义很直接：优势为正的动作，把概率比抬得越高越好；优势为负的，压得越低越好。

问题在于：**$r_t(\theta)$ 可以变得任意大或任意小。** 如果 $\hat{A}_t$ 很大，梯度会把 $\pi_\theta(a_t)$ 无限往上推，r_t 可能从 1 涨到 100。而这时旧数据已经严重失真——旧策略采的数据，已经不能准确反映新策略的行为。结果就是：**更新步长过大，策略剧烈震荡，甚至崩溃。**

TRPO 用"KL 散度硬约束"来解决（要求新旧策略的 KL 不超过某阈值），但实现复杂。PPO 走了一条更简单的路：**把目标函数本身裁剪掉，让"离旧策略太远"变得不划算。**

8.4 Clipped Surrogate Objective：PPO 的灵魂

PPO 的目标：

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \Big[\min \big(r_t(\theta)\hat{A}_t, \\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \big) \Big]$$

其中 ϵ（本项目 `clip_coef=0.20`）是裁剪幅度。`clip(x, 1-ε, 1+ε)` 把 x 限制在 $[0.8, 1.2]$。

分两种情况理解（这是 PPO 最需要想清楚的地方）：

情况 1：优势为正 $\hat{A}_t > 0$（这个动作是好的）

目标想让 r_t 变大（提高好动作概率）。但 `clip` 把 r_t 的上限压到 $1+\epsilon = 1.2$：

- 当 $r_t \le 1.2$：`min` 取 $r_t\hat{A}_t$，正常鼓励增大；
- 当 $r_t > 1.2$：`clip(r_t)·A` 被压成 $1.2\hat{A}_t$，此时 $r_t\hat{A}_t > 1.2\hat{A}_t$，`min` 取后者，**梯度被截断为 0**——不再继续增大 r_t。

效果：**好动作的概率最多被抬到旧策略的 1.2 倍，再往上就"白费劲"，梯度消失。**

情况 2：优势为负 $\hat{A}_t < 0$（这个动作是坏的）

目标想让 r_t 变小（压低坏动作概率）。`clip` 把 r_t 的下限压到 $1-\epsilon = 0.8$：

- 当 $r_t \geq 0.8$：正常鼓励减小；
- 当 $r_t < 0.8$：`clip(r_t)·A` 被抬成 $0.8\hat{A}_t$（注意 $\hat{A}_t$ 为负，$0.8\hat{A}_t$ 比 $r_t\hat{A}_t$ 更"负"⋯ 需仔细），`min` 保证不会无限压低。

效果：**坏动作的概率最多被压到旧策略的 0.8 倍。**

> 一句话总结：**`min` 保证"只要概率比已经偏离 1 太远，就停止继续推它"，从而把每次更新限制在旧策略附近的小邻域内。**

代码对应（`train.py`）：

```python
policy_loss = torch.maximum(
    -flat_advantages[batch] * ratio,
    -flat_advantages[batch] * ratio.clamp(1.0 - ppo.clip_coef, 1.0 + ppo.clip_coef),
).mean()
```

注意这里用 `torch.maximum` 而不是公式里的 `min`：因为代码**最小化**损失（梯度下降），而公式是**最大化**目标（梯度上升），负号把 `min` 翻成了 `max`。`-A·ratio` 与 `-A·clip(ratio)` 取 `max`，等价于 $A \cdot ratio$ 与 $A \cdot clip(ratio)$ 取 `min`。

8.5 完整的 PPO 损失

PPO 的总损失是三部分之和：

$$L = L^{\text{CLIP}}(\theta) - c_1\, H[\pi_\theta] + c_2\, L^{VF}(\phi)$$

即：**裁剪后的策略损失 − 熵奖励 + 价值损失**。代码里：

```python
loss = policy_loss + ppo.value_coef * value_loss - entropy_coef * entropy.mean()
```

熵奖励（entropy bonus）

减去 `entropy_coef * entropy`（等价于在目标里加上熵奖励），鼓励策略保持一定随机性、防止过早收敛到确定性策略而失去探索。项目里熵系数会**线性衰减**（`entropy_coef=0.002` → `entropy_final_coef=0.0002`）：

```python
entropy_coef = ppo.entropy_final_coef + (ppo.entropy_coef - ppo.entropy_final_coef) * schedule
```

直觉：**早期需要大量探索（熵高），后期接近收敛时逐渐减小探索、走向确定。**

价值损失（value loss）

Critic 的均方误差，但同样加了裁剪（防止价值更新也迈太大步）：

```python
value_clipped = flat_values[batch] + (new_value - flat_values[batch]).clamp(-ppo.clip_coef, ppo.clip_coe
f)
value_loss = 0.5 * torch.maximum(
    (new_value - flat_returns[batch]).square(),
    (value_clipped - flat_returns[batch]).square(),
).mean()
```

`value_clipped` 把新价值限制在旧价值 ±0.2 的邻域内，与策略裁剪同源。`0.5` 是 MSE 的标准系数，让梯度恰好是 `(V - target)`。

8.6 minibatch 与多轮 update epoch

一次 rollout 采到 `16384` 个样本，PPO 要**反复利用**它们做多轮更新（`update_epochs=5`）：

```python
for _ in range(ppo.update_epochs):
    indices = indices[torch.randperm(batch_size, device=device)]  # 每轮打乱
    for start in range(0, batch_size, ppo.minibatch_size):
        batch = indices[start : start + ppo.minibatch_size]
        ... # 用 minibatch 2048 算梯度、更新
```

结构是：5 个 epoch × 8 个 minibatch（16384/2048）= 40 次梯度更新。

为什么能"复用"同一批数据多次？因为 PPO 的裁剪目标保证了：**每更新一次，新旧策略的差距就被限制在 ±20% 内**，所以旧数据在几轮内仍然"基本有效"。若没有裁剪（裸策略梯度），多轮复用会立刻发散——这正是"为什么不能无限更新"的答案。

8.7 approximate KL、target KL 与梯度裁剪

三重保险，各管一层：

approximate KL（监控用）

用概率比的函数近似新旧策略的 KL 散度：

```python
with torch.no_grad():
    approximate_kl = ((ratio - 1.0) - log_ratio).mean()
```

这个量 $r - 1 - \log r \geq 0$，是 KL 的一个近似（当 $r \approx 1$ 时与 KL 同阶）。它**不参与梯度**，只用于监控和早停。

target KL（早停用）

```python
if np.mean(epoch_kls) > ppo.target_kl:   # 0.025
    break
```

如果某个 epoch 的平均 KL 超过 0.025，就**提前停止本轮剩余的 epoch**，不再继续用这批（已经过时的）数据更新。这是 PPO 原论文的经典早停策略：KL 太大说明策略已经漂移太远，再更新风险高。

梯度裁剪（梯度层）

```python
nn.utils.clip_grad_norm_(model.parameters(), ppo.max_grad_norm)  # 0.5
```

把梯度的**全局范数**裁剪到 0.5，防止某个 minibatch 出现异常大梯度把参数打飞。这是深度学习里通用的稳定化手段。

> 三重保险的层次：**目标函数裁剪**（clipped objective，管"更新方向"）、**KL 早停**（管"更新轮数"）、**梯度裁剪**（管"单步更新幅度"）。三者叠加，PPO 才如此稳健。

8.8 为什么 PPO 仍是 on-policy

虽然 PPO 复用了同一批数据做 5 轮更新，它仍是 on-policy 算法，原因是：

1. **数据只来自"上一个策略"**：每个 rollout 只用**当前策略**采集，采完更新几次就**丢弃**，从不用历史策略或其它来源的数据。
2. **复用是"有限、受控"的**：5 轮 + KL 早停 + 裁剪，保证策略在旧数据上只做小步更新，旧数据不会"过期太久"。
3. **对比 off-policy**（如 DQN、SAC）：它们有**经验回放池**，长期保存并反复采样大量历史数据，且用重要性采样修正分布偏移。PPO 没有回放池。

一句话：**PPO 用"小步、多次、用完即丢"的方式，在 on-policy 的框架内榨取数据价值，而不是像 off-policy 那样长期囤积数据。**

8.9 逐段读懂 PPO 更新循环

把 `train.py` 的更新段完整串一遍（已在上文拆解，这里归纳成一张时序图）：

```text
1. 采样 64 步 × 256 环境 → 缓冲 (obs, action, log_prob, value, reward, done)
2. GAE 倒扫 → advantages, returns = advantages + values
3. 展平 (64,256) → (16384), 优势标准化 (减均值/除标准差)
4. for epoch in 1..5:
     打乱样本索引
     for minibatch in 1..8:
```

```
        evaluate 旧动作 → 新 log_prob、entropy、新 value
        算 ratio → clipped policy loss
        算 clipped value loss
        算总 loss = policy + value_coef*value - entropy_coef*entropy
        backward + 梯度裁剪 + optimizer.step()
        记录 p_loss、v_loss、entropy、kl
    若平均 KL > target_kl(0.025): break
5. 更新观测归一化器
6. 打印指标、写 TensorBoard、保存 checkpoint
```

其中"优势标准化"这一步：

```python
flat_advantages = (flat_advantages - flat_advantages.mean()) / (flat_advantages.std() + 1e-8)
```

把优势归一化到均值 0、方差 1，让不同批次、不同阶段的优势尺度一致，学习率的作用更稳定。这是 PPO 工程实现里的常见稳定化技巧。

8.10 实践：打印裁剪目标，构造极端概率比

任务 1：理解裁剪的分段行为

写一个小脚本，固定优势 $\hat{A} = 1$，让概率比 r 从 0.5 变到 2.0，分别计算：

1. 未裁剪目标 $A \cdot r$；
2. 裁剪后目标 $\min(A \cdot r,\ A \cdot \text{clip}(r, 0.8, 1.2))$。

画表（或打印）对比，标出"裁剪开始起作用的区间"。再对 $\hat{A} = -1$ 重复，观察对称性。

任务 2：构造概率比过大和过小的样例

在 CartPole 的 PPO 实验里，人为制造两种情形：

1. 让新旧策略在某状态上概率差 10 倍（比如把新策略的 logits 加上一个大的常数偏移），打印 `ratio` 和 `log_ratio`，观察 `approximate_kl` 如何飙升、`target_kl` 如何触发早停。
2. 把 `clip_coef` 从 0.2 改成 0.0（不裁剪）和 1.0（几乎不限制），各跑一段，观察训练是否发散/震荡。体会裁剪幅度的作用。

任务 3：回答"clip_coef=0.2 的误解"

写一段文字，解释为什么 **`clip_coef=0.2` 不表示"动作只能改变 20%"**。

（提示：裁剪的是**概率比** r_t，不是动作值，也不是策略参数。动作 a 本身可以变很大，只要它在新旧策略下的概率之比不超过 1.2 就行。一个动作从概率 0.01 涨到 0.012，动作值可能天差地别，但概率比只是 1.2。）

8.11 验收标准

- [ ] 能否写出 $r_t(\theta)$ 的定义，并解释 $r_t > 1$、< 1、$= 1$ 的含义？

- [ ] 能否写出 clipped objective，并分别解释优势为正、为负时裁剪如何工作？
- [ ] 能否说明 `torch.maximum` 在代码里对应公式里的 `min`（负号关系）？
- [ ] 能否解释 entropy bonus 为何衰减、value loss 为何也裁剪？
- [ ] 能否说明 minibatch 与 epoch 的关系，以及"为什么能复用数据多次"？
- [ ] 能否区分 approximate KL、target KL、梯度裁剪三者各自的作用？
- [ ] 能否论证 PPO 是 on-policy 算法？
- [ ] 能否驳斥"clip_coef=0.2 表示动作只能改 20%"？

8.12 常见误区

1. **把概率比和动作变化混为一谈** —— clip 限制的是概率比，不是动作的物理幅度。
2. **以为 PPO 有经验回放、是 off-policy** —— 没有回放池，采完即更、用完即丢。
3. **漏掉 entropy 的负号** —— 总损失里熵是**减**（`- entropy_coef * entropy`），因为要**最大化**熵来保持探索。
4. **以为 KL 参与梯度** —— `approximate_kl` 在 `no_grad()` 里算，只监控/早停，不进损失。
5. **以为 clip 越小越好** —— clip 太小（如 0.01）会几乎锁死策略、学不动；太大（如 1.0）等于没裁剪。0.2 是经验上的甜点。

8.13 本章小结

- PPO 用概率比 r_t 度量新旧策略差异，用 clipped objective 把更新限制在旧策略邻域。
- 总损失 = 裁剪策略损失 − 熵奖励 + 裁剪价值损失。
- minibatch × 多 epoch 复用数据，靠裁剪保证数据"短期内仍有效"。
- approximate KL 监控、target KL 早停、梯度裁剪，三重保险保稳定。
- PPO 是 on-policy：数据来自当前策略、有限复用、用完即丢。

下一步：进入第 09 章：双足机器人环境，把算法落地到真实物理世界——回答"物理世界怎样转换成神经网络可以处理的数字"。

PART III · 机器人、观测与奖励

第 09 章：双足机器人环境

> 核心问题：**物理世界怎样转换成神经网络可以处理的数字？**

前面八章，策略网络一直面对的是抽象的张量。这一章落到实地：一个真实的双足机器人，怎样在 MuJoCo 物理引擎里被模拟？策略输出的 14 个数字，怎样变成真实的关节运动？240 Hz 的物理、60 Hz 的控制、256 个 GPU 世界之间是什么关系？读完这章，你应当能"看懂机器人的身体"。

9.1 学习目标

完成本章后，你应当能够：

1. 说明 14 个主动关节与自由浮动根关节的分工。
2. 解释 240 Hz 物理频率、60 Hz 控制频率和 action repeat 的关系。
3. 说明 position actuator、力矩限制和动作变化限制各自的作用。
4. 解释接触、摩擦、滑移、腾空和外力在模拟中的意义。
5. 说明 MuJoCo-Warp 如何批量模拟 256 个 GPU 世界。
6. 区分"策略动作、执行器目标、实际关节角度、实际电机力矩"四个概念。

9.2 机器人的身体：14 个主动自由度

打开 `biped_mujoco.xml`，机器人由一根"脊柱 + 两条腿"组成。骨盆（pelvis）之上是躯干（torso），之下分左右两条腿。

自由浮动根关节：机器人的骨盆用一个 `freejoint`（6 自由度：3 平移 + 3 旋转）悬浮在空间里，它决定机器人"整体在哪里、朝哪"。**根关节不是策略动作**——你不能直接命令机器人在空中平移，只能通过蹬地、摆腿间接影响它。这是双足机器人的本质：全身都在"浮动"，稳定完全靠脚与地面的接触。

14 个主动关节（策略真正能控制的）分四组：

部位	关节	数量	作用
腰部	侧倾 roll + 俯仰 pitch	2	调整上身重心，抵消推力、适应地形
髋（左右各 3）	偏航 yaw、侧摆 roll、俯仰 pitch	6	大腿的旋转、外展、前后摆
膝（左右各 1）	俯仰 pitch	2	小腿屈伸
踝（左右各 2）	俯仰 pitch、侧摆 roll	4	脚掌姿态

左右对称：`left_*` 与 `right_*` 各 7 个。注意**没有腰部偏航（yaw）**——设计者有意省略，因为抗横向推力最需要侧倾、地形俯仰需要腰部俯仰，第三个腰关节只会扩大动作空间却不直接带来收益。这是"任务驱动的最小动作空间"的设计智慧。

9.3 四个频率/层次：从数字到物理

物理频率 240 Hz

MuJoCo 的积分步长 `timestep="0.0041666667"`（即 1/240 秒）。物理引擎每 1/240 秒解一次运动方程。这是"真实世界"的时间尺度，接触、碰撞、摩擦都在这层被精确求解。

控制频率 60 Hz

策略每 1/60 秒（约 16.7 ms）才做一次决策、输出一次动作。它比物理慢 4 倍。

action repeat = 4

$$\text{action_repeat} = \frac{\text{physics_hz}}{\text{control_hz}} = \frac{240}{60} = 4$$

一个策略动作会被重复执行 4 个物理子步。代码里（`env.py` 的 `step()`）：

```python
for _ in range(self.config.action_repeat):   # 4 次
    mjw.step(self.model, self.data)          # 每次推进 1/240 s 的物理
```

为什么不让策略也在 240 Hz 决策？因为：

1. **减少决策开销**：策略前向是昂贵的（512×3 的 MLP），每 1/240 秒跑一次太慢；
2. **高频控制对足式机器人意义有限**：控制回路、执行器带宽、机械惯性都不支持超高频指令；
3. 60 Hz 是人形/四足机器人控制的常见选择（约 16 ms 周期）。

动作变化限制（max_action_delta = 0.10）

策略输出的动作 a 不能"瞬间跳变"，每周期最多变化 ± 0.10：

```python
action_delta = (action - self.last_action).clamp(-max_action_delta, max_action_delta)
applied = self.last_action + action_delta
```

这是对"执行器无法瞬间达到目标"的现实约束：电机有带宽，关节角速度有限，命令变化太快只会让执行器饱和、震荡。限制变化率让动作序列更平滑，也防止策略学会"高频抖动"这种物理上不可能、却能骗奖励的作弊。

9.4 从动作到力矩：position actuator

策略输出的是 $a \in [-1, 1]^{14}$。这些数字怎样变成物理力？链路是：

$$a \xrightarrow{\text{变化率限制}} \text{applied} \xrightarrow{\text{缩放}} \text{目标角} \xrightarrow{\text{PD 控制器}} \text{力矩}$$

代码（`env.py`）：

```
PYTHON
target = (self.nominal + self.config.action_scale * applied).clamp(self.ctrl_low, self.ctrl_high)
self.ctrl.copy_(target)
```

- `nominal`：中立姿态（机器人站立时的关节角，来自 XML 的 keyframe）。
- `action_scale = 0.45`：把 $[-1,1]$ 的动作缩放为相对中立角的偏移（最大 ± 0.45 rad $\approx \pm 26°$）。
- `clamp(ctrl_low, ctrl_high)`：再裁剪到每个关节的物理限位内（如髋俯仰 `range="-1.2 0.9"`）。

这个 `target` 被写进 `ctrl`，交给 position actuator。XML 里每个关节配了一个 position actuator：

```
XML
<position name="a_left_knee_pitch" joint="left_knee_pitch" ctrlrange="0 1.8"
          forcerange="-90 90" kp="90" kv="7"/>
```

position actuator 内部是一个 **PD 控制器**：给定目标角 q_{des}，它算出力矩

$$\tau = kp\,(q_{des} - q) - kv\,\dot{q}$$

其中 `kp=90`（比例增益，像弹簧）、`kv=7`（微分增益，像阻尼）。**策略给的是"目标角度"，不是"力矩"**——真正发力的是这个底层 PD 控制器。而 `forcerange="-90 90"` 把力矩限制在 ± 90 N·m，模拟电机扭矩上限。

9.5 四个概念的严格区分

这是本章最重要的验收点。四个词，四个不同的物理量：

概念	是什么	谁产生	代码位置
策略动作 a	$[-1,1]^{14}$ 的抽象信号	策略网络（Actor 采样 + tanh）	`model.act()`
执行器目标 q_{des}	关节角目标值（rad）	动作经过缩放、限位	`env.step()` 的 `target`
实际关节角度 q	关节真实转过的角度	物理引擎根据力矩积分	`qpos`
实际电机力矩 τ	关节实际承受的力矩（N·m）	PD 控制器 + 力矩限幅	`actuator_force`

它们为什么不相等？ 因为物理世界有惯性、摩擦、执行器带宽。策略命令膝盖"转到 30°"，但膝盖不会瞬间到位——PD 控制器发力、电机限幅、关节还要对抗重力和惯性，最终实际角度可能只到了 27°，且过程有延迟。策略学到的是"发出怎样的目标角，能最有效地达到我的运动意图"，而不是直接控制角度本身。

> 一句话记忆：**策略给"目标"，执行器算"力矩"，物理算出"实际角度"。** 中间每一层都有延迟、限幅和非线性。

9.6 接触、摩擦、滑移、腾空与外力的物理

双足机器人最难的正是"脚和地面的关系"。这些概念在观测和奖励里反复出现，值得先建立物理直觉：

接触（contact）

脚底是否碰到地面。项目里脚不是一整块平板，而是 **4 个球形足垫**（`left_foot_rear_outer` 等），组成一个"支撑多边形"。为什么用 4 个球而不是一块 box？因为 MuJoCo-Warp 的 GPU 版本对"单个 box 与高度场"的连续碰撞检测（CCD）支持有限，4 个球能提供更稳定、更真实的接触。脚底接触状态（`contacts`）和法向力（`forces`）都进了观测。

摩擦与滑移（friction / slip）

摩擦让脚"抓地"，滑移是脚在地上打滑（没有摩擦住）。项目用 MuJoCo 的**椭圆摩擦锥**（`cone="elliptic"`）和六维接触（`condim="6"`）。"打滑"是奖励里被明确惩罚的坏行为（第 11 章），因为它既低效又危险。

腾空（flight）

双脚都离地。短暂的腾空（如跑动、跳跃的瞬间）是正常的，但**长时间双脚腾空**意味着机器人"飞起来了"，不是稳定的行走。项目用 `flight_grace_seconds=0.05` 设置 50 ms 的宽限，超过才开始惩罚"连续腾空"。

外力（external force）

课程里逐级增强的**随机侧向/斜向推力**，直接施加在躯干（torso）上，模拟被推搡。策略**收不到"推力大小"这个输入**（观测里没有它），只能通过身体姿态的物理反馈来感知并恢复平衡。这是对"抗扰动鲁棒性"的考验。

9.7 MuJoCo-Warp：256 个 GPU 世界

传统的 MuJoCo 在 CPU 上模拟，一次一个环境，或靠多进程。MuJoCo-Warp 把物理求解器重写成 Warp（NVIDIA 的 GPU 计算框架），让**一次 kernel 同时推进几百个独立世界的物理**。

`env.py` 里初始化：

```python
self.data = mjw.put_data(self.mj_model, self.mj_data, nworld=self.num_envs, nconmax=96, njmax=384)
```

- `nworld=256`：256 个并行世界；
- `nconmax=96`、`njmax=384`：每个世界的最大接触数、最大关节数（预分配显存）。

然后每一步 `mjw.step(self.model, self.data)` 就在 GPU 上一次性推进全部 256 个世界。物理状态（`qpos`、`qvel`、`ctrl`、`actuator_force`、`sensordata`）都是 GPU Tensor，通过 `wp.to_torch(...)` 直接与 PyTorch 共享内存，**零拷贝**地把物理结果喂给策略网络。

> 这就是第 1 章"批量环境完胜逐个循环"的极致体现：连物理模拟本身都并行了，256 个机器人同时训练，样本吞吐量（SPS）提升一到两个数量级。

9.8 对应代码全景

关注点	文件	关键内容
机器人结构	`biped_mujoco.xml`	14 关节、freejoint、position actuator、传感器
环境封装	`env.py`	`CupBipedEnv`，`step()`、`reset()`、观测与奖励
物理参数	`config.py`	`physics_hz`、`control_hz`、`action_scale`、`max_action_delta`
世界初始化	`env.py __init__`	`mjw.put_data(..., nworld=num_envs)`

传感器（`biped_mujoco.xml` 的 `<sensor>`）为观测提供原始数据：IMU 的陀螺仪 `base_gyro`、速度计 `base_vel`、姿态四元数 `base_quat`、位置 `base_pos`、脚的速度/位置/角速度、脚的触摸传感器 `touch`。观测（第 10 章）就是由这些传感器值加工而来。

9.9 实践：可视化窗口里改一个关节

任务 1：跑脚本化演示，观察身体

```powershell
.\.venv\Scripts\python.exe -m water_cup_rl.robot_demo --seconds 30
```

观察机器人身体结构：躯干、左右腿、四个脚垫、绿色终点横条。注意它是"浮动"的——没有固定在地面的基座。

任务 2：只改变一个关节目标

参考 `robot_demo.py` 的 `_scripted_demo`，它用正弦波驱动髋俯仰/膝/踝关节。修改它，让它**只驱动一个关节**（比如只让左膝正弦摆动），观察：

1. 关节目标角（你设的值）与实际关节角（窗口里看到的角度）是否一致？
2. 身体重心（骨盆）如何随单关节运动而偏移？
3. 哪些脚垫在与地面接触？接触状态如何变化？

这帮你建立"策略动作 → 执行器目标 → 实际角度 → 力矩 → 重心与接触变化"的完整因果链。

任务 3：验证四个概念

在代码里打印一组数，用实际值印证 9.5 节的四个概念：

```python
import torch
from water_cup_rl.env import CupBipedEnv
from water_cup_rl.config import EnvConfig

env = CupBipedEnv(EnvConfig(), num_envs=1, device="cuda:0")
action = torch.zeros((1, 14), device="cuda:0")   # 策略动作: 全 0
obs, reward, *_ = env.step(action)
print("策略动作 a:", action[0][:4].tolist())
print("执行器目标 q_des:", env.ctrl[0, :4].tolist())
print("实际关节角 q:", env.qpos[0, 7:11].tolist())
print("实际力矩 tau:", env.actuator_force[0, :4].tolist())
env.close()
```

观察：全 0 动作对应的目标角是什么？（应约等于 `nominal`，即中立姿态角。）实际关节角和目标角的差从何而来？

9.10 验收标准

- [ ] 能否列出 14 个主动关节的四组分布（腰 2、髋 6、膝 2、踝 4）？
- [ ] 能否说明：为什么根关节不是策略动作？
- [ ] 能否计算并解释 action repeat = 4 的来历？
- [ ] 能否画出"动作 → 目标角 → 力矩 → 实际角"的链路，并指出每层的限幅/延迟？
- [ ] 能否区分策略动作、执行器目标、实际关节角度、实际电机力矩？
- [ ] 能否说明：为什么脚用 4 个球垫而非一块平板？
- [ ] 能否解释 MuJoCo-Warp 如何实现 256 个世界的 GPU 并行？

9.11 常见误区

1. **以为策略直接输出力矩** —— 策略输出的是"目标角"，力矩由 position actuator 的 PD 控制器产生。
2. **以为实际角度 = 目标角** —— 有惯性、阻尼、力矩限幅，实际角总是滞后于目标角。
3. **混淆 physics_hz 和 control_hz** —— 物理 240 Hz、控制 60 Hz，一个动作管 4 个物理子步。
4. **以为根关节可被直接控制** —— 根关节是自由的，只能通过腿部接触间接影响。
5. **以为推力会作为输入喂给策略** —— 推力是"不可观测的扰动"，策略只能靠身体反馈感知。

9.12 本章小结

- 14 个主动关节（腰 2 + 髋 6 + 膝 2 + 踝 4）+ 自由浮动根关节。
- 240 Hz 物理 / 60 Hz 控制 / action repeat 4，动作变化限 0.10。

- 策略给目标角 → position actuator 的 PD 控制器出力矩（含限幅）→ 物理积分出实际角。
- 接触、摩擦、滑移、腾空、外力是双足行走的核心物理，也是观测与奖励的素材。
- MuJoCo-Warp 把 256 个世界放进 GPU 并行，物理状态与 PyTorch 零拷贝共享。

下一步：进入第 10 章：观测与高程感知，回答"策略需要看到什么，才能做出可泛化的落脚决策"——把物理状态精心裁剪成 78 维观测。

PART III · 机器人、观测与奖励

第 10 章：观测与高程感知

> 核心问题：策略需要看到什么，才能做出可泛化的落脚决策？

上一章机器人有了身体，这一章给它一双"眼睛"。观测（observation）是策略认识世界的唯一窗口：给多了会泄露答案、诱发过拟合，给少了又不足以做决策。本章逐项拆解 78 维观测的构成，重点讲清**前方高程扫描**为什么是"让策略学会自己抬脚"的关键，而不是把"该迈哪只脚"直接告诉它。

10.1 学习目标

完成本章后，你应当能够：

1. 逐项说出 78 维观测的组成，并复述其归一化方式。
2. 区分本体感觉（proprioception）与外部感知（exteroception）。
3. 说明观测归一化为什么必要。
4. 解释前方 5×3 高程扫描如何采样、如何随朝向旋转。
5. 理解信息不足、信息泄露和观测噪声三个概念。
6. 论证"增加有意义的输入 ≠ 盲目增加网络参数"。

10.2 78 维观测全景

观测在 `env.py` 的 `observation()` 里拼接，共 78 维。逐项列出（这是必须熟记的表）：

组	内容	维度	归一化
关节角	14 个关节相对中立姿态的角度	14	`qpos[7:] - nominal`
关节速度	14 个关节角速度	14	`qvel[6:] / 8.0`
骨盆线速度	世界坐标系下骨盆速度	3	`qvel[:3] / 2.0`
躯干角速度	IMU 陀螺仪	3	`base_gyro / 5.0`
投影重力	重力在身体坐标系的投影	3	天然在 [-1,1]
躯干高度	骨盆/IMU 的 z 坐标	1	原始
水平朝向	朝向的单位向量（x,y 分量）	2	天然在 [-1,1]
横向偏移	相对赛道中心的横向偏差	1	`/ 0.5`

组	内容	维度	归一化
上一步动作	上一周期的实际动作	14	天然在 [-1,1]
脚接触状态	左右脚是否接触	2	0/1
脚法向力	左右脚接触力	2	`/ 100` 裁剪到 [0,3]
参考速度	本等级目标速度	1	原始
剩余距离	后脚到终点的归一化距离	1	裁剪到 [-0.25,1.5]
剩余时间	归一化剩余步数	1	在 [0,1]
课程等级	当前等级归一化	1	`/ 7`
前方高程	5×3 局部高度图	15	`/ 最大振幅` 裁剪 [-1.25,1.25]
合计		78	

代码对应（`env.py` 的 `observation()`）：

```python
observation = torch.cat((
    self.qpos[:, 7:] - self.nominal,                # 14 关节角
    self.qvel[:, 6:] / 8.0,                         # 14 关节速度
    self.qvel[:, :3] / 2.0,                         # 3 线速度
    state["base_gyro"] / 5.0,                       # 3 角速度
    gravity,                                        # 3 投影重力
    state["base_pos"][:, 2:3],                      # 1 高度
    heading,                                        # 2 朝向
    ((self.qpos[:, 1] - self._world_track_centers()) / 0.5)[:, None],  # 1 横向偏移
    self.last_action,                               # 14 上一步动作
    state["contacts"].float(),                      # 2 接触
    (state["forces"] / 100.0).clamp(0.0, 3.0),      # 2 法向力
    target_speed[:, None],                          # 1 参考速度
    remaining_distance[:, None],                    # 1 剩余距离
    ((step_limits - self.steps).float() / step_limits.clamp_min(1))[:, None],  # 1 剩余时间
    (self.levels.float() / 7.0)[:, None],           # 1 课程等级
    terrain_scan,                                   # 15 高程
), dim=-1)
```

10.3 本体感觉 vs 外部感知

观测里的信息天然分成两类：

- **本体感觉（proprioception）**：机器人对自己身体状态的感知——关节角、关节速度、线速度、角速度、投影重力、姿态、高度、接触。这些来自 IMU、编码器、触摸传感器，是"我自己的身体怎么样了"。
- **外部感知（exteroception）**：机器人对外部世界的感知——前方的地形高程、离终点的距离、离赛道中心的偏移。这些来自"看/测"外部环境。

为什么这两类都要有？ 只有本体感觉，机器人能保持平衡、能走路，但面对起伏地形时是"盲走"，会被绊倒；只有外部感知，机器人知道前面有坡，但不知道自己身体姿态，无法协调肌肉去应对。**鲁棒的双足行走 = 本体感觉（平衡）+ 外部感知（规划落脚点）的融合。**

10.4 观测归一化：让 78 个通道"说同一种语言"

观测里各通道量纲差异巨大：关节角在 ±0.5 rad 量级，关节速度可达 ±几 rad/s，接触力可达上百牛顿，高度是 0.98 米左右的绝对坐标。如果直接喂给网络，数值大的通道（如高度）会主导梯度，数值小的通道（如朝向的 ±1）会被淹没。

解决方案是归一化，分两层：

1. **固定缩放**（在拼接时做）：关节速度 `/8`、线速度 `/2`、角速度 `/5`、接触力 `/100`、横向偏移 `/0.5`…… 用设计好的常数把各通道压到大致同一量级（多在 [-3, 3] 内）。
2. **运行时自适应归一化**（`RunningNormalizer`）：`model.py` 里再套一层"减均值、除标准差"，随训练数据动态调整：

```python
normalized = (value - self.mean) / torch.sqrt(self.var + 1.0e-8)
return normalized.clamp(-self.clip, self.clip)   # 裁剪到 [-10, 10]
```

两层叠加，网络看到的输入始终是"零均值、单位方差"的良好分布，训练稳定得多。`RunningNormalizer` 用增量公式更新 running mean/var，避免存下全部历史观测。

10.5 前方高程扫描：策略的"眼睛"

这是 v23 的核心新增（观测从 v22 的 63 维扩展到 78 维，多出的 15 维就是它）。它模拟了一台**深度相机或激光雷达**：在机器人前方采样 5 个距离 × 3 个横向位置的地形高度。

配置（`config.py`）：

```python
terrain_scan_forward_m = (0.15, 0.35, 0.60, 0.90, 1.20)   # 前方 5 个距离
terrain_scan_lateral_m = (-0.18, 0.0, 0.18)               # 左中右 3 列
```

采样逻辑（`env.py` 的 `_terrain_scan`）：

```python
forward = heading / torch.linalg.vector_norm(heading, dim=-1, keepdim=True).clamp_min(1e-6)
lateral_axis = torch.stack((-forward[:, 1], forward[:, 0]), dim=-1)
sample_xy = base_xy[:, None, :] + distance_grid.reshape(1, -1, 1) * forward[:, None, :] \
                                + lateral_grid.reshape(1, -1, 1) * lateral_axis[:, None, :]
height = self._sample_terrain_height(sample_xy)   # 双线性插值采样高度场
return (height / scale).clamp(-1.25, 1.25)
```

三个关键设计：

1. **随朝向旋转**：采样点不是固定在世界坐标，而是沿机器人**当前的朝向向量**向前、沿其**侧向轴**左右分布。机器人转向时，"前方"也跟着转——就像真实的深度相机装在机器人身上。
2. **双线性插值**：采样点落在高度场网格的格子之间，用 `_sample_terrain_height` 做双线性插值，得到平滑的高度。
3. **归一化到振幅**：除以最大课程振幅（0.075 m）并裁剪，让不同等级的地形高度都落在同一尺度内。

10.6 高程扫描给了什么、没给什么

这是理解"泛化"的关键，也是本章核心问题的答案。

它给了什么

- **局部地形起伏**：前方 15 cm～1.2 m 的地面是凸起还是凹陷、偏左还是偏右。
- **足够规划落脚**：结合自身状态，策略能据此判断"下一步踩在哪、抬多高、身体该不该前倾"。

它没给什么

- **完整的未来地形**：只有前方 1.2 m 内的 15 个采样点，不是全图。
- **"该迈左脚还是右脚"的答案**：高程只描述地形，不提供任何步态相位的提示。策略必须自己学会"看到前面有凸起 → 抬脚跨过"这种**决策**，而不是被直接告知"这里抬左脚"。

> 这正是设计的高明之处：**高程扫描降低了"感知地形"的难度，却绝不降低"学习走路"的难度。** 它帮策略看到障碍，但不告诉它怎么跨。于是策略学到的是一套"看地形 → 自主落脚"的**通用能力**，而非"记住这条跑道哪里有个坎"的死记硬背（第 13 章会验证这一点）。

10.7 信息不足、信息泄露与观测噪声

观测设计的三条边界，值得反复权衡：

信息不足（insufficient information）

给得太少，策略无法做出正确决策。比如没有高程扫描时，机器人面对起伏地形就是"盲走"，只能靠脚碰到障碍后的被动反馈（摔倒）来"事后"学习，效率极低。信息不足的典型症状：**无论怎么训练都学不会，或只能靠大量试错碰运气。**

信息泄露（information leakage）

给得太多，把"答案"或"不该知道的东西"直接塞进观测。例如：

- 直接把"终点坐标的绝对位置"给策略（而非相对距离），它可能学会"朝一个固定世界坐标走"，换条赛道就失效；
- 直接给"推力大小和方向"，策略就能"作弊式"地提前准备，但现实中你无法预知外力；
- 直接把"步态相位"给策略，它就失去了"自己形成步态"的意义。

信息泄露的典型症状：**训练表现极好，但留出地形/新场景上崩盘（过拟合）。**

观测噪声（observation noise）

给的信息是对的，但加入了噪声。适度的噪声是一种**正则化**（domain randomization 的一种），能防止策略过度依赖某个精确读数，提高鲁棒性。比如给关节速度加高斯噪声，逼策略学会"在传感器不那么准的情况下也能走路"。本项目在观测上暂时没有显式加噪，但**推力本身就是一种不可观测的扰动**，客观上起到了类似作用。

> 一句话总结三者的关系：**太少学不会，太多会过拟合，加噪能抗过拟合。** 观测设计就是在"足够"与"不过度"之间走钢丝。

10.8 为什么"增加有意义的输入" ≠ "盲目加参数"

你可能想问：既然要看到地形，为什么不干脆把网络加大、让它"自己从原始数据里学"？

答案在于**归纳偏置（inductive bias）**：

- **有意义的高程输入**：直接把"前方地形高度"这个**结构化、强相关**的信号喂给网络，网络只需学"高度和落脚的关系"，学习空间被大幅缩小，样本效率高。
- **盲目加大网络**：网络要自己从一堆低层信号里"重新发现"地形的概念，需要海量样本，且更容易过拟合到训练地形。

这一权衡可以概括为：

> 没有为解决过拟合盲目扩大网络：更大的网络可能更容易记住固定地形；多地形随机化、留出评估和高程输入是更直接的解决方案。

所以正确的顺序是：**先给"对的信息"（高程、投影重力、剩余距离……），再靠"随机化 + 留出评估"逼它泛化，最后才考虑网络容量。** 网络的 1,139,229 个参数（1M～2M 预算内）是刻意的克制。

10.9 实践：打印并排列高程值

任务 1：把 15 个高程值排成 5×3 表格

用 `env.py` 的 `_terrain_scan` 思路，或直接在 `robot_demo.py` 的 `observation()` 里提取最后 15 维，把它们重排成 5 行（距离）× 3 列（左中右）：

```python
import numpy as np
# 假设 terrain_scan 是 15 维
scan = obs[-15:]
grid = scan.reshape(5, 3)   # 行=前方距离 0.15/0.35/0.60/0.90/1.20, 列=左中右
print(grid)
```

观察：机器人站在平地上、还没进入起伏区时，这 15 个值应该全接近 0；接近起伏区时，某些格子会出现正/负值。把机器人（在可视化里）开到起伏区边缘，再次打印，对比前后变化。

任务 2：验证"随朝向旋转"

在 `robot_demo.py` 里，让机器人在起伏区里转一个身（改变 heading），重新采样前方高程。观察：同样 15 个采样点，采样到的世界位置是否随朝向变了？（提示：朝向变了，`forward` 向量变了，采样点跟着转。）

任务 3：信息泄露的思考题

假设你给观测加上"推力大小与方向"这一维，写出：

1. 训练时策略可能怎样"利用"这个信息？（提前摆出抗推姿势？）
2. 为什么这在真实部署中不可行？
3. 去掉它之后，策略必须转而依赖什么？（身体姿态的物理反馈）

把这三点写成一小段，印证 10.7 节"信息泄露"的概念。

10.10 验收标准

- [ ] 能否说出 78 维观测的 16 组内容，并对关键组说出归一化方式？
- [ ] 能否区分本体感觉与外部感知，并各举 3 个观测维度的例子？
- [ ] 能否说明两层归一化（固定缩放 + RunningNormalizer）各自的作用？
- [ ] 能否解释 5×3 高程扫描的采样方式，以及"随朝向旋转"的意义？
- [ ] 能否说明高程扫描"帮策略看到障碍，但不告诉它怎么跨"？
- [ ] 能否区分信息不足、信息泄露、观测噪声，并各举一例症状？
- [ ] 能否论证"给对的信息优先于加大网络"？

10.11 常见误区

1. **以为观测越全越好** —— 给太多会泄露答案、诱发过拟合。观测设计是"足够 vs 过度"的权衡。
2. **把高程扫描当成"步态提示"** —— 它只描述地形，不提供左右脚相位信息。
3. **忽略归一化** —— 量纲差异会让大数值通道主导梯度。
4. **以为加参数能替代好特征** —— 盲目加大网络反而更容易记住固定地形，过拟合风险更高。
5. **混淆信息不足与泄露的"症状方向"** —— 不足是"学不会"，泄露是"训练好但泛化差"。

10.12 本章小结

- 78 维观测 = 本体感觉（平衡）+ 外部感知（地形/终点）+ 任务参数（速度/距离/时间/等级）。
- 两层归一化让异构通道落在同一尺度。
- 前方 5×3 高程扫描随朝向旋转、双线性插值，是策略的"局部地形眼睛"。
- 观测设计在"信息不足、信息泄露、噪声"之间权衡；给对的信息优先于加参数。

下一步：进入第 11 章：奖励工程与局部最优，回答"为什么奖励增加不一定代表走得更好"——把目光从"看什么"转向"奖励什么"。

PART III · 机器人、观测与奖励

第 11 章：奖励工程与局部最优

> 核心问题：为什么"奖励增加"不一定代表"走得更好"？

奖励是强化学习唯一的教师。教师给错了题、留了空子，学生（策略）就会用你意想不到的方式"刷分"，而那个刷分行为往往根本不是你想教的能力。本章系统拆解本项目的奖励函数，并直面强化学习最著名的问题之一——**奖励黑客（reward hacking）**和**局部最优**。

11.1 学习目标

完成本章后，你应当能够：

1. 逐项解释前进、直立、能效三类正奖励的意图。
2. 逐项解释力矩、动作变化、承重滑移、能耗、连续腾空五类惩罚的意图。
3. 用自己的话定义 reward hacking，并举出物理引擎漏洞的例子。
4. 说明"原地不动、身体冲线、滑行、跳跃"四类局部最优各自的成因。
5. 论证"通用物理约束"优于"直接指定步态"。
6. 解释为什么节能、少滑移的前进会自然产生交替步态。

11.2 奖励函数全景

奖励在 `env.py` 的 `step()` 里计算，共 8 项（3 正 5 负）+ 终局项。完整式子（配 config 里的默认系数）：

```python
reward = (
    progress_reward_scale * rewarded_progress          # +10.0 × 有效前进位移
    + moving_upright_reward * upright * useful_speed   # +0.006 × 直立 × 速度比例
    + energy_efficiency_reward * energy_efficiency     # +0.012 × 能效
    - torque_penalty * torque_cost                     # -0.002 × 归一化力矩²
    - action_smoothness_penalty * smoothness_cost      # -0.002 × 动作变化²
    - slip_penalty * slip_cost.clamp(0, 6)             # -0.015 × 承重滑移
    - energy_penalty * (effective_power / 400).clamp(0,3)  # -0.004 × 功率
    - prolonged_flight_penalty * flight_cost           # -0.020 × 连续腾空
)
# 终局:
reward += where(done, where(success, finish_bonus, failure_reward), 0)
```

先记住一个设计原则，它统领所有项：**奖励不写"步态应该长什么样"，只写"什么样的运动更省力、更稳、更不伤机器"。** 交替步态不是被奖励指定的，而是被这些物理约束"逼"出来的最优解。

11.3 三类正奖励

前进奖励（progress）

```python
def _rewarded_progress(progress, forward_cap_m):
    return progress.clamp(min=-0.03, max=forward_cap_m)   # 每周期最多 0.010 m
```

- `progress` 是这一周期骨盆的 x 向位移（前进为正）。
- `clamp(min=-0.03, max=0.010)`：**前进有封顶，后退有惩罚下限**。封顶 0.01 m 是为了防止"大步冲刺"刷出超大奖励（逼它用正常步频，而非一步跨一大步）；下限 -0.03 是为了惩罚明显后退，但给微小抖动留一点容忍。

这是**密集奖励**的核心：每一步的有效前进都立刻得到正反馈，解决了稀疏奖励的启动问题（第 4 章）。

直立奖励（upright × useful_speed）

```python
upright = torch.exp(-5.0 * gravity[:, :2].square().sum(-1))   # 越竖直越接近 1
useful_speed = (self.qvel[:, 0] / target_speed).clamp(0.0, 1.0)
```

- `upright`：投影重力的 x、y 分量越小，身体越竖直（重力应几乎完全沿 z 向下）。用 `exp` 把"偏离竖直"平滑映射到 (0,1]。
- `useful_speed`：当前前进速度相对目标速度的比例，裁剪到 [0,1]，**只奖励"以合适速度前进"，不奖励超速**（超速也拿不到更多分）。
- 二者**相乘**：只有"既竖直、又在前进"时才给分，防止"倒着走但竖直"或"前倾冲刺但快速"这类偏科。

能效奖励（energy_efficiency）

```python
def _energy_efficiency(progress, effective_energy, robot_weight_n):
    energy_equivalent_distance = effective_energy / robot_weight_n
    return where(progress > 0,
                 progress / (progress + energy_equivalent_distance + 1e-6),
                 zeros_like(progress))
```

这是最巧妙的一项。它衡量"每前进一米，花了多少能量"：

- 分子是前进距离，分母是"前进距离 + 用掉的能量折算成的'等效距离'"（能量 ÷ 体重）。
- 如果前进 1 m 几乎不花能量（高效），比值接近 1；如果花了巨额能量才挪 1 m（低效），比值接近 0。
- `where(progress>0, ...)`：没有前进时能效为 0（原地白耗能不给分）。

> 这项奖励的深层作用：**它把"节能"变成了一种"单位能耗的前进效率"信号，逼策略找到省力的步态。**这正是"交替步态自然涌现"的关键推手之一（见 11.7 节）。

11.4 五类惩罚

力矩惩罚（torque_penalty）

```python
ratio = (self.actuator_force / self.force_limit).clamp(-1.0, 1.0)
torque_cost = ratio.square().mean(-1)
```

- `actuator_force` 是实际力矩，`force_limit` 是每个关节的力矩上限，相除得到"用力占满负荷的比例"。
- 平方后取均值：**惩罚"接近满负荷用力"**。逼策略省力，而不是时刻用最大扭矩硬扛。这也间接减少发热和机械磨损。

动作平滑惩罚（action_smoothness_penalty）

```python
smoothness_cost = (action_delta / max_action_delta).square().mean(-1)
```

惩罚动作的**变化量**（相对允许的最大变化 0.10 归一化）。意图：逼动作平滑、连贯，防止高频抖动的"抽搐"策略。抽搐既伤执行器，又往往是为了骗奖励（快速切换目标角制造瞬时前进假象）。

承重滑移惩罚（slip_penalty）

```python
def _loaded_slip(foot_velocity, contact_force, speed_reference, force_reference, force_clip):
    speed_cost = (speed / speed_reference).clamp(0, 3).square()   # 脚水平速度
    load = (contact_force / force_reference).clamp(0, force_clip) # 接触力权重
    return (speed_cost * load).sum(-1)
```

- **"承重（loaded）"滑移**：不是简单的"脚动了"，而是"脚在**受力着地**时还水平滑动"。`speed_cost`（脚水平速度）乘以 `load`（接触力权重），确保只惩罚"着地打滑"，不惩罚"抬脚摆动"（那本来就该动）。
- 意图：**逼策略踩稳、不在地面滑行。** 滑行是经典的奖励漏洞（见 11.5）。

能耗惩罚（energy_penalty）

```python
effective_power = motor_power + slip_dissipation_weight * slip_loss_power
```

- `motor_power = sum(|tau · qdot|)`：14 个电机的机械功率。
- `slip_loss_power = sum(mu · N · v)`：滑移耗散的功率（摩擦系数 × 法向力 × 滑移速度，库仑摩擦的耗散估计）。
- 二者之和是"总有效功率"，除以 400 W 归一化后惩罚。意图：**总体省电。**

连续腾空惩罚（prolonged_flight_penalty）

```python
flight_cost += ((flight_substeps - grace_substeps).clamp_min(0).float() / max(grace_substeps, 1)).clamp(0, 1)
```

- 统计双脚同时离地的连续子步数，超过 `flight_grace_seconds=0.05`（50 ms，约 12 个物理子步）才开始惩罚。
- 意图：**允许正常的"腾空瞬间"（如跑动、跨步），但惩罚"长时间双脚离地"的跳跃/飞行式作弊。**

11.5 Reward Hacking：当策略比奖励设计者更聪明

奖励黑客（reward hacking） 指：策略找到了一个**最大化奖励、却完全违背设计意图**的行为。它没有"作弊"的主观恶意，只是忠实地优化了你给它的那个数字——而那个数字恰好有漏洞。

在机器人任务里，典型的 reward hacking：

1. **原地不动 + 微调重心刷"直立奖励"**：如果直立奖励权重过高，策略可能选择"站着不动"——它竖直（upright≈1）、不前进（useful_speed=0），乘积仍为 0，其实刷不到分。但若设计不当（比如只奖励 upright 不乘速度），就会变成"站桩"策略。这正是本项目把 `upright * useful_speed` 相乘的原因。
2. **物理引擎漏洞**：MuJoCo 的碰撞、摩擦、contact 求解有数值近似。策略可能学会"脚尖轻轻点地制造接触计数"、"高频微振动骗过接触检测"、"利用 solver 的软接触穿透地面前进"等，在真实物理里根本不可能发生。本项目的对抗措施包括：椭圆摩擦锥、六维接触、承重滑移惩罚、连续腾空惩罚。
3. **滑行（sliding）**：如果只奖励前进位移、不惩罚滑移，策略可能学会"拖着脚在地面滑行"，前进很快、几乎不用抬脚，但这不是"行走"。`slip_penalty` 就是专门治这个的。

> 核心教训：**你奖励什么，就会得到什么（未必是你想要的）。** 奖励工程的第一步不是"加更多正奖励"，而是"审视每一项奖励可能被怎么钻空子"。

11.6 四类局部最优

即使没有明显漏洞，策略也可能收敛到"次优但稳定"的行为：

局部最优	表现	成因	本项目对策
原地不动	站着不动，靠能效/直立拿一点分	前进奖励不足以抵消"动起来会摔"的风险	前进奖励权重高（10.0），且失败有进度信用
身体冲线	上身前倾，靠躯干越过终点	若"越线"只看躯干位置，前倾即可	成功条件要求**双脚足底中心**都过线
滑行	拖脚前进	只奖励位移、不惩罚滑移	承重滑移惩罚 + 滑移耗散计入能耗

局部最优	表现	成因	本项目对策
跳跃	频繁腾空"飞"过起伏	腾空能快速越过障碍、且若不计入成本	连续腾空惩罚 + 能效惩罚

注意"身体冲线"的成因在项目里已被成功条件堵死：

```python
def _feet_reached_finish(foot_positions, finish_x):
    return (foot_positions[:, :, 0] >= finish_x[:, None]).all(dim=-1)
```

必须**两只脚的足底中心都越过终点**才算成功，头部或躯干前倾越线不算。这是奖励/成功条件设计里"堵漏洞"的典范。

11.7 为什么节能、少滑移的前进会自然产生交替步态

现在可以回答一个关键问题：**如何在不写"两脚必须交替"的前提下，解释交替步态会自然涌现？**

推理链：

1. **前进奖励**逼机器人移动，且只奖励**有效位移**（每周期封顶 0.01 m）。
2. **能效奖励**逼它**省力**地移动。对双足机器人，最省力的前进方式是什么？——**让重心平稳前移、两脚交替支撑和摆动**，而不是同时起跳（要克服整个身体的重力）或拖地滑行（有摩擦耗散）。
3. **承重滑移惩罚 + 滑移耗散计入能耗**逼它**踩稳**：脚着地时不能滑动，所以必须"抬脚 → 前摆 → 落脚"，这正是"迈步"。
4. **连续腾空惩罚**排除了"双脚同时离地跳跃"的取巧方案。
5. **动作平滑 + 力矩惩罚**排除了"高频抽搐"等非物理方案。

把这些约束合起来，唯一"既前进、又省力、又不滑、又不飞、又不抽搐"的运动模式，**恰好就是两条腿交替迈步的稳定步态**。

> 换句话说：**交替步态不是被"写"进奖励的，而是"被物理约束和能效约束一起挤压出来的唯一出口"**。这正是"通用奖励"优于"手工指定步态"的根本原因——你得到的是一套**符合物理规律的通用运动能力**，而不是一段只对特定场景成立的"脚本"。而能自然涌现步态的前提，是奖励设计者把所有"取巧出口"都堵死了。

11.8 对应代码与审计工具

关注点	文件	说明
奖励计算	`env.py step()`	8 项奖励 + 终局项
奖励参数	`config.py`	各 penalty/scale 系数

关注点	文件	说明
奖励审计	`reward_audit.py`	用脚本化动作对比不同行为的奖励

`reward_audit.py` 是专门用来"审计奖励是否合理"的工具。它用四种**脚本化动作**（不训练，直接给固定动作）跑环境，对比它们的奖励：

- `neutral`（全 0，中立站立）
- `gentle`（轻柔的正弦摆腿）
- `random`（随机动作）
- `bounce`（高频大摆幅，制造跳跃）

看它的输出格式：

```text
CASE     N  RETURN   LIFE   DIST   BEST  SPEED  FALL  SLIP  FLIGHT  P_MOTOR  P_SLIP  EFF  COT
neutral  ...
gentle   ...
```

如果奖励设计合理，"gentle（温柔走）"的回报应明显高于"neutral（站着）"和"random（乱动）"，而"bounce（跳跃）"应因腾空/能耗惩罚而回报偏低。这是一个"不训练、只看奖励排序是否合理"的快速 sanity check。

11.9 实践：构造假想数据，比较奖励排序

任务 1：四种假想行为的手工排序

不跑代码，先凭直觉判断：对"站立、前进、摔倒、滑行"四种行为，本项目的奖励函数会给它们怎样的相对排序？写下你的预测和理由。

任务 2：用 reward_audit 验证

```powershell
.\.venv\Scripts\python.exe -m water_cup_rl.reward_audit --worlds 24 --device cuda:0
```

对照输出，验证你的预测。特别关注：

1. `neutral` 与 `gentle` 的 RETURN 谁高？为什么？
2. `bounce` 的 FLIGHT 和 EFF（能效）列是否如预期地差？
3. 是否有某个行为"奖励高但你直觉上觉得不对"？如果有，那就是奖励设计的疑点。

任务 3：设计一个"奖励漏洞"并堵上它

假设去掉 `slip_penalty`（把 `slip_penalty` 设为 0），推测策略会学会什么行为？写一段文字：漏洞是什么、策略会怎么钻、你会加回哪一项来堵。这是"奖励工程师"思维的第一步训练。

11.10 验收标准

- [ ] 能否逐项说出 3 类正奖励和 5 类惩罚的意图？
- [ ] 能否解释"前进封顶 0.01 m"和"后退下限 -0.03 m"各自的用意？
- [ ] 能否用自己的话定义 reward hacking，并举 2 个物理引擎漏洞的例子？
- [ ] 能否说明四类局部最优（原地/冲线/滑行/跳跃）的成因与对策？
- [ ] 能否论证"通用物理约束优于直接指定步态"？
- [ ] 能否复述 11.7 节的推理链：为什么节能+少滑移+前进会自然涌现交替步态？

11.11 常见误区

1. **以为奖励越多越好** —— 奖励项越多，越容易互相冲突、被钻空子。每加一项都要问"它可能被怎么骗"。
2. **以为"奖励上升 = 走得更好了"** —— 奖励上升可能只是策略学会了某个漏洞行为。必须结合成功率、滑移、腾空、能耗、可视化共同判断（第 14 章）。
3. **把步态写进奖励** —— 直接指定"左右脚相位"会得到僵硬的脚本步态，无法泛化到新地形。
4. **忽略成功条件的漏洞** —— 只奖励"越线"却不限定"双脚都过"，策略就会前倾冲线。
5. **以为惩罚越狠越好** —— 惩罚太重会压垮所有探索，策略干脆躺平（什么都不做反而惩罚最小）。

11.12 本章小结

- 奖励 = 前进 + 直立×速度 + 能效（正），减去力矩、动作变化、承重滑移、能耗、连续腾空（负）+ 终局。
- 奖励黑客是"优化了数字而非意图"；物理引擎漏洞、滑行、跳跃、冲线都是经典案例。
- 局部最优源于奖励/成功条件的漏洞，对策是"堵出口 + 引导出口"。
- 交替步态是"前进 + 节能 + 防滑 + 防腾空"共同挤压出的唯一物理最优，而非被显式指定。

下一步：进入第 12 章：课程学习与复习机制，回答"为什么不能一开始就训练最难地形，为什么只训练最高级也不够"。

PART IV · 课程、泛化与实验

第 12 章：课程学习与复习机制

> 核心问题：为什么不能一开始就训练最难地形？为什么只训练最高级也不够？

直接让机器人从"平地学走路"跳到"73 mm 起伏 + 35 N 推力"的最难地形，它连站都站不稳，更遑论学会走——这是"课程学习（curriculum learning）"要解决的"起步"问题。但反过来，一路只往前冲、练到 Level 7 就把 Level 0 忘光，又会出现"灾难性遗忘（catastrophic forgetting）"——这是"复习机制"要解决的"保住"问题。本章拆解本项目的课程调度器。

12.1 学习目标

完成本章后，你应当能够：

1. 解释课程难度、晋级门槛、统计窗口三个概念。
2. 说出八级课程各维度的递增规律（距离、速度、振幅、推力、事件率）。
3. 区分 frontier 与 review，说明 70/30 分配的意图。
4. 说明复习回合为什么参与 PPO 却不决定晋级。
5. 用自己的话解释灾难性遗忘与任务分布漂移。
6. 计算给定等级时 256 个环境的等级分布。

12.2 八级课程：由易到难的五条线

课程同时沿着**五个维度**递增（见 `config.py`）：

等级	距离	参考速度	时限	最大垂直偏差	最大推力	推力事件率
0	1.50 m	0.18 m/s	12.0 s	平地	0 N	无
1	2.00 m	0.20 m/s	13.0 s	0.006 m	4 N	约 5.0 s/次
2	2.50 m	0.22 m/s	14.0 s	0.012 m	7 N	约 4.0 s/次
3	3.00 m	0.24 m/s	15.0 s	0.020 m	11 N	约 3.0 s/次
4	3.50 m	0.26 m/s	16.0 s	0.030 m	16 N	约 2.5 s/次
5	4.00 m	0.28 m/s	17.0 s	0.042 m	22 N	约 2.0 s/次
6	4.75 m	0.31 m/s	18.5 s	0.058 m	28 N	约 1.4 s/次

等级	距离	参考速度	时限	最大垂直偏差	最大推力	推力事件率
7	5.50 m	0.34 m/s	20.0 s	0.075 m	35 N	约 1.0 s/次

观察规律：

1. **距离、速度、时限**单调递增：走得更远、更快、但给的时间也更多（净难度仍上升）。
2. **地形振幅**从 0 平滑增到 75 mm（约腿长 720 mm 的 10%），逐级只加约 6～17 mm，保证"跨一步不太难"。
3. **推力**从 0 增到 35 N，且**事件率**从 5 秒一次加密到 1 秒一次——不只是"推得更重"，还"推得更频繁"。

> 关键设计：**每一级的难度增量都足够小，让"上一级学到的策略"能作为"下一级的起点"**，而不是推倒重来。这就是课程学习的本质——把一条困难的学习曲线，切成若干段"刚好能迈过去的台阶"。

12.3 晋级门槛与统计窗口

课程不会"练一阵就自动升级"，而是由**成功率**驱动，且用**统计窗口**平滑噪声。

打开 `curriculum.py` 和 `config.py`：

```python
class CurriculumConfig:
    window_episodes = 1_500          # 统计窗口：最近 1500 个回合
    promote_success_rate = 0.70      # 晋级：成功率 ≥ 70%
    demote_success_rate = 0.20       # 降级：成功率 ≤ 20%
    promote_minimum_episodes = 1_500 # 晋级前至少采 1500 个回合
    demote_minimum_episodes = 2_000  # 降级前至少采 2000 个回合
```

`CurriculumScheduler` 维护一个 `deque(maxlen=1500)` 存最近 1500 个回合的成功/失败（0/1）：

```python
def record(self, success, episode_level=None):
    if episode_level is not None and episode_level != self.level:
        return False                        # 复习回合不参与晋级判定
    self.outcomes.append(float(success))
    self.episodes_at_level += 1
    rate = self.success_rate                # sum(outcomes) / len(outcomes)
    ...
```

三个要点：

1. **统计窗口（1500）**：不用"最近一次"或"最近 10 次"判断，而是用最近 1500 个回合的成功率。这平滑了单个回合的随机性（一次摔倒不该触发降级，一次侥幸成功也不该触发晋级）。
2. **晋级门槛（70%）**：最近 1500 回合成功率达 70% 才晋级。70% 而非 100%，是因为要允许一定的探索失败，同时"足够熟"。

3. **降级门槛（20%）**：成功率跌到 20% 以下会**降级**——这是对抗"冲太猛导致全面崩盘"的保险，也是课程学习的"双向"设计（不只是单调上升）。

12.4 frontier 与 review：70/30 分配

当等级 > 0 时，256 个环境被分成两拨：

- `frontier`（**前沿**）：70% 的环境训练**当前等级**（正在攻克的最高难度）。
- `review`（**复习**）：30% 的环境复习**所有已掌握等级**（Level 0 到当前等级-1）。

分配逻辑在 `train.py` 的 `curriculum_levels()`：

```python
def curriculum_levels(level, number, review_fraction):
    if level <= 0:
        return [0] * number              # Level 0 时全练 0
    review = min(number - 1, max(1, round(number * review_fraction)))  # 30% ≈ 77 个
    total_weight = level * (level + 1) / 2.0
    review_levels = []
    for slot in range(review):
        target = (slot + 0.5) / review * total_weight
        cumulative = 0.0
        for mastered_level in range(level):
            cumulative += mastered_level + 1
            if target <= cumulative:
                review_levels.append(mastered_level)
                break
    return [level] * (number - review) + review_levels
```

复习等级不是均匀分布的，而是**向较近等级倾斜**：

- 权重设计：已掌握等级 0..level-1 各有权重 1, 2, 3, ..., level（`cumulative += mastered_level + 1`）。
- 所以 Level 5 时复习 Level 4 的概率 > 复习 Level 0 的概率。
- 但 Level 0 的权重（=1）不为 0，**不会被彻底丢弃**——最基础的站立技能仍会偶尔被复习到。

> 为什么复习要"偏向近期"？因为临近等级的技能与当前任务最相关、也最容易遗忘；而早期技能（如站立）已经练得极熟、遗忘慢，只需少量复习维持即可。

12.5 复习回合参与 PPO，但不决定晋级

这是本章最精妙的一个设计点，务必理解：

- **复习回合参与 PPO 更新**：30% 复习环境采到的数据，会和前沿数据一起进入 rollout 缓冲、一起算优势、一起更新网络。也就是说，复习**确实在训练**网络，帮它"保持"旧技能。
- **复习回合不决定晋级**：`record()` 里第一行就是：

```python
if episode_level is not None and episode_level != self.level:
    return False   # 非前沿等级（复习回合）不进入 outcomes，不影响晋级判定
```

也就是说，一个在 Level 2 复习中"成功"的回合，**不会**被计入"Level 5 的成功率"里。

为什么必须这样分离？ 因为复习等级比前沿等级简单得多，成功率天然高。如果复习回合也计入晋级判定，那么"前沿成功率"会被复习的高成功率**虚高**，课程就会在"Level 5 还没真学会"时就错误晋级到 Level 6，导致在更难的地形上全面崩盘。

> 一句话：**复习是用来"保技能"的，晋级是用来"测新技能"的。二者数据必须分开，否则晋级信号会被污染。** 因此，"晋级成功率高"与"旧等级没有遗忘"是两个不同指标，不能混为一谈。

12.6 灾难性遗忘与任务分布漂移

灾难性遗忘（catastrophic forgetting）

神经网络的一个著名缺陷：**学习新任务时，会"覆盖"掉旧任务的知识。** 原因是你只有一套权重，梯度更新往新任务方向走，旧任务的表现就退化。

在课程学习里，这表现为：策略一路练到 Level 7（最难地形），某天你让它回 Level 0（平地），它反而走得比以前差——因为它已经把"平地走路"的精细控制"忘"在了"应对 35 N 推力"的更新里。

任务分布漂移（task distribution shift）

课程学习本身会制造分布漂移：训练数据的分布随等级推进而不断变化。如果只练当前级，网络看到的永远是"当前难度"，它会对"当前难度"过拟合，失去跨难度的稳健性。

复习机制的对策

30% 复习正是同时对抗这两个问题：

1. **对抗遗忘**：定期回访旧等级，用旧任务的梯度"刷新"对应技能，防止权重被单向覆盖。
2. **对抗漂移**：让训练分布始终是"前沿 + 所有旧等级"的混合，而非单一难度，网络学到的策略对难度变化更稳健。

> 如果把课程学习比作"爬梯子"，那么**复习就是"回头看看下面的台阶还稳不稳"**。只爬不回头，最终你站在高处，脚下的台阶却已经塌了。

12.7 实践：计算 256 个环境的等级分布

任务 1：手算等级分布

用 `curriculum_levels(level, 256, 0.30)` 的逻辑，分别计算 Level 2、Level 5、Level 7 时，256 个环境里各等级的**大致**数量分布。

先手算 Level 2 这个简单情形：

- review = round(256 × 0.30) = 77（且 min(255, max(1, 77)) = 77）
- 前沿：256 − 77 = 179 个环境练 Level 2
- 复习：77 个环境，复习 Level 0 和 Level 1，权重分别为 1 和 2
  - 总权重 = 3
  - Level 0 占比 1/3 → 约 26 个
  - Level 1 占比 2/3 → 约 51 个

再算 Level 5 和 Level 7 时复习等级的权重分布（0..4 权重 1..5；0..6 权重 1..7），列出每个等级的期望数量。

任务 2：写代码验证

把 `curriculum_levels` 函数抄到自己的脚本里，分别传入 `level=2/5/7`，`number=256`，`review_fraction=0.30`，用 `collections.Counter` 统计每个等级的数量，与手算对照。

任务 3：验证"复习不决定晋级"

读 `curriculum.py` 的 `record()`，写一个最小测试：构造一个 `CurriculumScheduler`，先喂入若干 `(success=True, episode_level=当前level)`，再喂入 `(success=True, episode_level=其它level)`，观察后者是否改变 `success_rate`。确认复习回合被正确排除在晋级判定之外。

12.8 验收标准

- [ ] 能否说出八级课程递增的五个维度，并举例说明？
- [ ] 能否解释：为什么每级难度增量要"足够小"？
- [ ] 能否说明统计窗口（1500）和晋级门槛（70%）如何配合、各自解决什么问题？
- [ ] 能否区分 frontier 与 review，说明 70/30 和"复习偏向近期"的设计意图？
- [ ] 能否解释：复习回合为什么参与 PPO 却不决定晋级？混在一起会怎样？
- [ ] 能否用自己的话解释灾难性遗忘与任务分布漂移，以及复习如何对抗？
- [ ] 能否说明"晋级成功率高"与"旧等级没遗忘"是两个不同指标？

12.9 常见误区

1. **以为课程 = 单调升到最高级** —— 本项目有**降级**机制（20%），课程是双向的。
2. **把复习回合的成功计入晋级** —— 会让前沿成功率虚高，提前晋级、全面崩盘。
3. **以为复习只是"怀旧"** —— 复习对抗的是灾难性遗忘和分布漂移，是训练的硬需求。
4. **以为晋级门槛越高越好（如 95%）** —— 门槛太高会让晋级极慢、卡在低等级；70% 是"足够熟 + 允许探索失败"的平衡。
5. **混淆"晋级"与"能力"** —— 晋级成功率高 ≠ 旧技能没忘。二者要用不同指标分别衡量（第 13、14 章）。

12.10 本章小结

- 八级课程沿距离/速度/振幅/推力/事件率五条线递增，每级增量足够小。
- 晋级由"最近 1500 回合成功率 ≥ 70%"驱动，降级由"≤ 20%"触发，双向调节。
- frontier（70% 前沿）+ review（30% 复习、偏向近期但保留早期）。
- 复习参与 PPO 保技能，但不进晋级判定，避免污染前沿信号。
- 复习对抗灾难性遗忘与分布漂移，是课程学习能"爬高又站稳"的关键。

下一步：进入第 13 章：随机地形与泛化，回答"模型是在看地形行动，还是记住了固定路线"——把"真正学会"与"过拟合"区分开。

PART IV · 课程、泛化与实验

第 13 章：随机地形、泛化与过拟合

> 核心问题：**模型是在看地形行动，还是记住了固定路线？**

如果机器人只在**同一条**跑道上训练，它可能不是"学会了走路"，而是"背下了这条跑道的每一寸起伏"——换个没见过的地方，立刻摔得四脚朝天。这就是**过拟合**（overfitting）在强化学习里的形态。本章讲清楚：如何用**程序化随机地形 + 留出地形评估**来区分"真正学会"和"死记硬背"。

13.1 学习目标

完成本章后，你应当能够：

1. 解释程序化随机地形如何生成，以及为什么它能防止"记住路线"。
2. 说明每级 6 条训练地形 + 2 条留出地形的意义。
3. 说明回合级地形变体抽样的作用。
4. 区分训练分布、验证分布与 domain randomization。
5. 解释训练成功率与 holdout 成功率的差距为何是过拟合的信号。
6. 说明随机种子在公平对比中的作用。

13.2 为什么"固定跑道"会骗你

设想一个极端的对照组：机器人永远在同一条平坦、笔直的跑道上训练。训练几千小时后，它的"训练成功率"接近 100%。但这是真的"会走路"吗？

不是。它可能学到了：

1. **记住地形**：这条跑道哪里有个缓坡、哪里是平的，它已经"背"下来了。策略里编码的不是"通用的落脚规则"，而是"对这条特定高度场的查表"。
2. **记住初始条件**：起点位置、朝向、姿态都固定，它只需复现一段固定的动作序列。
3. **过拟合到这条轨迹的动力学**：连每一步的接触时间、身体姿态都精确记忆。

一旦把机器人放到一条**从未见过**的跑道上，这些"记忆"全部失效，它就会暴露出真实的走路能力——很可能约等于零。

> 这就是强化学习版过拟合的本质：**策略记住了训练环境的具体细节，而不是学到任务的一般规律。** 与监督学习"背答案"完全同构，区别在于这里的"答案"是一段地形和一条轨迹。

13.3 程序化随机地形：逼策略学"规则"而非"地图"

本项目的对策是**程序化随机地形**（`terrain.py` 的 `build_terrain`）：地形不是手工设计的固定形状，而是用随机过程**生成**的，每次训练/每个等级都不同。

生成流程（`terrain.py`）：

```python
rng = np.random.default_rng(config.seed)     # 固定种子，可复现
for level, amplitude in enumerate(terrain_amplitude_curriculum):
    for variant in range(terrain_variants_per_level):   # 8 条变体
        noise = rng.normal(0, 1, (ROWS_PER_LANE, HEIGHTFIELD_COLS))
        for _ in range(6):                              # 沿 x 平滑
            noise = (noise + np.roll(noise, 1, 1) + np.roll(noise, -1, 1)) / 3.0
        for _ in range(3):                              # 沿 y 平滑
            noise = (noise + np.roll(noise, 1, 0) + np.roll(noise, -1, 0)) / 3.0
        phase = rng.uniform(0, 2*pi)
        cycles = (2.0 + level) * rng.uniform(0.85, 1.15)
        wave = np.sin(2*pi*cycles*x + phase)[None, :]   # 正弦波
        raw = 0.55 * noise/np.std(noise) + 0.45 * wave  # 噪声 + 正弦混合
        terrain[start:end] = np.clip(0.5 + np.tanh(raw) * (amplitude/vertical_span), 0, 1)
```

解读这套"配方"：

1. **高斯噪声 + 多次邻域平均**：`roll` 平均模拟了"平滑"——让相邻格子高度连续，避免出现物理上不可能的尖刺。
2. **正弦波 + 随机相位/周期**：叠加一个低频起伏，让地形有"山丘和洼地"的结构，周期随等级增加（`2.0 + level`）。
3. **噪声与正弦按 0.55/0.45 混合**：既有随机的局部起伏，又有确定性的整体起伏。
4. **tanh 压缩到振幅**：把任意幅度的原始信号压进 $[-\text{amplitude}, +\text{amplitude}]$，再映射到高度场的 $[0, 1]$。
5. **固定种子**：`default_rng(config.seed)` 保证"同一配置生成同一批地形"，使实验可复现。

关键结论：因为地形是随机的，策略不可能"记住某一条固定路线"。它若想在任何随机地形上都拿到奖励，就必须学会**"读取前方高程 → 调整落脚"的通用规则**，而不是背地图。这就是随机化逼迫泛化的机制。

13.4 训练地形 vs 留出地形：真正公平的考试

即便地形是随机的，还有一个陷阱：**策略可能仍然"记住"了它见过的那些随机地形实例**（过拟合到训练集的特定样本）。所以要像监督学习那样，划分**训练集**和**留出集（holdout）**：

```python
terrain_variants_per_level = 8    # 每级生成 8 条独立地形
terrain_training_variants = 6     # 0~5 号用于训练
# 6~7 号从不进入训练，专用于检验泛化
```

分配逻辑在 `env.py` 的 `_assign_levels_y`：

```python
if self.terrain_split == "train":
    low, high = 0, terrain_training_variants          # 0..5
elif self.terrain_split == "holdout":
    low, high = terrain_training_variants, terrain_variants_per_level  # 6..7
```

训练时，每个回合的 `reset` 会在 **6 条训练地形里随机抽一条**（回合级抽样）：

```python
self.terrain_variants[indices] = torch.randint(low, high, ...)
```

评估时，`evaluate.py` 用 `--terrain-split holdout`，只在 **2 条留出地形**上跑，这些地形**从未出现在训练中**。

> 这 2 条留出地形是"最诚实的考官"：策略从没见过它们，如果在上面也能走好，说明它掌握的是"通用的走路能力"；如果只在训练地形上走得好、留出地形上一塌糊涂，那就是过拟合的铁证。

13.5 回合级抽样：每局都换新地形

为什么要在**回合级**抽样地形，而不是"每个环境固定一条地形练到底"？

因为如果每个环境固定一条地形，256 个环境就会分别**过拟合到各自的 256 条地形**，策略学到的是"256 条地形的合集"，而每一条都记得死死的。回合级抽样则强制：**同一个环境，下一回合可能换到另一条完全不同的地形**，逼策略不能依赖"我熟悉这条地形"的捷径，只能依赖"我看着前方、边看边走"的通用策略。

这本质上是 domain randomization（**域随机化**） 的一种：

- **训练分布**：6 条随机地形 × 随机抽样，代表"策略在训练时看到的世界的多样性"。
- **验证分布（holdout）**：2 条从未见过的地形，代表"策略在测试时面对的新世界的分布"。
- 目标：让训练分布**足够多样**，使得在验证分布上的表现也良好。

13.6 训练成功率 vs holdout 成功率的"差距"

这是本章的"诊断仪表"：

现象	解读
train ≈ 95%，holdout ≈ 85%	良好泛化，小差距是正常噪声
train ≈ 95%，holdout ≈ 40%	**严重过拟合**，策略记住了训练地形
train ≈ 40%，holdout ≈ 40%	还没学会（欠拟合），连训练集都搞不定
train ≈ 95%，holdout ≈ 95%	理想，但需警惕"holdout 也被间接泄露"的可能

据此可以判断：**"训练地形 95%、留出地形 40%"代表严重过拟合，而不是已经掌握 Level 7。** 判断标准不是"训练集多高"，而是"训练与留出的差距有多大"。

为什么不能只看训练成功率？因为训练成功率可以被"记忆"虚高。一个只会背地图的策略，训练成功率可以高达 100%，但它毫无泛化能力。**只有"训练和留出都高、且差距小"，才证明学到的是真本领。**

13.7 随机种子与公平对比

第 0 章讲过 seed 保证可复现。在泛化评估里，seed 还有一层更重要的作用——**公平性**：

1. **地形由 seed 决定**：`terrain.py` 用 `config.seed` 生成地形。两个对比实验如果用不同 seed，就会得到**不同的地形集**，那么"谁更泛化"的结论就混杂了"谁碰到的地形更简单"这个噪声。
2. **评估要固定地形**：评估 Level 6/7 时，`evaluate.py` 会加载 checkpoint 里的 `env` 配置（含 seed），用**同一个种子**重建地形，保证"训练时见到的训练地形"和"评估时重建的地形"一致，从而公平对比 train vs holdout。
3. **结论要跨 seed 稳健**：一个"改进了泛化"的结论，应当至少在 2～3 个不同 seed 下都成立。只在一个 seed 上变好，可能是"这个种子碰巧让留出地形变简单了"。

> 纪律：**评估泛化时，train 和 holdout 必须来自同一套地形配置（同 seed）；对比不同方案时，必须用同 seed；验证结论时，要跨 seed。** 三者缺一，结论就不可信。

13.8 对应代码全景

关注点	文件	关键内容
地形生成	`terrain.py`	`build_terrain`（噪声+正弦+tanh）、`HEIGHTFIELD_COLS`、`ROWS_PER_LANE`
训练/留出划分	`config.py`	`terrain_variants_per_level=8`、`terrain_training_variants=6`
回合级抽样	`env.py`	`_assign_levels_y` 的 `torch.randint(low, high)`
评估	`evaluate.py`	`--terrain-split train/holdout`、确定性推理
可视化	`robot_demo.py`	`--terrain-variant 0..5 训练, 6..7 留出`

评估命令示例：

```powershell
.\.venv\Scripts\python.exe -m water_cup_rl.evaluate checkpoints\terrain_biped_v23\latest.pt --episodes 64
--level 7 --terrain-split holdout --device cuda:0
```

注意 `evaluate.py` 用 `deterministic=True` 推理（取策略均值，不采样），排除动作采样的随机性，让评估结果只反映"策略能力"而非"采样运气"：

```python
with torch.inference_mode():
    action, _, _ = model.act(observation, deterministic=True)
```

13.9 实践：制作 train vs holdout 对比表

任务 1：两组评估

对同一个 checkpoint，分别在训练地形和留出地形上评估 Level 6、7：

```powershell
# 训练地形
.\.venv\Scripts\python.exe -m water_cup_rl.evaluate <ckpt> --episodes 64 --level 6 --terrain-split train
--device cuda:0
.\.venv\Scripts\python.exe -m water_cup_rl.evaluate <ckpt> --episodes 64 --level 7 --terrain-split train
--device cuda:0

# 留出地形
.\.venv\Scripts\python.exe -m water_cup_rl.evaluate <ckpt> --episodes 64 --level 6 --terrain-split holdou
t --device cuda:0
.\.venv\Scripts\python.exe -m water_cup_rl.evaluate <ckpt> --episodes 64 --level 7 --terrain-split holdou
t --device cuda:0
```

把四次结果整理成一张表：

Level	地形	Success	Fall	Timeout	Distance	COT
6	train					
6	holdout					
7	train					
7	holdout					

任务 2：读差距，下结论

根据表回答：

1. train 与 holdout 的成功率差距有多大？
2. 这个差距说明策略是"学会了走路"还是"记住了训练地形"？
3. 如果差距很大，你认为可能是哪一项（地形随机化不足？观测泄露？）导致的？—— 这是结课项目的热身。

任务 3：可视化留出地形

```powershell
.\.venv\Scripts\python.exe -m water_cup_rl.robot_demo --checkpoint <ckpt> --level 7 --terrain-variant 6 -
-seconds 120
```

`--terrain-variant 6` 是留出地形（6~7 号）。观察策略在从未见过的地形上表现如何：是稳健地边看边走，还是磕磕绊绊、频繁摔倒？把视觉观察与任务 1 的数据对照，看"数据"和"眼睛"是否给出同一结论。

13.10 验收标准

- [ ] 能否解释程序化随机地形如何防止"记住路线"？
- [ ] 能否说明 6 训练 + 2 留出的划分意义，以及回合级抽样的作用？
- [ ] 能否区分训练分布、验证分布、domain randomization？
- [ ] 能否判断"train 95% / holdout 40%"代表过拟合而非掌握？
- [ ] 能否说明：为什么"训练与留出的差距"比"训练成功率"更能反映泛化？
- [ ] 能否说明 seed 在公平对比中的三重作用（同配置可复现、同 seed 对比、跨 seed 验证）？

13.11 常见误区

1. **只看训练成功率** —— 会被"背地图"虚高，必须看 holdout 和差距。
2. **以为随机地形 = 一定不拟合** —— 若每条环境固定一条地形练到底，仍会拟合到那 256 条。要配合回合级抽样。
3. **在评估时开随机采样** —— 动作采样噪声会污染评估结果，应用确定性推理。
4. **跨 seed 比较不同方案** —— 地形不同，比较无意义。同 seed 才有可比性。
5. **忽略"holdout 也可能被间接泄露"** —— 如果观测/奖励里泄露了"地形全局特征"，holdout 也可能被"猜"到。要警惕。

13.12 本章小结

- 固定跑道会让策略"背地图"而非"学走路"；程序化随机地形逼它学通用规则。
- 6 训练 + 2 留出地形，留出地形是"从未见过"的最诚实考官。
- 回合级抽样 + domain randomization 扩大训练多样性。
- 泛化的度量 = train 与 holdout 的**差距**，而非训练成功率本身。
- seed 是公平对比的前提：同配置可复现、同 seed 对比、跨 seed 验证。

下一步：进入第 14 章：训练指标与故障诊断，回答"训练卡住时，应该先看什么"——把各类指标变成诊断工具。

PART IV · 课程、泛化与实验

第 14 章：训练指标与故障诊断

> 核心问题：训练卡住时，应该先看什么？

训练没进展、回报不涨、甚至越来越差——这时候新手最容易犯的错误是"乱调超参数"或"凭一段动画下结论"。本章教你一套**系统化的诊断方法**：先看懂每一类指标的含义，再学会从"现象 → 证据 → 假设 → 单变量验证"的链条定位问题。

14.1 学习目标

完成本章后，你应当能够：

1. 逐组解释终端输出的指标（TASK / MOVE / FEET / CTRL / ENERGY）。
2. 解释 policy loss、value loss、entropy、KL 四个训练量各自的含义。
3. 区分四类常见故障：探索不足、奖励漏洞、价值网络失准、课程过难。
4. 用"现象—证据—假设—单变量验证"的方法写一份诊断记录。
5. 论证"不能用总回报单独判断步态"。

14.2 终端指标全景

训练时每个 PPO iteration 打印一次（`train.py` 的 `training_dashboard`），分成五组：

```text
[ITER]  迭代、总步数、课程等级、SPS、ETA、KL
[TASK]  SUCCESS、REACH、FALL、TIMEOUT、RETURN
[MOVE]  DISTANCE、BEST、SPEED、UPRIGHT、LIFE
[FEET]  LOADED-SLIP、FLIGHT、PROLONGED-FLIGHT
[CTRL]  TORQUE、NORMALIZED-TORQUE、ACTION-CHANGE
[ENERGY] MOTOR、SLIP-LOSS、EFFICIENCY、COT、PUSH
```

[ITER] 进度与速度

指标	含义	诊断用途
迭代 / 总步数	训练进度	确认训练在推进
课程等级	当前前沿等级	课程是否在晋级
SPS	每秒环境步数	吞吐量是否正常（异常低说明 GPU 没吃满）

指标	含义	诊断用途
ETA	预计剩余时间	规划时间
KL	近似 KL	策略更新幅度是否失控（见 14.5）

[TASK] 任务结果

指标	含义
SUCCESS	前沿等级成功率（课程晋级用的那个）
REACH	双脚都到终点的比例
FALL	摔倒比例
TIMEOUT	超时比例
RETURN	平均回合回报

关键：SUCCESS 与 REACH 的关系。SUCCESS = REACH 且未摔倒（`success = reached & ~fallen`）。如果 REACH 高但 SUCCESS 低，说明策略"能到终点但途中摔了"——它可能在"冲线式"地前扑，到了终点却倒下了。这本身就是一条诊断线索。

[MOVE] 运动质量

指标	含义
DISTANCE	回合结束时的实际前进距离
BEST	回合中曾达到的最大前进距离
SPEED	平均前进速度
UPRIGHT	竖直程度（1 最竖直）
LIFE	平均存活时间（秒）

`BEST > DISTANCE` 说明机器人"前进过又退回来了"（比如摔倒前达到 4 m，倒地后滚回 3.5 m）。`UPRIGHT` 低说明姿态不稳。

[FEET] 脚步质量

指标	含义
LOADED-SLIP	承重滑移（越低越好）
FLIGHT	双脚腾空时间占比
PROLONGED-FLIGHT	连续腾空（>50ms）占比

这些直接反映"步态是否健康"：SLIP 高 = 在滑行；FLIGHT 高 = 在跳跃。它们比 RETURN 更能揭示步态本质。

[CTRL] 控制代价

指标	含义
TORQUE	平均力矩（N • m）
NORMALIZED-TORQUE	归一化力矩代价
ACTION-CHANGE	动作变化代价

反映"策略是否在暴力用力 / 高频抖动"。

[ENERGY] 能耗

指标	含义
MOTOR	电机机械功率（W）
SLIP-LOSS	滑移耗散功率（W）
EFFICIENCY	能效
COT	运输能耗比（越低越节能）
PUSH	平均推力 + 推力时间占比

`COT`（cost of transport）是衡量"走单位距离花多少能量"的经典指标，越低越节能。`PUSH` 中 Level 0 应为 0（无推力）。

14.3 四个训练量：loss、value、entropy、KL

这些是 TensorBoard 里 `loss/` 标签下的曲线：

量	含义	健康范围/趋势	异常信号
policy loss	裁剪后的策略损失	小幅波动即可，不必单调下降	剧烈震荡或发散
value loss	价值预测的均方误差	应随训练下降	居高不下 = Critic 没学好
entropy	策略熵	早期高、后期缓降	过早趋近 0 = 失去探索
approx KL	新旧策略 KL	通常 < 0.025（target KL）	持续超阈值 = 更新太猛

> 重要提醒：PPO 的 policy loss 不是"越小越好"的普通损失。它依赖当前批次的优势、受裁剪影响，绝对值意义不大，更要看**趋势和稳定性**。熵和 KL 才是"更新是否健康"的温度计。

14.4 四类常见故障的诊断树

这是本章的核心。训练卡住时，按这个顺序排查：

故障 1：探索不足（entropy 崩了）

现象：entropy 快速下降到接近 0，策略过早收敛到一个确定动作，回报停滞在低位。

证据：entropy 曲线早期就贴地；ACTION-CHANGE 趋近 0（动作几乎不变）；机器人重复同一动作。

假设：熵奖励太小、或 `log_std` 被裁到下限、或学习率过大导致过早收敛。

验证：单变量提高 `entropy_coef`（或调大 `log_std` 下限），看 entropy 是否回升、回报是否突破。

故障 2：奖励漏洞（reward hacking）

现象：RETURN 上升，但 SUCCESS、步态质量（SLIP/FLIGHT/COT）没有改善，甚至变差。

证据：RETURN 高 + LOADED-SLIP 高（在滑行）、或 FLIGHT 高（在跳）、或 UPRIGHT 低（在扑）。

假设：某项奖励权重失衡，策略钻了空子（第 11 章的四种局部最优）。

验证：用 `reward_audit.py` 审计奖励排序；可视化观察策略行为；单变量调整可疑奖励权重。

故障 3：价值网络失准（value loss 高）

现象：value loss 居高不下，优势估计噪声大，策略更新方向乱，训练震荡。

证据：value loss 曲线不降；KL 剧烈波动；回报曲线大幅震荡。

假设：Critic 容量不足、学习率不匹配、或回报目标尺度问题。

验证：检查 value loss 是否随训练下降；必要时单独调 Critic 学习率或 `value_coef`。

故障 4：课程过难（晋级太快 / 门槛不对）

现象：晋级到某级后 SUCCESS 骤降、FALL 飙升，然后降级，反复横跳。

证据：SUCCESS 在晋级后立刻跌到 <20%；课程等级在两级间震荡。

假设：晋级门槛（70%）或窗口（1500）设置不当，导致"过早晋级"。

验证：调高门槛、加大窗口、或减小每级难度增量；观察晋级后 SUCCESS 是否平稳。

14.5 诊断记录格式："现象—证据—假设—单变量"

第 0 章定下实验纪律，这里落到"诊断"场景。每次排查卡住，都要写：

```
TEXT
现象：Level 5 训练 200 个 iteration，RETURN 停滞在 25 左右，SUCCESS 约 30%。
证据：TensorBoard 显示 entropy 已从 2.5 降到 0.3；ACTION-CHANGE 趋近 0；
      robot_demo 里机器人几乎在重复"原地小幅摆腿"。
假设：探索不足导致过早收敛到局部最优。
自变量（只改一个）：把 entropy_coef 从 0.002 提高到 0.01。
控制变量：seed=7、num_envs=256、其它超参数不变、训练步数相同。
结果：entropy 回升到 0.8，RETURN 突破 40，SUCCESS 升到 45%。
```

> 结论：支持假设，探索不足是主因。
> 后续：在 entropy 回升的前提下，观察是否引入新的滑移问题。

为什么必须"单变量"？ 因为强化学习训练本身带噪声，如果你同时改了熵、学习率、网络宽度，就无法归因"哪个改动起了作用"。第 0 章已强调，这里再次强调——这是诊断可信度的根基。

14.6 为什么不能只看总回报

评估步态时，应遵循以下原则："不能只用总回报判断步态，必须同时引用成功率、摔倒率、滑移、腾空、能耗和可视化证据。"

原因：**总回报是多个奖励项的加权和，会被奖励漏洞"污染"**。一个滑行的策略，前进奖励高、回报高，但它不是"会走路"。只有同时看：

1. SUCCESS / FALL（是否真的成功、是否常摔）；
2. LOADED-SLIP / FLIGHT（脚步是否踩稳、是否在跳）；
3. COT / MOTOR（是否节能）；
4. UPRIGHT（姿态是否稳）；
5. **可视化**（亲眼确认行为）。

这几条**交叉印证**，才能判断"步态是否健康"。任何一个单一指标都可能被欺骗，组合起来才可信。

14.7 对应代码与工具

工具	用途
`train.py` 终端输出	每 iteration 的 `[ITER]...[ENERGY]` 快照
TensorBoard	`runs/<name>/` 下的历史曲线
`evaluate.py`	批量确定性评估（成功率、各指标均值）
`reward_audit.py`	审计奖励排序是否合理
`robot_demo.py`	可视化回放，眼见为实

TensorBoard 启动：

```powershell
.\.venv\Scripts\python.exe -m tensorboard.main --logdir runs\terrain_biped_v23
```

推荐同时开两个窗口：一个跑训练看终端，一个开 TensorBoard 看曲线，隔一段时间用 `robot_demo.py --live-reload` 可视化最新 checkpoint。

14.8 实践：写一份诊断记录

任务 1：观察一次训练的完整指标

启动一次训练（或用一个已有的 run 目录），观察至少 100 个 iteration 的终端输出，记录以下各项的**变化趋势**：

- entropy、approx KL、value loss（从 TensorBoard）；
- RETURN、SUCCESS、FALL、TIMEOUT；
- LOADED-SLIP、FLIGHT、COT。

任务 2：写"现象—证据—假设—单变量"诊断记录

假设你观察到"RETURN 停滞但 SLIP 很高"，按 14.5 节的格式写一份完整的诊断记录。要求：

1. 现象描述具体（有数字）；
2. 证据来自至少两类来源（曲线 + 终端 + 可视化）；
3. 假设可证伪；
4. 只改一个自变量；
5. 结果与结论清晰。

任务 3：交叉印证训练与可视化

对同一个 checkpoint，先看 `evaluate.py` 的数据（成功率、滑移、COT），再用 `robot_demo.py` 可视化。写一段话，说明"数据"与"眼睛看到的"是否一致，以及你更相信哪个、为什么。

14.9 验收标准

- [ ] 能否逐组说出终端五组指标的含义，并对关键指标（SUCCESS vs REACH、BEST vs DISTANCE、COT）做出诊断解读？
- [ ] 能否说明 policy loss、value loss、entropy、KL 各自的健康趋势和异常信号？
- [ ] 能否区分探索不足、奖励漏洞、价值失准、课程过难四类故障的"现象 + 证据 + 验证"？
- [ ] 能否写出一份符合"现象—证据—假设—单变量"格式的诊断记录？
- [ ] 能否论证"总回报不能单独判断步态"，并列出必须同时引用的其它证据？

14.10 常见误区

1. **看一段动画就下结论** —— 轶事证据不可靠，必须配合批量评估数据。
2. **只看 RETURN** —— 会被奖励漏洞污染，必须交叉看 SUCCESS/SLIP/FLIGHT/COT/UPRIGHT。
3. **同时改多个超参数** —— 无法归因，等于瞎调。
4. **忽视 entropy/KL** —— 这两个才是"训练是否健康"的体温计，比 loss 更重要。
5. **混淆 SUCCESS 与 REACH** —— REACH 高 SUCCESS 低，意味着"到终点但摔了"，是不同的病。

14.11 本章小结

- 终端五组指标（ITER/TASK/MOVE/FEET/CTRL/ENERGY）+ 四个训练量（loss/value/entropy/KL）构成完整仪表盘。
- 四类常见故障各有"现象—证据—假设—验证"的诊断路径。
- 诊断必须走"单变量验证"，否则不可信。
- 步态健康要"成功率 + 摔倒率 + 滑移 + 腾空 + 能耗 + 可视化"交叉印证，绝不能只信总回报。

下一步：进入第 15 章：Checkpoint 续训与迁移，回答"什么时候用 resume，什么时候用 warm-start"，学会安全地保存、续训和迁移模型。

PART IV · 课程、泛化与实验

第 15 章：Checkpoint、续训与迁移

> 核心问题：什么时候用 `--resume`，什么时候用 `--warm-start`？

训练了几十个小时，总不能因为断电就从头再来。Checkpoint 就是"存档"，让你能续训、评估、回放、迁移。但存档里存的东西远比"网络权重"多，续训也远比"重新加载再跑"微妙——尤其当观测维度从 63 变成 78 时，能不能把旧模型"接着用"就是一个需要精心处理的问题。本章讲清楚这一切。

15.1 学习目标

完成本章后，你应当能够：

1. 说出一个 checkpoint 里保存的全部内容。
2. 解释 `--resume` 的完整续训包含哪些状态。
3. 解释观测维度改变后为何要 `--warm-start`，以及它的实现原理。
4. 说明运行目录与 checkpoint 版本的关系。
5. 论证"复制权重 ≠ 恢复优化器"。
6. 独立读取一个 checkpoint 并打印关键信息。

15.2 Checkpoint 里到底存了什么

看 `train.py` 的 `save_checkpoint`：

```python
payload = {
    "model": model.state_dict(),          # 网络参数 (Actor + Critic + log_std + 归一化器 buffer)
    "optimizer": optimizer.state_dict(),  # 优化器状态 (Adam 的动量、二阶矩等)
    "update": update,                     # 当前 PPO update 序号
    "global_step": global_step,           # 全局环境步数
    "config": config_dict(...),           # env + ppo + curriculum 的完整配置
    "curriculum": curriculum.state_dict(),# 课程状态 (当前等级、outcomes、该等级回合数)
}
```

四类东西：

内容	作用	谁需要
模型参数 `model`	Actor/Critic 权重、`log_std`、归一化器的 mean/var	续训、评估、可视化都要

内容	作用	谁需要
优化器状态 `optimizer`	Adam 的动量 `m`、二阶矩 `v`	只有续训需要
训练进度 `update/global_step`	训练到哪一步了	只有续训需要（决定学习率调度、ETA）
课程状态 `curriculum`	当前等级、成功率窗口	只有续训需要
配置 `config`	该次训练的全部超参数	评估/迁移时校验兼容性

注意一个细节：`model.state_dict()` 里已经包含 `RunningNormalizer` 的 `mean`、`var`、`count`（因为它们是 `register_buffer`）。这意味着**观测归一化的统计量也被存档了**——续训时归一化器从上次的状态继续，而不是重新开始累积。

15.3 保存的原子性：为什么要先写 .tmp 再替换

`save_checkpoint` 里有个值得学习的工程细节：

```python
temporary = path.with_suffix(".tmp")   # 先写临时文件
torch.save(payload, temporary)
temporary.replace(path)                # 再原子替换
```

为什么？因为 `torch.save` 写一个大文件需要时间。如果直接写 `latest.pt`，中途断电/崩溃就会留下一个**半截的损坏文件**。先写 `.tmp`、写完再 `replace`（原子操作），保证 `latest.pt` 要么是完整的旧文件、要么是完整的新文件，**永远不会是半成品**。这是"安全保存"的标准做法。

15.4 `--resume`：完整续训

```powershell
.\.venv\Scripts\python.exe -m water_cup_rl.train --resume checkpoints\terrain_biped_v23\latest.pt ...
```

`--resume` 会恢复**全部四类状态**，等价于"从中断的地方继续跑"。看 `train.py` 的恢复逻辑：

```python
model.load_state_dict(checkpoint["model"])        # 恢复参数
optimizer.load_state_dict(checkpoint["optimizer"]) # 恢复优化器动量
curriculum.load_state_dict(checkpoint["curriculum"]) # 恢复课程等级
start_update = int(checkpoint["update"]) + 1       # 从下一个 update 继续
global_step = int(checkpoint["global_step"])       # 恢复全局步数
```

关键点：`--resume` 恢复的是"训练的完整状态"，不是"只把权重拷过来"。 尤其是：

1. **优化器状态**：Adam 的动量 `m` 和二阶矩 `v` 记录了"过去梯度的历史"。如果只拷权重、不拷优化器，Adam 会"失忆"，学习率调度和动量累积都要重新开始，续训的前几百步会异常震荡。**复制权重 ≠ 恢复优化器**——这是本章的核心验收点。
2. **课程状态**：当前等级、成功率窗口一起恢复。否则续训后会从 Level 0 重新爬课程，白白浪费前面的训练。
3. **全局步数**：学习率线性衰减依赖 `global_step`，恢复它才能让学习率从"正确的中间值"继续衰减，而不是重新从最高学习率开始。

15.5 `--resume` 的严格版本校验

`--resume` 不是"随便哪个 checkpoint 都能续"。`train.py` 有版本校验：

```python
checkpoint_env = checkpoint["config"]["env"]
if checkpoint_env.get("backend") != "mujoco-warp" or checkpoint_env.get("objective_version") != 23:
    raise RuntimeError(
        "Only terrain-perception objective v23 checkpoints can be resumed; "
        "v22 checkpoints have a smaller observation vector"
    )
```

原因：v22 的观测是 63 维，v23 是 78 维（多了 15 维高程扫描）。**观测维度变了，网络第一层的输入维度就变了，旧权重无法直接套到新网络上。** 所以 v22 的 checkpoint 不能用 `--resume` 续训 v23。实际续训前，必须检查 checkpoint 的观测维度与目标模型是否一致。

> 一句话：**`--resume` 要求"网络结构、观测定义、环境版本"完全一致。** 只要观测维度变了（哪怕只是加了一维），就必须走 `--warm-start`。

15.6 `--warm-start`：跨观测维度的迁移

当观测从 63 维变到 78 维时，不能直接续训，但旧策略已经学会了走路，丢掉重练太浪费。`--warm-start` 就是为这种"结构变化但知识可迁移"的场景设计的。

```powershell
.\.venv\Scripts\python.exe -m water_cup_rl.train --warm-start checkpoints\terrain_biped_v22\latest.pt --start-level 5 ...
```

核心思路：复制能复制的，新维度从零开始

看 `train.py` 的 `load_policy_warm_start`：

```python
def load_policy_warm_start(model, source):
    target = model.state_dict()
    expandable = {"actor.0.weight", "critic.0.weight"}   # 第一层权重可"扩列"
    normalizer_vectors = {"obs_normalizer.mean", "obs_normalizer.var"}
```

```
    for name, source_value in source.items():
        if name not in target:
            continue
        target_value = target[name]
        if target_value.shape == source_value.shape:
            target[name] = source_value                    # 形状相同：直接复制
        elif name in expandable and source_value.shape[0] == target_value.shape[0] \
             and source_value.shape[1] < target_value.shape[1]:
            target_value.zero_()                           # 新维度权重先清零
            target_value[:, : source_value.shape[1]].copy_(source_value)  # 旧维度复制
        elif name in normalizer_vectors and ...:
            target_value[: source_value.numel()].copy_(source_value)  # 归一化器前缀复制
        else:
            raise RuntimeError("Cannot warm-start parameter ...")
```

三个关键处理：

1. **形状相同 → 直接复制**：比如隐藏层的权重、输出层权重、Critic 的大部分，都没变，直接搬。
2. **第一层权重（`actor.0.weight`、`critic.0.weight`）→ "扩列"**：第一层输入维度从 63 变 78，权重矩阵从 `(512, 63)` 变 `(512, 78)`。旧权重复制到前 63 列，**新增的 15 列（对应高程扫描）初始化为 0**。这样：
   - 旧观测维度（前 63 维）对网络的影响与旧模型**完全一致**；
   - 新高程维度一开始**不贡献任何信号**（权重为 0），初始动作与旧模型一致；
   - 之后由 PPO 学会"利用"这些高程输入（对应权重从 0 慢慢长起来）。
3. **归一化器向量 → "前缀复制"**：`obs_normalizer.mean/var` 从 63 维扩到 78 维，旧的 63 个值复制过去，新 15 维用默认值（mean=0, var=1），等训练时再更新。

warm-start 与 resume 的区别

	`--resume`	`--warm-start`
恢复模型参数	✅ 完全恢复	⚠️ 只恢复能对应的部分，新维度归零
恢复优化器状态	✅	❌ 优化器从头开始
恢复课程状态	✅	❌ 课程从 `--start-level` 重新开始
恢复全局步数	✅	❌ 从 0 开始
适用场景	结构完全一致的续训	观测/结构变化后的"迁移起步"

> 记忆：**`--resume` 是"接着上次跑"，`--warm-start` 是"借旧模型的脑子，开始一段新训练"。** 前者保留一切，后者只保留"能迁移的知识"，优化器和课程都清零重启。

15.7 运行目录与 checkpoint 版本

每次训练用 `--run-name` 生成独立目录（第 0 章）：

```text
checkpoints/<run-name>/latest.pt
checkpoints/<run-name>/update_000025.pt
runs/<run-name>/  (TensorBoard)
```

版本管理要点：

1. **不要覆盖旧 checkpoint**：每次实验用新 `--run-name`，旧结果永远可回溯。
2. **`latest.pt` vs `update_*.pt`**：`latest.pt` 是最新状态（随时可续训）；`update_*.pt` 是历史快照（`checkpoint_every=25` 保存一次），用于回滚或对比不同阶段。
3. **checkpoint 里带 config**：`payload["config"]` 记录了该次训练的完整超参数，评估或迁移时能据此校验兼容性（如 `evaluate.py` 会检查 `objective_version == 23`）。

15.8 实践：读取 checkpoint，打印关键信息

任务 1：写一个 checkpoint 检查器

```python
import torch
from pathlib import Path

ckpt = torch.load("checkpoints/terrain_biped_v23/latest.pt",
                  map_location="cpu", weights_only=False)
print("keys:", list(ckpt.keys()))
print("objective_version:", ckpt["config"]["env"]["objective_version"])
print("backend:", ckpt["config"]["env"]["backend"])
print("global_step:", ckpt["global_step"])
print("update:", ckpt["update"])
print("curriculum level:", ckpt["curriculum"]["level"])
print("num_envs:", ckpt["config"]["ppo"]["num_envs"])
print("gamma:", ckpt["config"]["ppo"]["gamma"])
print("clip_coef:", ckpt["config"]["ppo"]["clip_coef"])
print("model params:", sum(p.numel() for p in ckpt["model"].values()))
print("optimizer has state:", bool(ckpt["optimizer"]["state"]))
```

运行并记录输出。特别确认 `optimizer` 里是否有状态（`state` 非空），以及 `curriculum` 里的等级。

任务 2：验证"复制权重 ≠ 恢复优化器"

思考并写下来：

1. 如果只 `model.load_state_dict(ckpt["model"])` 而不 `optimizer.load_state_dict(...)`，续训会发生什么？
2. Adam 的 `state` 里存的 `exp_avg`（动量）和 `exp_avg_sq`（二阶矩）是什么？为什么丢了它们，续训初期的更新方向会异常？
3. 学习率线性衰减为什么依赖 `global_step`？

任务 3：模拟 warm-start 的"扩列"

构造一个简化场景：旧模型第一层 `(4, 3)`（3 维输入），新模型 `(4, 5)`（5 维输入）。用 `load_policy_warm_start` 的思路，把旧权重复制到新矩阵前 3 列、后 2 列清零。打印结果，确认：

1. 前 3 列的权重与旧模型完全一致；
2. 后 2 列为 0；
3. 如果给一个"前 3 维 = 旧输入、后 2 维 = 任意"的输入，新模型的输出与旧模型一致（因为新维度权重为 0）。

这印证了"warm-start 后初始动作与旧模型一致"的机制。

15.9 验收标准

- [ ] 能否说出 checkpoint 里保存的四类内容，及各自的服务对象？
- [ ] 能否解释 `--resume` 恢复哪些状态，为什么"复制权重 ≠ 恢复优化器"？
- [ ] 能否说明：为什么观测维度一变，就必须用 `--warm-start` 而非 `--resume`？
- [ ] 能否解释 warm-start 里"第一层权重扩列、新维度归零"的原理和目的？
- [ ] 能否区分 `--resume` 与 `--warm-start` 在优化器、课程、步数上的处理差异？
- [ ] 能否独立读取一个 checkpoint 并打印 objective version、global step、curriculum level、PPO 配置？
- [ ] 能否说明"先写 .tmp 再 replace"的原子保存为何重要？

15.10 常见误区

1. **以为 checkpoint 只有权重** —— 还有优化器状态、课程状态、全局步数、完整配置。
2. **用 `--resume` 续训结构变化的模型** —— 观测维度变了会直接报错或静默错位，必须 `--warm-start`。
3. **只拷权重就以为能续训** —— 丢了优化器动量和学习率调度，续训会异常震荡。
4. **warm-start 后以为能继续旧课程等级** —— warm-start 会重置课程，需要用 `--start-level` 显式指定起点。
5. **覆盖旧 checkpoint** —— 新实验不换 `--run-name`，旧结果被覆盖，无法回溯对比。

15.11 本章小结

- Checkpoint = 模型参数 + 优化器状态 + 训练进度 + 课程状态 + 配置，五类缺一不可。
- `--resume` 完整恢复全部状态，要求结构完全一致（同 objective version）。
- 观测维度变化时用 `--warm-start`：旧维度权重复制、新维度归零，优化器与课程重置。
- 原子保存（.tmp → replace）保证 checkpoint 永不损坏。
- 每次实验独立 `--run-name`，是版本管理与可复现的基础。

AFTERWORD

结语：让证据引导下一次实验

从一个观测出发，讲清完整闭环

现在，请从一个观测开始追踪整轮训练：Actor 如何输出动作，环境如何推进物理并返回奖励，轨迹如何进入 rollout 缓冲区，Critic 如何提供价值估计，GAE 如何构造优势，PPO 如何使用旧策略的数据更新参数，课程调度如何安排下一轮任务，checkpoint 又如何保存可继续训练的状态。

能够把这些环节连起来，意味着你已经可以把公式、代码和机器人行为放在同一张地图上。若某处仍然说不清楚，就返回对应章节，用一个更小的例子重建理解。

用对照与证据检验“学会了”

回报更高、训练地形成功率更高、留出地形成功率更高，以及动作更稳定、更节能，回答的是不同的问题。一个可信的实验结论应说明：改变了什么，哪些条件保持一致，收益出现在哪里，哪些失败仍然存在。

综合实践要求你在不直接指定左右脚交替的前提下，提出一个改进未见地形表现的方案，并使用训练与留出评估、运动质量指标和可视化给出结论。即使结果否定了假设，只要对照清楚、过程可复查、解释有边界，也是一份有价值的实验成果。

让新的问题引导下一次实验

你可以重写最小 PPO，把推导重新连接到实现；也可以深入观测、奖励或课程机制，研究一个具体选择如何影响行为。无论选择哪条路，都应让每次修改对应一个可以检验的问题。

当你能够解释一个动作为什么出现，判断一次修改是否有效，并清楚描述它尚未解决的问题，就具备了继续探索的基础。让机器人学会应对变化，也让自己的理解经得起证据的检验。

PRACTICE

综合实践

实验记录与对照原则

每次实验必须记录：

1. 问题：想验证什么？
2. 假设：预计哪个指标会怎样变化？
3. 自变量：本次只修改什么？
4. 控制变量：seed、训练步数、环境数量等保持什么不变？
5. 结果：训练与留出地形分别怎样？
6. 证据：曲线、终端数据和可视化是否一致？
7. 结论：支持还是反驳假设？
8. 后续：下一次实验改变什么？

禁止只根据一小段动画宣布“学会走路”，也不要同时修改多个奖励权重后猜测是哪项产生效果。

综合项目

项目 A：最小 Policy Gradient

不使用机器人环境，完成一个可以稳定学习的小任务。提交代码、回报曲线和对策略梯度的文字解释。

阶段项目 B：手写最小 PPO

使用 Gymnasium 的简单环境，实现独立于本项目训练器的最小 PPO。要求包含 rollout、GAE、clipped objective、value loss 和 entropy。

阶段项目 C：机器人训练诊断

选择一个真实 checkpoint，同时在训练地形与留出地形评估。结合 success、fall、slip、flight、energy 和视频判断策略问题。

综合项目：提高未见地形泛化

在不直接指定左右脚交替的前提下，提出一个改进方案。可以调整地形随机化、观测噪声、课程复习比例或 PPO 超参数，但每次只改变一个主要变量。

最终报告至少包含：

- 基线配置；
- 实验假设；
- 训练预算和随机种子；

- train/holdout Level 6、7 成功率；
- 滑移、腾空、能耗与摔倒率；
- 可视化观察；
- 失败案例；
- 是否真正提高泛化的结论。

WORKSHEET

实验记录模板

实验基本信息

项目	填写内容
实验编号 / 日期	_____
代码版本 / 配置版本	_____
运行名称 / 随机种子	_____
checkpoint / 日志位置	_____
训练预算 / 环境数量	_____

问题、假设与对照

研究问题：_____

预期变化与理由：_____

唯一主要自变量：_____

保持一致的条件：_____

结果与证据

评估条件	基线结果	本次结果	证据位置
train / Level 6	_____	_____	_____
holdout / Level 6	_____	_____	_____
train / Level 7	_____	_____	_____
holdout / Level 7	_____	_____	_____

同时记录：摔倒、滑移、腾空、能耗，以及可视化观察。

结论与下一步

结果是否支持假设：_____

失败案例与解释边界：_____

下一次只改变什么：_____